A Survey of Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge

Sunkwan Kim

kim.sunk@northeastern.edu
Northeastern University

Abstract. Zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs) allow a verifier to check a computation with a proof far shorter than the computation, without learning the prover's private inputs. Over the last decade they have moved from theory to deployed systems, and the number of constructions has grown quickly. This survey organizes the main construction families by their setup model: circuit-specific trusted setup (GGPR, Pinocchio, Groth16), universal and updatable setup (Sonic, PLONK, Marlin), and transparent setup (Fractal, the random-oracle variant of Spartan, and Supersonic with class groups), with recursion and folding (Halo, Nova) built on top. We give a single formal definition with a strict succinctness requirement and explain the modular pipeline behind most schemes, including lookup arguments. We then compare the schemes in normalized tables of asymptotic costs and setup parameters, with their assumptions and the exact counts the papers report, and give a source for every entry. Schemes that are often called zk-SNARKs but miss the strict bound are compared separately. A further section draws together the developments since 2022: small and binary fields with challenges from extension fields, lookup-centric and STARK-based zkVMs, folding in place of recursive verification, the gap between proven and conjectured security of hash-based commitments, and lattice-based folding and commitments. We conclude with open problems in prover efficiency and proof size, post-quantum security, recursion overhead and the security of folding, Fiat–Shamir in practice, and SNARGs from standard assumptions.

Keywords: zk-SNARKs · Zero-Knowledge Proofs · Polynomial Commitment Schemes · Interactive Oracle Proofs · Sum-Check Protocol · GKR Protocol · Arithmetization · Small and Binary Fields · Fiat–Shamir Transform · Trusted Setup · Recursion and Folding · Lookup Arguments · zkVMs · Proximity Gaps · Post-Quantum Security · Lattice-Based Cryptography

Contents

1 Introduction **3**

2 Preliminaries **5**
2.1 Notation and mathematical background 5
2.2 Interactive proofs and arguments 7
2.3 Cryptographic tools 8

3 Defining a zk-SNARK **9**
3.1 Why knowledge assumptions and random oracles? 12

4 Anatomy of a modern zk-SNARK **14**
4.1 Arithmetization 14
4.2 Information-theoretic protocols: PCPs, IOPs and polynomial IOPs 17
4.3 Lookup arguments 19
4.4 Polynomial commitment schemes 20
4.5 Compiling to a non-interactive argument 24

5 Construction families of zk-SNARKs **27**
5.1 Circuit-specific setup — GGPR, Pinocchio, Groth16 27
5.2 Universal and updatable setup — Sonic, PLONK, Marlin 30
5.3 Transparent setup — Fractal, Spartan, Supersonic (DARK) 31
5.4 Zero-knowledge arguments outside the strict definition 34

6 Recursion, accumulation and folding **36**
6.1 Incrementally verifiable computation 36
6.2 Halo and accumulation schemes 37
6.3 Folding: Nova 37
6.4 Generalizations 39

7 Comparison **40**

8 Developments since 2022 **47**

9 Open problems and future directions **49**
9.1 Prover efficiency and proof size 50
9.2 Post-quantum zk-SNARKs 52
9.3 Recursion overhead and the security of folding 53
9.4 Fiat–Shamir in practice 54
9.5 Foundations: SNARGs from standard assumptions 55

10 Conclusion **55**

A Supplementary tables and timeline **73**

1 Introduction

Zero knowledge began as a theoretical notion: Goldwasser, Micali and Rackoff asked how much knowledge a proof must communicate beyond the validity of the statement, and they defined proofs that communicate none [GMR85]. About three decades later, short non-interactive proofs underlie blockchain scaling and deployed private-payment systems, which rely on zero knowledge. This survey is about proofs that combine zero knowledge, succinctness and non-interactivity: the zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK).

Why succinct, non-interactive and zero-knowledge. The original motivation is verifiable computation: a weak client outsources a computation to an untrusted server and checks a short proof instead of redoing it [Tha22, Ch. 1, pp. 119–120]. Each property in the name serves this goal. *Succinctness* makes checking much cheaper than computing. In a zk-SNARK both the proof and the verifier's work are polylogarithmic in the size of the computation, apart from the time to read the public input (§3). *Non-interactivity* means a proof is a single message that anyone can check at any time. It can therefore be stored, for instance permanently on a blockchain [Tha22, §7.2, p. 294], and verified inside another proof, which is the basis of recursive composition [Val08] (§6). *Knowledge soundness* guarantees that a convincing prover actually knows a witness, which matters for statements that are always true (§2.2). *Zero knowledge* lets the witness stay private, as in anonymous payments, where the proof shows that a transaction is valid without revealing which coin it spends; the serial number published with the transaction lets the network reject a second spend of the same coin [BCG+14, abstract, §1.3, pp. 6–8].

Applications. zk-SNARKs are used wherever a verifier must check a computation it cannot or will not redo, or a statement about data it should not see. On a blockchain, a single SNARK can replace a batch of transaction signatures that every node would otherwise store and verify [AST24, §1, p. 1]. A zkVM is "a SNARK that allows the witness-checking procedure to be specified as a computer program written in the assembly language of a specific instruction set architecture" [AST24, abstract]. ZK-rollups deployed by 2024, such as zkSync Era, Polygon zkEVM and Scroll [CET+24, §4, p. 7], rely on SNARKs. Composing proofs sets their trust model: proving the verifier of a FRI-based SNARK inside a pairing-based SNARK such as Groth16 keeps the latter's trusted setup, since "the trust assumption reduces to the weakest component" [CET+24, §3.1, p. 5]. Two of these rollups, Polygon zkEVM and Scroll, used SNARKs "without the zero-knowledge property, solely relying on non-interactivity and succinctness" [CET+24, §3.1, p. 5]. (Polygon zkEVM's Mainnet Beta was sunset on 3 July 2026 [Pol26].) Zero knowledge, by contrast, is the point of private payments and of credentials, whose verifier learns that the holder has a valid credential but nothing about the credential itself [PHGR13, §2.1, p. 2].

Failures in deployment. Deployment has also exposed failures that a security proof does not cover. Of the 141 vulnerabilities Chaliasos et al. collected, 99 are in circuits and 124 break soundness [CET+24, Tables 2–4]. A SNARK variant deployed in Zcash had extra group elements in its structured reference string (SRS) that would have permitted unlimited counterfeiting [Tha22, §17.5, fn. 15, p. 573; CET+24, App. A.1, p. 20]. Later, a zkVM fuzzer found eleven new bugs in RISC Zero, Nexus and Jolt, three of them soundness bugs [HWC26, abstract, §1]. Of 70 real Circom bugs collected, 56 compile in their full codebases, and circuit-analysis tools found only 11 of these (19.6%) [KSHC26, abstract, §III-B, p. 3, §V, p. 4].

From theory to practice. The PCP-based arguments of Kilian and Micali [Kil92; Mic94] (§2.2) remained far from practical. Gennaro, Gentry, Parno and Raykova replaced PCPs with quadratic span and arithmetic programs [GGPR13], and Pinocchio built on them a general-purpose system whose verification was cheaper than native execution for some applications [PHGR13] (§5.1). Groth's 2016 scheme reduced a proof to three group elements, 128 bytes on the BN-256 curve [Gro16; CHM+20, Fig. 1]. Since then, the field has diversified along several axes: reusable universal setups [GKM+18; MBKM19; GWC19; CHM+20], transparent setups with no trapdoor [BBHR18; COS20; Set20; BFS20], recursion and folding [BGH19; BCMS20; KST22], and faster provers (§9.1). Liang et al. classify over 40 zk-SNARKs published since 2013 [LHW+25, abstract].

The problem this survey addresses. With this growth, the term "zk-SNARK" has come to cover schemes with very different guarantees. The papers do not agree on what "succinct" means. Hyrax asks for proofs "sub-linear in the size of the statement and the witness" [WTS+18, §1, p. 1], Aurora calls a scheme with a linear-time verifier "a zkSNARK" [BCR+19, abstract], and Binius classifies as SNARKs protocols "whose verifiers scale merely sublinearly, as opposed to strictly polylogarithmically" [DP25, §1, p. 1]. Setup models are often conflated: "preprocessing" is used both for circuit-specific trusted setups and for deterministic indexing over a universal one [CHM+20, §1, p. 3]. Reported costs use incompatible units, curves, security levels and benchmarks (§7). A reader comparing schemes therefore has to translate each paper into a common language first.

Contributions. This survey does that translation for the main families.

- **One definition** (§3). We define zk-SNARKs for arithmetic circuits with a two-phase setup, a global setup $\mathcal{S}_1$ and a circuit setup $\mathcal{S}_2$, so that circuit-specific, universal and transparent schemes are instances of one syntax (Remark 1, Table 3.1). We classify schemes by strict succinctness (proof size and verifier time polylogarithmic in $|C|$, apart from reading the statement x) and by a rule for when a scheme counts as zero-knowledge. We also explain why every zk-SNARK compared in this survey relies on a knowledge-type assumption, an idealized group model or a random oracle (§3.1).
- **A pipeline view** (§4). We trace how arithmetization, an information-theoretic protocol, a polynomial commitment and the Fiat–Shamir transform fill each part of the definition, and where trust enters, with lookup arguments in §4.3.
- **A taxonomy** (§5). We organize the families by setup model, and treat separately the zero-knowledge arguments that miss the strict bound, each with the reason it misses (§5.4).
- **Recursion and folding** (§6). We treat IVC (§6.1), accumulation and folding as a layer on top of any base argument, and say which of their outputs meet the strict definition.
- **A normalized, sourced comparison** (§7). We express every scheme's setup parameters, proof size, prover and verifier costs as $O_\lambda(\cdot)$ bounds in $|C|$ wherever the sources allow (the O_λ notation hides $\mathrm{poly}(\lambda)$ factors), note when a bound also depends on depth, input or witness size, and cite a source for every cell.
- **A map of recent developments** (§8). We organize the work since 2022 around five questions: small fields and their cost, the design of zkVMs, folding in place of recursive verification, the proven security of hash-based commitments, and post-quantum options that are not hash-based.

- **Open problems** (§9) in five areas: prover efficiency and proof size, post-quantum security, the cost and security of recursion, Fiat–Shamir in practice, and SNARGs from standard assumptions. They include securing the Fiat–Shamir transform against recent attacks, recursion that is proven secure with the concrete hash it uses, and a lattice-based zk-SNARK that is built and analyzed end to end, zero knowledge included.

Related surveys. Thaler's monograph treats the main ideas behind interactive proofs, arguments and zero-knowledge protocols in a unified, accessible way [Tha22, Ch. 1, p. 121]. Nitulescu [Nit20] traces soundness from interactive proofs to arguments of knowledge and presents the modular frameworks behind SNARKs. Liang et al. [LHW+25] systematize zk-SNARKs from theory to practice and study 11 libraries, Sheybani et al. [SAKK25] survey and benchmark open-source frameworks, and Chaliasos et al. [CET+24] systematize real-world implementation vulnerabilities. Two 2026 systematizations overlap most with ours. Guan et al. organize polynomial-IOP-based SNARKs by a finer front-end/back-end split, with lookup arguments and accumulation as integral components, and state a succinctness bound polynomial in $|x|$ and polylogarithmic in the witness without classifying schemes by it [GZL+26, abstract, §1.1, p. 2, App. A.2, p. 12]. Skatharoudis compares hash-based polynomial commitments across their soundness regimes, also attributing every cell to a primary source, but for commitment schemes rather than complete zk-SNARKs and without treating zero knowledge [Ska26, abstract, §1.3, pp. 3–4]. Our survey is narrower and more definitional: it applies one strict definition throughout, separates the schemes that meet it from those that do not, and normalizes the comparison to the circuit size.

Scope. We treat zk-SNARKs at the level of their constraint systems, information-theoretic protocols, commitments and compilers. We do not cover front-end compilers that turn programs into circuits (§4.1 points to [Tha22, Ch. 6]), the engineering of zkVMs beyond their two main designs (§8), or hardware acceleration of provers. Trusted-setup ceremonies appear only as part of the setup model (§5.1). Σ-protocols and other classical zero-knowledge proofs appear only as background (§2.2); arguments that miss only the strict succinctness bound are treated in §5.4.

Organization. Section 2 recalls background, the contributions above map §§3–9, and §10 concludes. Appendix A holds the per-paper counts (Table A.1), the arguments that miss the strict bound (Table A.2) and a timeline (Figure 7).

2 Preliminaries

This section fixes notation and recalls only the background that later sections use. For standard definitions we refer to Thaler's monograph [Tha22] and to Boneh and Shoup [BS23]. Non-interactive definitions are deferred to §3.

2.1 Notation and mathematical background

Notation

The table lists the symbols used across sections; the others are defined where they appear.

Table 2.1: **Notation.**

Symbol	Meaning	Introduced in
λ	security parameter	§3
$\mathrm{negl}(\lambda)$	a negligible function of λ	§3
$\mathrm{poly}(\cdot)$	some fixed polynomial in the listed quantities	§3
$O_\lambda(\cdot)$	big-O that hides $\mathrm{poly}(\lambda)$ factors	§1, §7
$\mathbb{F}$; $\mathbb{K}$	field of the circuit; extension field for challenges in small-field systems	§2.1; §8
C, $\lvert C\rvert$	public arithmetic circuit; its number of gates	§3
N	largest $\lvert C\rvert$ the global setup supports	§3
x, $\lvert x\rvert$; w	public statement and its size; private witness	§3
$\mathcal{R}_C$	relation $\{(x, w) : C(x, w) = 0\}$	§3
π	proof (the single prover message)	§3
$\mathcal{S}_1$, gp; $\mathcal{S}_2$	global setup and the global parameters; circuit setup	§3
$\mathsf{S_p}$, $\mathsf{S_v}$	prover's and verifier's parameters for one circuit	§3
$\mathcal{P}$, $\mathcal{V}$; $\tilde{\mathcal{P}}$, $\mathcal{A}$	prover and verifier; cheating prover, adversary	§2.2, §3
$\mathcal{E}$; Sim	knowledge extractor; zero-knowledge simulator	§2.2, §3
ρ	random oracle $\{0,1\}^* \to \{0,1\}^\lambda$	§2.3, §3
τ	secret evaluation point of a structured reference string	§3, §4.4, §5.1
$\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$	Reed–Solomon code on domain $\mathcal{L}$, degree $< d$	§2.1
R; δ	code rate $d/\lvert\mathcal{L}\rvert$; proximity parameter	§2.1
d	degree bound; depth of a layered circuit	§2.1; §4.2

Fields and polynomial identities

Circuits are defined over a finite field $\mathbb{F}$, either a prime field $\mathbb{F}_p$ or an extension field $\mathbb{F}_{p^e}$. Three of the constraints Thaler lists shape its choice [Tha22, §19.3.1, pp. 619–621]. Soundness errors of at least $1/|\mathbb{F}|$ force a large field. Schemes over discrete-logarithm or pairing groups must use the group's scalar field, a prime field of about 256 bits for bn256 and bls381 [BS23, p. 626] (the scalar field of BLS12-381 has a 255-bit modulus). Provers that run FFTs (§4) need an *FFT-friendly* field, one in which $p - 1$ has many small prime factors; in practice, $2^\ell \mid p - 1$ for a large ℓ [Tha22, p. 621]. Hash-based schemes escape the second constraint and can use binary fields [Tha22, p. 621], such as the $\mathbb{F}_{2^{64}}$ of STARK (scalable transparent argument of knowledge) [BBHR18, §1.3.2, p. 11] and the towers of Binius [DP25, §2.3]; §8 discusses small and binary fields. Most soundness proofs below rest on the Schwartz–Zippel lemma [Sch80], [Zip79]: a nonzero polynomial $g : \mathbb{F}^m \to \mathbb{F}$ of total degree at most d vanishes at a uniform point of S^m, for a finite $S \subseteq \mathbb{F}$, with probability at most $d/|S|$ [Tha22, Lemma 3.3, p. 157]. A verifier can therefore test a claimed polynomial identity at one random point instead of everywhere.

Reed–Solomon codes and proximity

Let $\mathcal{L} \subseteq \mathbb{F}$ be an evaluation domain, usually a multiplicative subgroup or coset of power-of-2 size. The Reed–Solomon code $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ consists of the evaluations on $\mathcal{L}$ of polynomials of degree less than d. Its *rate* is $R = d/|\mathcal{L}|$ [ACFY24, §1, p. 1]; the coding literature writes

ρ, which this survey reserves for the random oracle. Distinct polynomials of degree less than d agree on fewer than d points [Tha22, §10.5.2, p. 388], so two codewords differ on more than a $1 - R$ fraction of $\mathcal{L}$. A function is *δ-far* from the code if it differs from every codeword on more than a δ fraction of $\mathcal{L}$, and *δ-close* otherwise. A *proximity test* accepts codewords and rejects δ-far functions while reading only a few positions, possibly with a prover's help [BBHR18b, §1, p. 14:2].

Three radii matter. Below the *unique-decoding radius* $(1 - R)/2$, at most one codeword is δ-close to a function [Tha22, §10.4.2, p. 374]. Up to the *Johnson radius* $1 - \sqrt{R}$, list decoding (finding all δ-close codewords) is feasible [CS26, §1.1, p. 131]; two 2026 preprints claim it up to capacity, with a fixed slack, over prime fields [BCP+26, Cor. 1.2, p. 3; Jer26, Thm. 1.1, p. 3, for $q = n^{O(1)}$]. Interpolation makes every function $(1 - R)$-close, so *capacity* $1 - R$ is the limit [CS26, §1, p. 129]. A *proximity gap* says that a line $\{f_0 + cf_1 : c \in \mathbb{F}\}$ is either almost entirely δ-close to the code or almost entirely δ-far [CS26, §1, p. 128]. *Correlated agreement* adds that if more than a small fraction of the line is δ-close, then f_0 and f_1 agree with codewords on one common set of $(1 - \delta)|\mathcal{L}|$ points [ACFY25, §1.2, p. 8]. Testing one random combination then covers both functions. Correlated agreement is proven up to the Johnson radius and was conjectured up to capacity [ACFY25, §1.2, p. 8]; the mutual form that WHIR (§4.4) uses is now also proven up to the Johnson radius [BCGM26, Thm. 5, p. 24:5], and §5.3 says what survives beyond it.

2.2 Interactive proofs and arguments

We use the standard notions of interactive proofs, arguments, zero knowledge and knowledge soundness [Tha22, Defs. 3.1–3.2, pp. 149–151, Def. 11.1, pp. 398–399, §7.4, pp. 305–307] and recall only the points the later sections depend on.

Proofs, arguments and public coins. A prover $\mathcal{P}$ holding (x, w) and a probabilistic polynomial-time verifier $\mathcal{V}$ holding x exchange messages until $\mathcal{V}$ accepts or rejects, written $\langle \mathcal{P}(w), \mathcal{V} \rangle(x)$ [GMR85, §2, p. 293; Tha22, §3.1, pp. 148–149]; §3 requires perfect completeness. In a *public-coin* protocol the verifier's messages are just its random coins [Tha22, §3.3, p. 153]. Only such protocols admit the Fiat–Shamir transform (§2.3). Sum-check and GKR (§4) are public coin. The interactive argument behind GGPR, Pinocchio and Groth16 is not: its verifier holds a secret key, so these schemes are made non-interactive with a structured reference string (SRS; Remark 1 of §3, §4.5) instead [Tha22, §17.5.1, p. 566]. Brassard, Chaum and Crépeau [BCC88] introduced the notion later named an *argument*, which requires soundness only against polynomial-time provers [Tha22, Def. 3.2, pp. 150–151]. Succinctness essentially requires an argument rather than a proof [Tha22, §7.2, fn. 2, p. 295]. Kilian [Kil92] has the prover commit to a probabilistically checkable proof (§4.2) with a Merkle tree (§2.3) and open only the queried positions, the first argument whose communication can be smaller than the NP witness [Kil92, abstract, p. 723]. Micali's CS proofs remove the interaction with a random oracle [Mic94, p. 440]. Every scheme in this survey is an argument.

Zero knowledge and extraction. Zero knowledge is defined by simulation [GMR85, §3.2, p. 296]: an efficient simulator Sim that never sees w must produce a view indistinguishable from the verifier's view of a real interaction, perfectly, statistically or computationally [Tha22, Def. 11.1, pp. 398–399]. *Honest-verifier* zero knowledge, which asks this only for the honest $\mathcal{V}$, suffices before Fiat–Shamir, because there the simulator can program the hash [Tha22, pp. 399–400, p. 402, fn. 3]. Soundness alone can be vacuous for statements that are always true, such as "y has a preimage" under a surjective h [Tha22, §7.4, pp. 305–306]. A *proof or argument of knowledge* therefore requires an efficient extractor $\mathcal{E}$ that, given repeated access to any convincing prover $\tilde{\mathcal{P}}$, outputs a valid witness [Tha22,

p. 306]. Interactive extractors usually rewind the prover, as the special soundness of Σ-protocols does [Tha22, §12.2.1, p. 418]; a real verifier cannot rewind [Tha22, fn. 13, pp. 306–307]. Properties 2 and 4 of §3 are the non-interactive versions, and §3.1 explains what knowledge soundness costs in assumptions.

2.3 Cryptographic tools

Commitments

A commitment $c = \mathsf{Com}(m; z)$, computed with randomness z, is *binding* if no one can open it to two different messages and *hiding* if it reveals nothing about m. Each property holds statistically or computationally, and no scheme is statistically both [Tha22, §12.3, pp. 429–430]. Binding is what turns an oracle protocol into an argument, and hiding is what zero knowledge needs; §4 extends the notion to polynomials. Two commitments recur. A *Merkle tree* [Mer89, §6, pp. 227–230] commits to a vector with a collision-resistant hash h; opening one of its n entries reveals it with the $O(\log n)$ sibling hashes on its path to the root [Tha22, §7.3.2.2, pp. 299–300]. The root is binding but not hiding, so zero knowledge needs hiding leaves, such as $h(s_i, o_i)$ with random o_i [BS23, §8.12, p. 346]. Because it needs only a hash and no setup, it is the commitment layer of Kilian's and Micali's arguments and of the hash-based schemes (§5.3, §5.4). A *Pedersen commitment* [Ped91] to $m \in \mathbb{Z}_q$ is $g^m h^z$ for random generators g, h of a group of prime order q. It is perfectly hiding, binding under the discrete-logarithm assumption, transparent because no one knows $\log_g h$, and additively homomorphic, as is its vector form $h^z \prod_i g_i^{u_i}$ [Tha22, §12.3, pp. 432–434, §14.2, p. 470]. It underlies Bulletproofs, Hyrax and Nova (Tables A.1–A.2 in Appendix A).

Bilinear pairings

A *pairing* $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable, bilinear and non-degenerate map between cyclic groups of prime order q, and it is *symmetric* if $\mathbb{G}_1 = \mathbb{G}_2$ [BS23, Def. 15.2, p. 625]. Encodings g^m can be added homomorphically but not multiplied, and a pairing supplies exactly one multiplication: $e(g_1^a, g_2^b) = e(g_1^c, g_2)$ holds exactly when $c = ab$ [Tha22, p. 500]. A verifier can therefore check quadratic relations among hidden values. KZG uses this to check the evaluation identity $f(X) - f(z) = \psi(X)\,(X - z)$ at a secret point [KZG10, §3.2], and linear-PCP SNARKs such as Groth16 use it to check their verification equations [Tha22, §17.5.1, pp. 566–567] (§4, §5.1).

Random oracle model

In the random oracle model (ROM) [BR93] every party, honest or adversarial, may query a uniformly random function ρ, and security is bounded in terms of the number of queries [BS23, §8.10.2, pp. 327–329]. Deployed schemes replace ρ with a concrete hash function such as SHA-3, so for them a ROM proof is only a heuristic [Tha22, p. 237]. The heuristic can fail. Canetti, Goldreich and Halevi construct schemes that are secure in the ROM but insecure under every instantiation [CGH04], although Thaler calls such examples "typically contrived" [Tha22, fn. 2, p. 237]. Remark 2 of §3 fixes how ρ enters the definitions, and §3.1 covers its roles and its price [KRS25].

Fiat–Shamir transform

Take a public-coin protocol with prover messages $a_1, \ldots, a_{r+1}$ and verifier challenges $c_1, \ldots, c_r$. The Fiat–Shamir transform, introduced for a three-move identification scheme [FS86, §3.1, p. 191] and since applied to any public-coin protocol, replaces each challenge by an oracle output,

$$c_i = \rho\big(\mathsf{gp}, \mathsf{S_v}, x, a_1, \ldots, a_i\big), \qquad i = 1, \ldots, r.$$

The proof $\pi = (a_1, \ldots, a_{r+1})$ is a single message, and the verifier checks it by recomputing the challenges [Tha22, §5.2, p. 240; DMWG23, Def. 3, p. 203]. This is *strong* Fiat–Shamir. *Weak* Fiat–Shamir omits the public parameters and the statement from the hash [DMWG23, Def. 3]. A prover can then pick a false x after fixing the transcript, so hashing x is needed for adaptive soundness [Tha22, p. 241]; §9.4 describes the vulnerabilities this has caused in practice.

The result is an argument in the ROM, and it inherits the assumptions of the interactive protocol [Tha22, §5.3, fn. 6, p. 242]. For constant-round protocols with negligible soundness error it is sound against polynomial-time provers [Tha22, p. 242], as Thaler proves for 3 messages [Tha22, Thm. 5.1, p. 247]. The generic loss, though, is exponential in the number of rounds: a non-interactive prover that succeeds with probability ε in time T yields an interactive one that succeeds with probability at least $(\varepsilon/T)^{O(r)}$ for r rounds, because it can retry messages offline [Tha22, pp. 242, 244]. For many-round protocols such as sum-check, a sufficient condition is *round-by-round soundness* [CCH+19]. A false statement starts in a "doomed" state, and whatever the prover sends next, the verifier's random challenge keeps it doomed except with negligible probability; a doomed full transcript is rejected. In the ROM this implies state-restoration soundness, which already suffices there [CCH+19; BCS16]. Sum-check-based protocols have this property, while sequential repetition of a protocol does not [Tha22, pp. 242–243, fn. 7].

3 Defining a zk-SNARK

This section gives one definition of a zk-SNARK that covers the main families of schemes, fixes when a scheme counts as zero-knowledge, and shows in Table 3.1 how representative schemes instantiate it. Section 3.1 explains why every zk-SNARK compared in this survey relies on a knowledge-type assumption, an idealized group model or a random oracle.

Setting. Fix a finite field $\mathbb{F}$. An arithmetic circuit C over $\mathbb{F}$ defines the relation $\mathcal{R}_C = \{(x, w) : C(x, w) = 0\}$, where x is the public statement and w the private witness. Here $C(x, w) = 0$ means that every output wire of C evaluates to 0 on input (x, w), i.e. the circuit is satisfied (some papers write $C(x, w) = 1$ for the same condition). We write $|C|$ for the number of gates of C, λ for the security parameter, and $\mathrm{negl}(\lambda)$ for a negligible function. Adversaries are stateful: an adversary called more than once in an experiment keeps its state between calls.

Definition 1 (zero-knowledge Succinct Non-interactive ARgument of Knowledge)**.** A zk-SNARK for arithmetic circuits is a tuple of efficient algorithms $(\mathcal{S}_1, \mathcal{S}_2, \mathcal{P}, \mathcal{V})$:

- $\mathcal{S}_1(1^\lambda, N) \to \mathsf{gp}$, the *global setup*: it runs once and outputs public parameters gp for all circuits of size at most N;
- $\mathcal{S}_2(\mathsf{gp}, C) \to (\mathsf{S_p}, \mathsf{S_v})$, the *circuit setup*: it preprocesses a circuit C with $|C| \le N$ and outputs public parameters $\mathsf{S_p}$ for the prover and $\mathsf{S_v}$ for the verifier;
- $\mathcal{P}(\mathsf{S_p}, x, w) \to \pi$, the *prover*: it outputs a single proof message π;
- $\mathcal{V}(\mathsf{S_v}, x, \pi) \to \{0, 1\}$, the *verifier*: it accepts or rejects,

satisfying the following properties for every size bound N. In each experiment the circuit C is chosen after gp is seen, and we only consider adversaries that always output a circuit with $|C| \le N$ (as in [CHM+20, §7], where the relation is restricted to circuits of size at most N).

1. Completeness. For every efficient adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c|c} (x,w)\notin\mathcal{R}_C & \mathsf{gp}\leftarrow\mathcal{S}_1(1^\lambda,N)\\ \vee & (C,x,w)\leftarrow\mathcal{A}(\mathsf{gp})\\ \mathcal{V}(\mathsf{S_v},x,\pi)=1 & (\mathsf{S_p},\mathsf{S_v})\leftarrow\mathcal{S}_2(\mathsf{gp},C)\\ & \pi\leftarrow\mathcal{P}(\mathsf{S_p},x,w)\end{array}\right]=1.$$

2. Knowledge soundness. For every efficient adversary $\tilde{\mathcal{P}}$ there exists an efficient extractor $\mathcal{E}$ such that

$$\Pr\left[\begin{array}{c|c} (x,w)\notin\mathcal{R}_C & \mathsf{gp}\leftarrow\mathcal{S}_1(1^\lambda,N)\\ \wedge & C\leftarrow\tilde{\mathcal{P}}(\mathsf{gp};r)\\ \mathcal{V}(\mathsf{S_v},x,\pi)=1 & (\mathsf{S_p},\mathsf{S_v})\leftarrow\mathcal{S}_2(\mathsf{gp},C)\\ & (x,\pi)\leftarrow\tilde{\mathcal{P}}(\mathsf{S_p},\mathsf{S_v})\\ & w\leftarrow\mathcal{E}(\mathsf{gp},\mathsf{S_p},\mathsf{S_v};r)\end{array}\right]\le\mathrm{negl}(\lambda),$$

where r is $\tilde{\mathcal{P}}$'s random coins, which the extractor also receives. Both the circuit and the statement are chosen adaptively: C after gp, and x after $(\mathsf{S_p},\mathsf{S_v})$.

3. Succinctness. The proof size and the verifier's running time are polylogarithmic in the circuit size, for fixed polynomials that do not depend on C:

$$|\pi|=\mathrm{poly}(\lambda,\log|C|),\qquad \mathrm{time}\big(\mathcal{V}(\mathsf{S_v},x,\pi)\big)=\mathrm{poly}(\lambda,|x|,\log|C|).$$

We also require $|\mathsf{S_v}|=\mathrm{poly}(\lambda,|x|,\log|C|)$. The setups may take time $\mathrm{poly}(\lambda,N)$ and $\mathrm{poly}(\lambda,|C|)$, since they run once (*preprocessing*); for the latter bound $\mathcal{S}_2$ has oracle access to gp, as Marlin's indexer has to the SRS [CHM+20, §7]. Adapting the term of [Gro16, §2.2], which requires the whole reference string to have size $\mathrm{poly}(\lambda)$, we call the scheme *fully succinct* if all parameters generated with fresh randomness have size $\mathrm{poly}(\lambda)$: $|\mathsf{gp}|=\mathrm{poly}(\lambda)$ and $\mathcal{S}_2$ is deterministic.

4. Zero knowledge. There is an efficient simulator $\mathsf{Sim}=(\mathsf{Sim}.\mathcal{S}_1,\mathsf{Sim}.\mathcal{S}_2,\mathsf{Sim}.\mathcal{P})$ such that for every efficient adversary $\mathcal{A}$,

$$\left|\Pr\left[\mathcal{A}^{\mathcal{O}_{\mathcal{P}}}(\mathsf{S_p},\mathsf{S_v})=1\;\middle|\;\begin{array}{c}\mathsf{gp}\leftarrow\mathcal{S}_1(1^\lambda,N)\\ C\leftarrow\mathcal{A}(\mathsf{gp})\\ (\mathsf{S_p},\mathsf{S_v})\leftarrow\mathcal{S}_2(\mathsf{gp},C)\end{array}\right]\right.$$
$$\left.-\Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(\mathsf{S_p},\mathsf{S_v})=1\;\middle|\;\begin{array}{c}(\mathsf{gp},\mathsf{td})\leftarrow\mathsf{Sim}.\mathcal{S}_1(1^\lambda,N)\\ C\leftarrow\mathcal{A}(\mathsf{gp})\\ (\mathsf{S_p},\mathsf{S_v},\mathsf{td}_C)\leftarrow\mathsf{Sim}.\mathcal{S}_2(\mathsf{gp},\mathsf{td},C)\end{array}\right]\right|\le\mathrm{negl}(\lambda),$$

where both oracles take a query (x,w) and answer $\perp$ if $(x,w)\notin\mathcal{R}_C$. Otherwise the real oracle $\mathcal{O}_{\mathcal{P}}$ returns $\mathcal{P}(\mathsf{S_p},x,w)$, and the simulated oracle $\mathcal{O}_{\mathsf{Sim}}$ returns $\mathsf{Sim}.\mathcal{P}(\mathsf{td}_C,x)$. So the adversary may ask for proofs of as many statements as it likes, each chosen after seeing the earlier proofs (*multi-statement* zero knowledge). The simulator never sees w; td and td_C are simulation trapdoors. Zero knowledge is *perfect* if the difference is 0, and *statistical* if the bound holds against unbounded $\mathcal{A}$ that make polynomially many oracle queries.

Remark 1 (setup models). The three kinds of setup are three ways of filling $\mathcal{S}_1$ and $\mathcal{S}_2$. In each case the experiments above give the right adaptivity.

- *Circuit-specific (trusted)*: $\mathcal{S}_1$ outputs nothing (gp is empty), and $\mathcal{S}_2$ uses secret randomness for each circuit, which must be discarded after setup. Choosing C "after gp" then just means C is fixed before the circuit's parameters (e.g. GGPR, Pinocchio, Groth16). Letting the adversary choose C is stronger than the definition in [Gro16, §2.2], where the relation comes from a benign relation generator; but Groth's definition also gives the adversary an auxiliary input, which the definition above omits, so neither definition implies the other [Gro16, §2.2, pp. 8–9].

- *Universal (trusted, often updatable)*: $\mathcal{S}_1$ uses secret randomness once, and $\mathcal{S}_2$ is deterministic, so anyone can preprocess a new circuit without a new trusted setup (e.g. Sonic, PLONK, Marlin; in Marlin, $\mathcal{S}_2$ is the indexer).
- *Transparent*: neither $\mathcal{S}_1$ nor $\mathcal{S}_2$ uses secret randomness; they are deterministic apart from public random coins (e.g. STARK, Fractal, Spartan except $\text{Spartan}_{\text{KE}}$, Supersonic with class groups).

A *common reference string* (CRS) is any reference string that a setup outputs. A *structured reference string* (SRS) is one with algebraic structure, such as powers of a secret, that only secret randomness can produce. We keep each paper's own term, so Groth16's circuit-specific CRS and PLONK's universal SRS are both structured in this sense; a transparent setup needs no SRS.

Remark 2 (random oracle model). Following [COS20, §9], for schemes made non-interactive by the Fiat–Shamir transform, a uniformly random function $\rho : \{0,1\}^* \to \{0,1\}^\lambda$ is sampled first in each experiment, and $\mathcal{S}_1, \mathcal{S}_2, \mathcal{P}, \mathcal{V}$ and every adversary get oracle access to ρ. In knowledge soundness, $\tilde{\mathcal{P}}$ makes at most $\text{poly}(\lambda)$ queries, and the extractor also receives the list of those queries and their answers ([COS20, §9] fixes one extractor for all such provers, with black-box access to $\tilde{\mathcal{P}}$, so it sees those queries). In zero knowledge, the simulator answers the adversary's queries to ρ itself, so it may program ρ, in addition to or instead of using td and td_C.

Remark 3 (simulation extractability). Knowledge soundness concerns a single, isolated prover. An attacker who sees other parties' proofs may instead try to *maul* one into a proof of a different statement without knowing its witness [FR26, §1, pp. 35–36]. *Simulation extractability* requires extraction "even after seeing simulated proofs for potentially false statements" [FR26, abstract, p. 35]; trivial forgeries, such as returning a received proof, do not count [FR26, §3, p. 48]. Sahai's simulation soundness was extended to this notion by De Santis et al. [FR26, §1, p. 36]. The definition above does not require it, but it matters in practice: Nova's proofs of incrementally verifiable computation (IVC, §6.1), for example, are malleable (§6.3). Section 9.4 returns to simulation extractability for sum-check-based schemes.

The definition adapts [CHM+20] for the two-phase setup, i.e. a generator plus an indexer, and the adaptive choice of the circuit; [Gro16] for the extractor that receives the prover's random coins, the simulation trapdoor, and the term *fully succinct*; [GKM+18] for zero knowledge against many adaptively chosen statements; and [BCCT12, §4.2, Defs. 4.1–4.2] for the notion of a SNARK and a succinctness bound that grows only polylogarithmically with the size of the computation. Non-interactivity is built into the syntax: there is one prover message. A Succinct Non-interactive ARGument (SNARG) is the weaker notion that drops zero knowledge and weakens knowledge soundness to soundness. We do not state soundness separately, because knowledge soundness implies it: for a false statement x no w satisfies $(x, w) \in \mathcal{R}_C$, so the event bounded in Property 2 reduces to $\mathcal{V}$ accepting, which therefore has negligible probability.

When a scheme counts as zero-knowledge. Strict succinctness separates the schemes of Tables 7.1 and A.2 (§5.4); a second rule decides which schemes count as zero-knowledge. We count a scheme as zero-knowledge if its paper specifies a zero-knowledge version, of the scheme itself or of a variant, and either proves zero knowledge or derives it from a named compiler or technique, such as blinding with random polynomials or a cited zero-knowledge transformation. If the paper only says that zero knowledge can be added by unspecified standard means, or does not treat it, we call the scheme *not zero-knowledge as presented*, even when zero knowledge could plausibly be added (§4.5 describes how). Such schemes are still discussed, for example as polynomial commitments (§4.4) or in §9, but have no rows

Table 3.1: How representative zk-SNARKs instantiate the definition. Each scheme fills the global setup $\mathcal{S}_1$ and the circuit setup $\mathcal{S}_2$ differently (Remark 1). Schemes made non-interactive by Fiat–Shamir are analyzed in the random oracle model (Remark 2). Sizes of the randomly generated parameters are in Table 7.1, column "Setup params".

Scheme	Setup model	$\mathcal{S}_1$ (global setup)	$\mathcal{S}_2$ (circuit setup)	Random oracle (Remark 2)	Source
Groth16	circuit-specific, trusted	empty	trusted CRS for C from secret randomness, whose simulation trapdoor must stay secret	no: non-interactive by construction	[Gro16, §2.2, §3.1]
PLONK	universal, updatable, trusted	KZG SRS of powers of a secret τ, for all circuits up to the size bound	deterministic preprocessing: commitments to the selector and permutation polynomials	yes: Fiat–Shamir	[GWC19, §1, §8]
Marlin	universal, updatable, trusted	universal SRS for size bound N	deterministic indexer output (index proving and verifying keys)	yes: Fiat–Shamir	[CHM+20, Fig. 1, §1.1, §7]
Fractal	transparent	none beyond the random oracle	deterministic holographic preprocessing of C	yes: preprocessing and Fiat–Shamir in the ROM	[COS20, §2.2, Thm. 3]
Spartan	transparent (knowledge-of-exponent variant $\text{Spartan}_{\text{KE}}$, §5.3: universal, trusted)	public parameters of the chosen polynomial commitment	deterministic computation commitment to C, with no trusted setup or secret trapdoor	yes: Fiat–Shamir	[Set20, §1, §6, Fig. 2]
Supersonic (DARK)	transparent with class groups (trusted with RSA)	group description and a generator, constant size	deterministic preprocessing of the underlying polynomial IOP	yes: Fiat–Shamir	[BFS20, abstract, §1, §4.3]

in Tables 7.1 and A.2. Plonky2, a PLONK-plus-FRI system (§6.4; Table A.2 in Appendix A), is an exception: its whitepaper describes how it blinds the prover's polynomials, but refers to a blog post for the proof, makes FRI zero-knowledge by "standard techniques", and leaves the zero knowledge of FRI openings unfinished [Pol22, §3.6 and fn. 6, pp. 8–9]. We keep it because it was the deployed instance of the hash-based, cycle-free recursion route of §5.3 and §6.4, and its row in Table A.2 states this caveat.

3.1 Why knowledge assumptions and random oracles?

The definition above does not say which assumptions a zk-SNARK may rest on, and in practice every zk-SNARK compared in this survey (Tables 7.1 and A.2) relies on a knowledge-type assumption, an idealized group model or a random oracle. This reflects a barrier identified by Gentry and Wichs [GW11].

The barrier. A cryptographic assumption is *falsifiable* if it can be phrased as a game between an adversary and an efficient challenger that can decide whether the adversary has

won. Gentry and Wichs show that, assuming some NP language has a sub-exponentially hard subset-membership problem, no SNARG for NP can be proven adaptively sound in the standard (CRS) model by a black-box reduction to any falsifiable assumption, unless that assumption is already false [GW11, Thm. 5.1, Cor. 5.2]. The reason is that soundness of a succinct argument is itself not falsifiable: to decide whether an adversary has proven a false statement, the challenger would have to decide whether $x \notin L$ [GW11, §1.1]. Moreover, because the proof is short, an inefficient adversary can pair false statements with fake proofs that no efficient reduction can tell apart from honestly generated ones [GW11, Lemma 4.1]. Since knowledge soundness implies soundness, the barrier extends to zk-SNARKs: any standard-model security proof of a zk-SNARK for circuit satisfiability must use a non-falsifiable assumption, a non-black-box reduction, or a reduction that runs long enough to decide the language. The subexponential-hardness SNARGs of §9.5 take the last route.

Knowledge assumptions. A knowledge assumption states that any efficient algorithm producing a certain kind of output must "know" a corresponding secret, in the sense that an efficient extractor can recover it. For instance, the q-power knowledge-of-exponent assumption (q-PKE) of GGPR and Pinocchio says that whoever outputs two encodings related by a secret factor α must know a linear combination of reference-string elements that explains them [GGPR13, §5.1]. Such assumptions are not (obviously) falsifiable, because they assert the existence of an extractor for every adversary [GW11, §2.2]. They are nonetheless essentially unavoidable: together with a suitable private information retrieval (PIR) scheme, extractable collision-resistant hashing (ECRH) suffices for (designated-verifier) SNARKs, and conversely SNARKs together with standard collision-resistant hashing imply a variant of it [BCCT12, §1.2, Thm. 1, §7, Prop. 7.1].

Pairing-based zk-SNARKs therefore take one of two routes. GGPR and Pinocchio assume q-PKE directly [GGPR13; PHGR13]. Groth16 is instead proven in the generic group model [Gro16], and Sonic, PLONK and Marlin in the algebraic group model [MBKM19; GWC19; CHM+20]. In these idealized models the adversary's group operations can be traced, so extraction is built in. In the algebraic group model, for instance, an adversary must give each group element's representation in terms of the SRS [GWC19, §2.2]. Sonic, PLONK and Marlin are also made non-interactive by Fiat–Shamir, so their analysis combines the algebraic group model with the random oracle model; Marlin's polynomial commitment can also be realized in the plain model under concrete knowledge assumptions [CHM+20, App. B.2]. Nor can black-box extraction replace these assumptions in the standard model: under the discrete-logarithm assumption, a scheme with a black-box extractor needs proofs of size $\Omega(|w|)$ for some NP relation. Fiat–Shamir arguments escape this bound because their extractor sees the prover's oracle queries [CGKS23, Thm. 1, Cor. 2, §1].

Random oracles. The random oracle model plays three roles. First, it makes public-coin protocols non-interactive through the Fiat–Shamir transform (§2.3), as in Micali's CS proofs [Mic94] and every scheme built from a polynomial IOP (PIOP, §4.2). Second, it provides extraction without a knowledge assumption: the extractor observes the prover's oracle queries and reconstructs the string it committed to [Val08, §4.3]. Third, it lets the zero-knowledge simulator program the oracle instead of using a trapdoor (Remark 2). Gentry and Wichs' barrier concerns only the standard model, and in the random oracle model falsifiable assumptions suffice. Hash-based schemes such as STARK, Aurora and Fractal need only the oracle itself, and PLONK has been proven knowledge-sound in the random oracle model from two falsifiable assumptions, the adaptive rational strong Diffie–Hellman assumption (ARSDH) and a related split form, SplitRSDH [LPS25] (§5.2).

The price. The random-oracle heuristic (§2.3) can fail: there is a practical attack on the Fiat–Shamir transform of a standard GKR-based argument [KRS25] (§4.2), and Dao et al. found 36 implementations across 12 proof systems that hash too little [DMWG23] (§9.4). Recursion also forces the heuristic step, since a verifier that queries the oracle must compute the hash inside the circuit [Val08, Remark 2; BCMS20, §1.1], and §6.1 shows that this is hard to avoid. In short, zk-SNARKs pay at least one of two costs: non-falsifiable assumptions or an idealized group model, or falsifiable assumptions in a heuristic random oracle model.

4 Anatomy of a modern zk-SNARK

Most zk-SNARKs in this survey follow one modular recipe: design a public-coin polynomial IOP for circuit or R1CS satisfiability, replace each polynomial the prover sends by a polynomial commitment, and remove interaction with the Fiat–Shamir transform. Every SNARK in Thaler's monograph follows it, "with the lone exception of those based on linear PCPs" [Tha22, §10.2, p. 360]. With arithmetization in front, this gives the pipeline of Figure 1, each stage filling a slot of the definition in §3. Arithmetization fixes the circuit C and hence the relation $\mathcal{R}_C$ (§4.1); the information-theoretic protocol gives soundness with no setup or cryptographic assumption (§4.2); lookup arguments run alongside the constraint system (§4.3). The commitment scheme's setup is the global setup $\mathcal{S}_1$, whose gp ranges from a structured reference string to only a hash function (§4.4). Preprocessing (indexing) is the circuit setup $\mathcal{S}_2$: its commitments to polynomials that encode C, placed in $\mathsf{S_v}$, let the verifier run in time sublinear (often polylogarithmic) in $|C|$ without reading C (§4.2, §4.5). Fiat–Shamir brings in the random oracle ρ of Remark 2 (§4.5). The linear-PCP schemes (GGPR, Pinocchio, Groth16) and, in part, the GKR-based schemes take other routes (end of §4.5).

4.1 Arithmetization

The first stage re-expresses the computation as polynomial constraints over $\mathbb{F}$; compiling high-level programs into circuits (the "front end") is out of scope [Tha22, Ch. 6]. The format affects prover cost and which protocols of §4.2 apply but, for SNARKs built from polynomial commitments, not the setup model, which the commitment scheme alone fixes (§4.5).

R1CS and QAPs. A rank-1 constraint system consists of matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{m \times n}$ and is satisfied by a vector z with $z_1 = 1$ such that $(\mathbf{A}z) \circ (\mathbf{B}z) = \mathbf{C}z$, where $\circ$ is the entrywise product [Tha22, §8.4, Eq. (8.3), p. 329]; the statement x and the witness w are entries of z. An arithmetic circuit converts into an R1CS instance of size proportional to $|C|$, with at most fan-in many nonzero entries per row [Tha22, §8.4.1, p. 330].

A quadratic arithmetic program (QAP) is the same object in polynomial form [Tha22, §17.4, fn. 7, p. 560]. Fix distinct points $\theta_1, \ldots, \theta_m \in \mathbb{F}$, one per constraint, interpolate each column of the R1CS matrices, and let the target polynomial t vanish on these points:

$$v_k(\theta_i) = \mathbf{A}_{i,k}, \quad w_k(\theta_i) = \mathbf{B}_{i,k}, \quad y_k(\theta_i) = \mathbf{C}_{i,k}, \qquad t(X) = \prod_{i=1}^{m} (X - \theta_i).$$

Then z satisfies the R1CS instance if and only if

$$t(X) \;\Big|\; \Big(\sum_k z_k\, v_k(X)\Big)\Big(\sum_k z_k\, w_k(X)\Big) - \sum_k z_k\, y_k(X)$$

[Tha22, §17.4, Eq. (17.9), pp. 560–562; GGPR13, Def. 11, §7.1], and the prover's evidence is the quotient $h(X)$. Addition gates are free, absorbed into the linear combinations [Gro16,

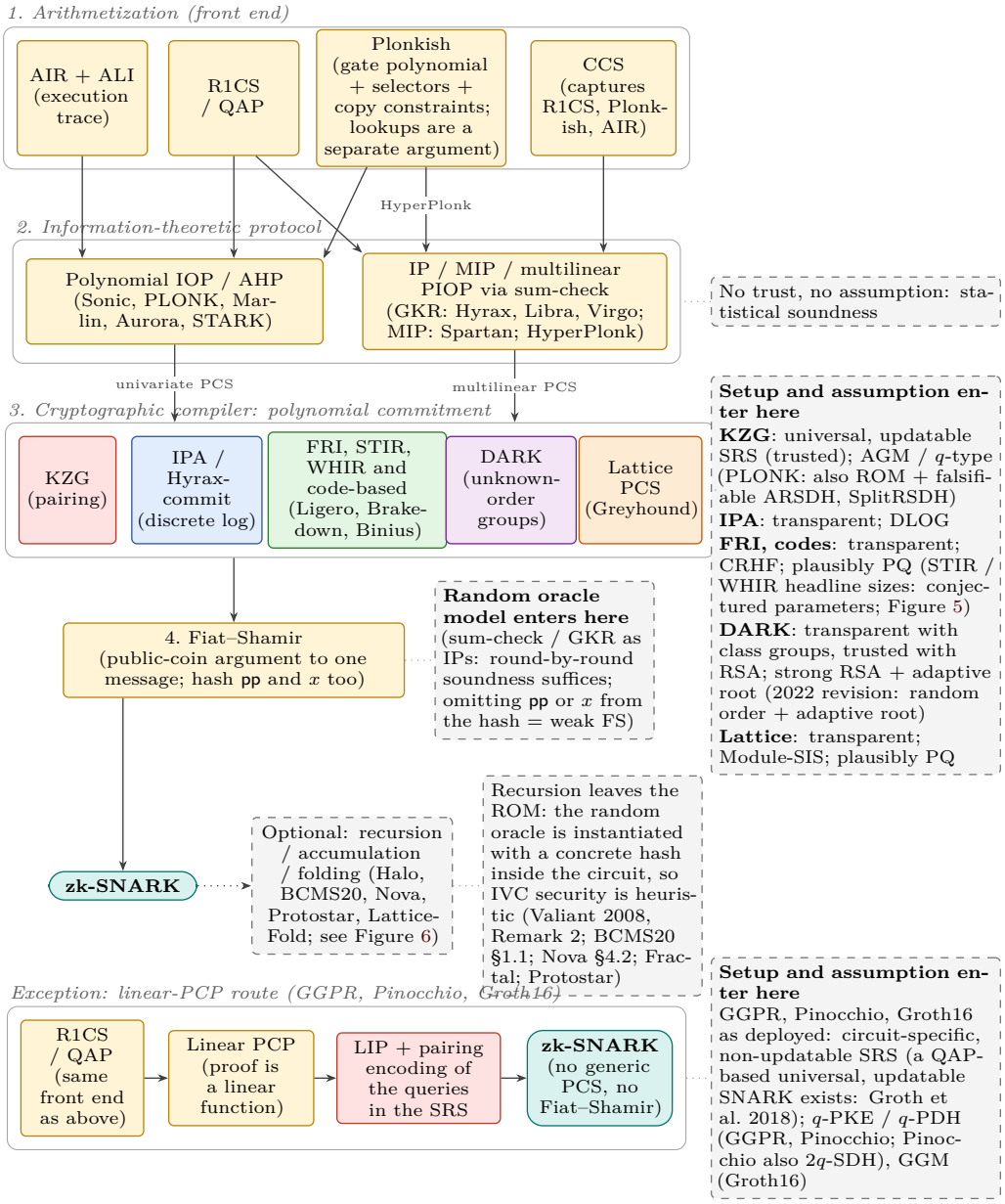


Figure 1: The modular zk-SNARK pipeline and where trust enters. The information-theoretic protocol (a polynomial IOP, or an IP/MIP run through sum-check) is statistically sound with no cryptographic assumption; the setup and the hardness assumption enter with the polynomial commitment, and the random oracle with Fiat–Shamir. ALI is STARK's algebraic linking IOP [BBHR18, §2.3]. The linear-PCP route (bottom) is the exception: its queries are fixed in a circuit-specific SRS and answered "in the exponent", so it uses neither a generic commitment scheme nor Fiat–Shamir. Colours (also in Figures 3 and 4; Figure 6 has its own key): red = pairing-based (trusted SRS), blue = discrete-log (transparent, not post-quantum), green = hash-based (transparent, plausibly post-quantum), purple = groups of unknown order, orange = lattice-based.

§2.3]; computing h takes interpolation and polynomial division, which is why QAP-based provers are quasi-linear (Table 7.1).

R1CS, or its QAP form, is the constraint system of GGPR, Pinocchio, Groth16, Marlin, Fractal, Spartan and Aurora (Table A.1).

Plonkish. PLONK's constraint system assigns gate i a left, right and output wire a_i, b_i, c_i and five selector values, and requires $(q_L)_i z_{a_i} + (q_R)_i z_{b_i} + (q_O)_i z_{c_i} + (q_M)_i z_{a_i} z_{b_i} + (q_C)_i = 0$ [GWC19, §6, p. 23]; the wiring is enforced separately by *copy constraints*, checked with a permutation argument [GWC19, §5.2, §7, step 2, p. 24]. "Plonkish" arithmetization generalizes the gate equation to higher-degree gates, up to degree 9 in Zcash's Orchard protocol [Tha22, §19.3.3, p. 627]. Formally, a Plonkish structure fixes a polynomial g in t variables with q monomials of total degree at most d, selectors $s \in \mathbb{F}^e$, and index vectors $T_0, \ldots, T_{m-1} \in \{0, \ldots, n+e-1\}^t$, and a witness w satisfies an instance x if

$$g\big(z[T_i[1]], \ldots, z[T_i[t]]\big) = 0 \quad \text{for all } i \in \{0, \ldots, m-1\}, \qquad z = (w, x, s)$$

[STW23, Def. 2.3, pp. 4–5]. Copy constraints disappear in this form, because the assignment is deduplicated [STW23, Remark 3, p. 5].

Copy constraints by a grand product. PLONK itself keeps copy constraints and checks them with a grand product. It interpolates the left, right and output values of row i at ω^i into wire polynomials f_L, f_R, f_O, where ω generates a multiplicative subgroup H of size n [GWC19, §7, step 1, p. 24]. List the $3n$ cells as $f_{(1)}, \ldots, f_{(3n)}$ and let σ cycle through the cells of each wire; the wiring holds exactly when $f_{(\ell)} = f_{(\sigma(\ell))}$ for every ℓ [GWC19, §5.2, p. 22]. For random verifier challenges $\beta, \gamma \in \mathbb{F}$, each cell is tagged with its position and with its image under σ:

$$\prod_{\ell=1}^{3n} \big(f_{(\ell)} + \beta\ell + \gamma\big) \;=\; \prod_{\ell=1}^{3n} \big(f_{(\ell)} + \beta\sigma(\ell) + \gamma\big).$$

A violated wiring passes with probability at most $3n/|\mathbb{F}|$, by unique factorization of both sides in β, γ [GWC19, Claim A.1, p. 34]. The prover commits to an accumulator U (PLONK's Z) holding the running ratio of the two products, and the verifier checks two identities on H: $L_1(X)(U(X) - 1) = 0$, where L_1 is the Lagrange polynomial for ω, so the ratio starts at 1, and a step identity relating $U(\omega X)$ to $U(X)$ and the tagged wire polynomials, so one pass around H returns it to 1 and forces the full products to be equal [GWC19, §5, pp. 19–21, §5.1, pp. 21–22].

AIR. An algebraic intermediate representation (AIR) is the format of STARK [BBHR18]. In the simplified form of SuperSpartan, it applies one constraint polynomial g in t variables (t even) to every pair of adjacent rows of an execution trace $z_0, \ldots, z_m \in \mathbb{F}^{t/2}$, each row holding a machine's registers; the public input x supplies the first and last rows and the witness the rows in between. It is satisfied if

$$g(z_{i-1}, z_i) = 0 \quad \text{for all } i \in \{1, \ldots, m\}$$

[STW23, Def. 2.4, pp. 6–7]. Because the same constraint applies at every step, AIR suits uniform, repeated computations, which is why STARK's succinctness is claimed for "a uniform computation" [BFS20, §1, p. 2] (Table A.2).

CCS. Customizable constraint systems unify the three formats. A CCS instance is satisfied if $\sum_{i=0}^{q-1} c_i \cdot \bigcirc_{j \in S_i} M_j z = \mathbf{0}$, for sparse matrices M_j, multisets S_i of size at most d, constants c_i, and $\bigcirc$ the Hadamard product [STW23, Def. 2.2, p. 4]. CCS captures "R1CS, Plonkish, and AIR without overheads" and is "not tied to any particular proof system" [STW23, abstract]; HyperNova folds CCS instances (§6.4).

Native arithmetic beyond one prime field. Two costs of arithmetization do not depend on the format: the choice of field and in-circuit hashing. A fixed prime field is costly when the computation lives in another field (§2.1 gives the other constraints on its choice): secp256k1 ECDSA verification over an FFT-friendly field gives a circuit about 25× larger than over its native field [BFK+24, §1, p. 277], an overhead that field-agnostic commitments (§4.4) remove for other prime fields. Univariate STARKs need a large power-of-two subgroup of $\mathbb{F}_p^*$, a condition on $p-1$; Circle STARKs move it to $p+1$ by working over the circle curve $x^2+y^2=1$, which makes the Mersenne prime $2^{31}-1$ usable [HLP24, Def. 1, p. 3, Lemma 1, p. 4, §3.1, p. 5].

Hash functions inside circuits. Merkle commitments, Fiat–Shamir challenges and recursion all hash inside the circuit, where cost is counted in native-field multiplications. Poseidon, a sponge designed for this cost model, needs about two orders of magnitude fewer R1CS constraints than SHA-256 for a Merkle membership proof [GKRRS21, §2.1–2.2, p. 5, §6.3, Table 4, pp. 14–15].

4.2 Information-theoretic protocols: PCPs, IOPs and polynomial IOPs

From PCPs to IOPs. The oldest route to succinctness is the probabilistically checkable proof (PCP), which the verifier spot-checks in a few positions; Kilian's Merkle commitment and Micali's random oracle (§2.2) turn it into an argument [Kil92; Mic94]. Valiant proved this compilation knowledge-sound in the random oracle model, for a PCP that is witness-extractable [Val08, §4; Tha22, §9.2.1, pp. 342–344]. Its weak point is the prover, who must write down the entire PCP. Interactive oracle proofs (IOPs) [BCS16], introduced independently as probabilistically checkable interactive proofs [RRR16], generalize PCPs and interactive proofs: over several rounds, the verifier only queries the prover's messages [Tha22, §10.1, p. 358]. Merkle-hashing each message and applying Fiat–Shamir compiles any public-coin IOP into a non-interactive argument in the random oracle model, "in a manner entirely analogous to the Kilian-Micali transformation" [Tha22, §10.1, p. 358]; STARK, Aurora and Fractal are compiled this way.

Polynomial IOPs. In a polynomial IOP (PIOP), some prover messages are polynomials of bounded degree, and the verifier learns only their evaluations at points of its choice [Tha22, §10.2, p. 359]. Bünz, Fisch and Szepieniec defined the notion and view "polynomial commitment schemes as a compiler for Polynomial IOPs" [BFS20, §1.1, p. 3]; Marlin's algebraic holographic proofs (AHPs) are "essentially equivalent" [BFS20, §1.2, p. 6], and PLONK's polynomial protocols [GWC19, §4, Def. 4.1] play the same role. Soundness at this layer is statistical [BFS20, §1.1, p. 3].

Two styles dominate. *Univariate* PIOPs (PLONK, Marlin, Aurora, STARK) encode vectors as evaluations over a set H and prove that a constraint polynomial vanishes on H by its quotient by the vanishing polynomial Z_H, which for a multiplicative subgroup is $X^{|H|}-1$, evaluable in $O(\log|H|)$ time [Tha22, §10.3.1, p. 363] (STARK uses additive subspaces of a binary field [BBHR18]). *Multilinear* PIOPs encode a vector of length 2^v by its multilinear extension, the unique multilinear polynomial in v variables that agrees with it on the Boolean hypercube [Tha22, §3.5, Fact 3.5], and reduce hypercube sums to one evaluation with the sum-check protocol of Lund, Fortnow, Karloff and Nisan [Tha22, Ch. 4, p. 166]. Spartan proves R1CS this way [Set20, abstract], SuperSpartan CCS [STW23, abstract], and HyperPlonk moves PLONK to the hypercube, removing the large FFT that is "the bottleneck in generating a Plonk proof" [CBBZ23, abstract]. The GKR- and MIP-based arguments below are multilinear PIOPs too [Tha22, §10.6, p. 395].

The sum-check protocol. The prover claims that a v-variate polynomial g over $\mathbb{F}$, of degree at most k in each variable (d is GKR's circuit depth below), sums to T over the Boolean hypercube. In round j it sends the univariate polynomial g_j, where $r_1, \ldots, r_{j-1}$ are the verifier's earlier challenges:

$$T = \sum_{\boldsymbol{b} \in \{0,1\}^v} g(\boldsymbol{b}), \qquad g_j(X_j) = \sum_{(b_{j+1},\ldots,b_v) \in \{0,1\}^{v-j}} g(r_1, \ldots, r_{j-1}, X_j, b_{j+1}, \ldots, b_v).$$

The verifier checks that g_j has degree at most k and that $g_{j-1}(r_{j-1}) = g_j(0) + g_j(1)$ (with T on the left when $j = 1$), then sends a fresh uniform $r_j \in \mathbb{F}$. After round v it checks $g_v(r_v) = g(r_1, \ldots, r_v)$ with a single evaluation query [Tha22, §4.1, pp. 168–171, Remark 4.2, p. 174]. A false claim survives only if some g_j differs from the honest polynomial yet agrees with it at r_j, so the soundness error is at most $vk/|\mathbb{F}|$ [Tha22, Prop. 4.1, p. 172]. The verifier does $O(vk)$ work plus the query, and the prover $O(2^v)$ evaluations of g for constant k [Tha22, p. 174].

Multilinear extensions make that prover cost real. The extension $\tilde{f}(X) = \sum_{\boldsymbol{b}} f(\boldsymbol{b})\, \chi_{\boldsymbol{b}}(X)$, with $\chi_{\boldsymbol{b}}(X) = \prod_{i=1}^{v} \big(X_i b_i + (1 - X_i)(1 - b_i)\big)$, can be evaluated at any point in $O(2^v)$ time [Tha22, Lemma 3.6, p. 159, Lemma 3.8, p. 163], and if g is a product of k multilinear polynomials the prover runs in time $O(k \cdot 2^v)$, linear and FFT-free [Tha22, Lemma 4.5, p. 194]. This is why sum-check drives the multilinear systems: it cuts HyperPlonk's prover from $O(|C| \log |C|)$ to $O(|C|)$ [CBBZ23, §1, p. 4], and Lasso (§4.3) and HyperNova (§6.4) build on it.

The GKR protocol. GKR runs one sum-check per layer of a *layered* arithmetic circuit, whose wires join only adjacent layers, numbered from the output (layer 0) to the input (layer d) [Tha22, §4.6.3, p. 210]. Let layer i have 2^{k_i} gates with value extension $\tilde{W}_i$. The *wiring predicates* $\mathrm{add}_i(a, b, c)$ and $\mathrm{mult}_i(a, b, c)$ equal 1 exactly when gate a of layer i adds, respectively multiplies, gates b and c of layer $i + 1$; they depend on C, not on its input [Tha22, §4.6.4, pp. 212–214]. Each layer is a sum over the next [Tha22, Lemma 4.7, p. 215]:

$$\tilde{W}_i(z) = \sum_{b,c \in \{0,1\}^{k_{i+1}}} \Big(\widetilde{\mathrm{add}}_i(z, b, c) \big(\tilde{W}_{i+1}(b) + \tilde{W}_{i+1}(c) \big) + \widetilde{\mathrm{mult}}_i(z, b, c)\, \tilde{W}_{i+1}(b)\, \tilde{W}_{i+1}(c) \Big).$$

Starting from the claimed outputs' extension at a random point, each iteration runs sum-check on this identity to reduce a claim about $\tilde{W}_i$ at one point to claims about $\tilde{W}_{i+1}$ at two points, which the prover merges by sending $\tilde{W}_{i+1}$ restricted to the line through them [Tha22, pp. 214–218]. The last claim concerns the input alone: the prover commits to the witness extension $\tilde{w}$ first and opens it once at the end, and no intermediate layer is ever committed [Tha22, §6.5.2, pp. 265–266; KRS25, §2.1, pp. 7–8]. The price is depth: rounds and communication are $O(d \log |C|)$ and the soundness error $O(d \log |C|/|\mathbb{F}|)$ [Tha22, §4.6.5, p. 221]. The verifier must also evaluate $\widetilde{\mathrm{add}}_i$ and $\widetilde{\mathrm{mult}}_i$ at a random point for every layer, which is fast for circuits with repeated wiring structure but not in general [Tha22, §4.6.6, p. 222] (Table A.2). Libra made the prover linear in $|C|$ by splitting each layer's sum-check into two [XZZ+19, abstract, §1.2, p. 4].

Why GKR came back, and why that is fragile. Committing only to the input makes GKR provers fast: Hyrax uses cryptography "only for the witness and for the prover's messages" [WTS+18, §1, pp. 1–2], Virgo's prover takes $O(|C| + n \log n)$ time for n inputs [ZXZS20, §4.1, p. 19, Thm. 3, p. 20], and logUp-GKR (§4.3) commits to no helper columns but the multiplicities [PH23, abstract, §2, p. 2]. The same property enables Khovratovich, Rothblum and Soukhanov's attack on the Fiat–Shamir version of such arguments: for any multilinear commitment and Fiat–Shamir hash, they construct a slightly deeper circuit that computes both internally, and an accepting proof of a false statement, so the compiled

argument "is not adaptively sound" [KRS25, Thm. 1, p. 4, §1.1.2, p. 4]. Committing to some intermediate values mitigates the attack but gives back part of the speed [KRS25, §5, p. 16]; the attack "does not seem to be applicable" to the canonical circuits of lookup and grand-product arguments [KRS25, Remark 3, p. 6]. Section 5.4 lists the schemes it affects, and §9.4 an extension to Spartan- and Aurora-style IOPs over program-generated R1CS [Fen26].

Multi-prover interactive proofs. In a multi-prover interactive proof (MIP) [BGKW88] the verifier questions two or more provers who cannot communicate during the protocol [Tha22, Def. 8.1, pp. 312–313]; a second prover, never seeing the first one's questions, "essentially functions as a polynomial commitment scheme" [Tha22, §8.1.1, pp. 313, 316]. For R1CS (§4.1), a claimed multilinear extension $\tilde{z}$ of the assignment is valid exactly when, for every row index $\boldsymbol{a}$,

$$\Big(\sum_{\boldsymbol{b}} \tilde{\mathbf{A}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b})\Big)\cdot\Big(\sum_{\boldsymbol{b}} \tilde{\mathbf{B}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b})\Big) \;-\; \sum_{\boldsymbol{b}} \tilde{\mathbf{C}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b}) \;=\; 0$$

[Tha22, §8.4.2, Eq. (8.4), p. 331], which a few sum-checks reduce to evaluations of $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$, $\tilde{\mathbf{C}}$ and $\tilde{z}$ at random points [Tha22, pp. 332–334]. In the argument a commitment to $\tilde{z}$ replaces the second prover and the low-degree test [Tha22, §8.3, pp. 327–328]; unlike in GKR, the number of sum-checks does not grow with depth, at the price of committing to the whole assignment [Tha22, pp. 321, 328]. Spartan (§5.3) compiles the two-prover protocol of Babai, Fortnow and Lund [BFL90] into a public-coin succinct argument "without employing FHE or low-degree tests" [Set20, §3.2, pp. 14–15].

Holography and preprocessing. A verifier that reads C cannot run in time polylogarithmic in $|C|$. In a holographic protocol such as an AHP, the verifier instead queries polynomials that an indexer computes from the circuit [CHM+20, §2.1, p. 10]; compiled, the indexer becomes a deterministic circuit setup $\mathcal{S}_2$ that outputs commitments to them [CHM+20, Thm. 8.1 and §8.1, pp. 41–42]. Fractal's holographic IOPs give a transparent $\mathcal{S}_2$ in the random oracle model [COS20, abstract], and Spartan's computation commitments, "a one-time, public computation to preprocess a given NP statement", play the same role for sum-check-based arguments [Set20, abstract]. In this sense these schemes are preprocessing without being circuit-specific (§3, Remark 1).

4.3 Lookup arguments

A lookup argument proves that committed values lie in a fixed table, replacing expensive bit decompositions for range checks, bitwise operations and big-number arithmetic [GW20, §1; STW24, abstract]. Figure 2 groups the arguments by technique, following Qu et al.'s account of the line that starts with Arya [QGX+26, §1.2, Table 1]. Two shifts organize the line: from grand products to logarithmic derivatives, and from table-sized commitments to sparse sums proven by sum-check (§4.2). Throughout, m is the number of lookups and n the table size.

From grand products to logarithmic derivatives. plookup sorts the queries together with the table and checks the result with one grand product, reusing PLONK's accumulator (§4.1) [GW20, §1, p. 2, Claim 3.1, pp. 4–5]. LogUp replaces sorting with the identity $\sum_i 1/(X+a_i) = \sum_i \nu_i/(X+b_i)$, where ν_i is the multiplicity of b_i, and proves it with one batched sum-check; the multiplicities are normalized if the table repeats values, and the field's characteristic must exceed the number of terms [Hab22, Lemma 5, p. 6, §3.1, pp. 10–11]. cq proves it on the full table with precomputed "cached quotient" KZG commitments, so that after $O(n \log n)$ preprocessing the prover's $O(m \log m)$ cost

is independent of the table size [EFG22, abstract, §1.2, p. 3]. logUp-GKR proves the fractional sum with GKR, so that besides the looked-up columns the prover commits only to the multiplicities [PH23, abstract, §3.1, pp. 3–4, §4, p. 7]; its authors call the paper an informal note that defers a formal security analysis [PH23, abstract, §3.2, p. 5]. Celer, for queries far outnumbering the table, reduces its main cost to one grand-product check of size m over the queries and commits to fewer than m elements [QGX+26, §1, fn. 1, p. 245, Table 1, p. 247].

Sparse sums and tables that are never committed. Spark [Set20, abstract] proves evaluations of a sparse polynomial, such as the extension of an R1CS matrix (§5.3), by offline memory checking, which compares fingerprints of read and write multisets with a grand product [Set20, §7.2, pp. 25–28; STW24, §4.1, pp. 17–21]. Lasso turns lookups into one such sparse sum, $\sum_{\boldsymbol{y}} \tilde{\mathbf{M}}(\boldsymbol{r}, \boldsymbol{y})\, \widetilde{\mathrm{Val}}(\boldsymbol{y}) = \tilde{\boldsymbol{a}}(\boldsymbol{r})$ at a random $\boldsymbol{r}$, for a claimed $m \times n$ matrix $\mathbf{M}$ with a single 1 in each row and the looked-up values $\boldsymbol{a}$ [STW24, §3, Eq. (5), p. 12]. For a *decomposable* table, whose entries are fixed multilinear functions of entries of $O(c)$ small subtables, "no party needs to commit to" the table, so it may have 2^{128} entries or more, and the prover "pays" only for the entries that the lookups read [STW24, abstract, §1.1, p. 4]. Jolt's never-materialized instruction table (§8) was first built on Lasso; Jolt now proves those lookups with Shout (below).

One-hot addressing. Twist and Shout drop the fingerprints and the grand product. In Shout, the prover commits to each address in one-hot form, so that correct reads become one sum-check over the address space plus checks of the one-hot form, and the prover pays only for the m nonzero entries [STZ26, §1.1, pp. 269–270, §4.3, Fig. 5, Thm. 1, p. 283, §4.4, pp. 284–285]. Twist handles read/write memory by committing to per-cycle increments instead of memory contents [STZ26, §1.1, p. 270]. Twist and Shout replace the memory checking of earlier versions of the Jolt zkVM and make the Jolt prover about 3× faster and proofs about 5× smaller on RISC-V (RV32IM) benchmarks [STZ26, abstract].

4.4 Polynomial commitment schemes

Definition. A polynomial commitment scheme (PCS), introduced and formally defined by Kate, Zaverucha and Goldberg [KZG10, abstract], lets a prover commit to a polynomial f of bounded degree with a short string and later prove claims $f(z) = y$. It consists of a setup (public parameters for a maximum degree), a commitment algorithm and an evaluation protocol, an interactive argument that the committed f satisfies $f(z) = y$. *Binding* says that a commitment cannot be opened to inconsistent evaluations; compiling a PIOP needs the stronger *extractability*, which "is to binding as knowledge-soundness is to standard soundness": any committer who passes the evaluation checks must know a polynomial of the claimed degree that explains its answers [Tha22, §7.4, p. 307]. For Fiat–Shamir security, Chiesa et al. argue, the right property is instead a state-restoration variant of *function binding*, a generalization of position binding that is incomparable to extractability: PLONK's linearized KZG satisfies it, although an attack of Lipmaa et al. shows it is not special-sound and, Chiesa et al. note, extends to extractability [CGKY26, §1, pp. 191–192, 195] (§4.5). Zero knowledge also needs hiding commitments and zero-knowledge evaluation [Tha22, §13.3, p. 454].

Pairing-based: KZG. A KZG commitment to f is $g^{f(\tau)}$ for a secret τ, computed from a reference string of powers g^{τ^i}; the evaluation proof is a single group element, a commitment to the quotient $(f(X) - f(z))/(X - z)$ [KZG10, §1, p. 2, §3.2, p. 6]. In the notation of §2.3, for a commitment $c_f = g_1^{f(\tau)}$, a claim $f(z) = y$ and a proof $\pi_z = g_1^{\psi(\tau)}$ with

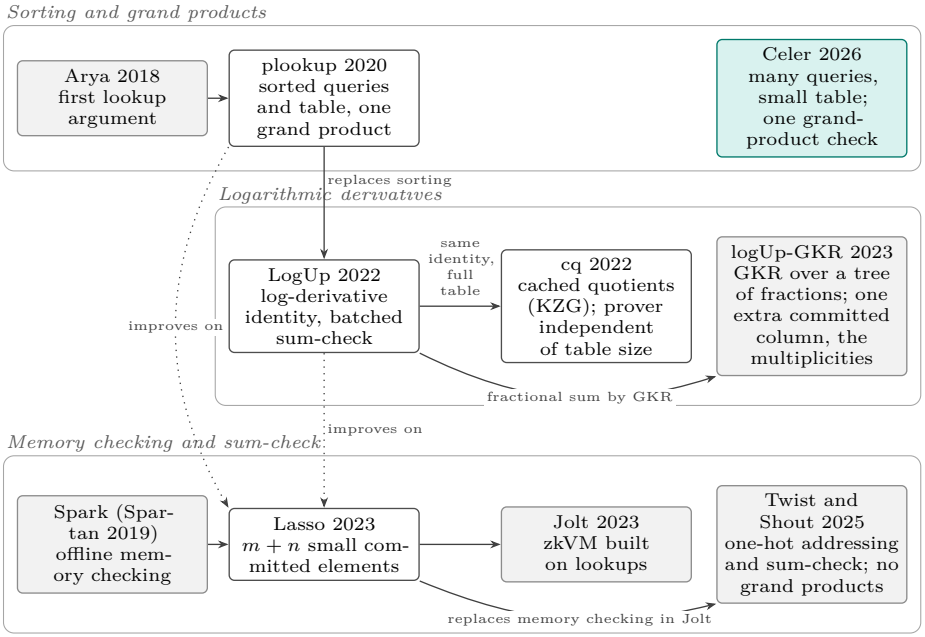


Figure 2: Lookup arguments by technique. Solid arrows mark a construction that builds on or replaces the earlier one; dotted arrows mark a comparison the later paper makes ("improves on"). White nodes commit to a number of field elements linear in the number of lookups m; Celer (teal) commits to fewer than m when $m \gg n$ [QGX+26, §1.1–1.2, Table 1]. Grey nodes are not classified by this count. Arya is drawn as Qu et al. describe it [QGX+26, §1.2]. Sources: [GW20], [Hab22], [EFG22], [PH23], [Set20], [STW24], [AST24], [QGX+26], [STZ26].

$\psi(X) = (f(X) - y)/(X - z)$, the verifier checks

$$e\big(c_f \cdot g_1^{-y},\ g_2\big) \ = \ e\big(\pi_z,\ g_2^{\tau} \cdot g_2^{-z}\big),$$

which is the identity $f(\tau) - y = \psi(\tau)(\tau - z)$ in the exponent and needs only g_2 and g_2^{τ} [KZG10, §3.2, p. 7; Tha22, §15.2, Eq. (15.2), p. 502]. If $f(z) \neq y$, then ψ is not a polynomial, and passing the check requires "dividing in the exponent" by $\tau - z$, which the t-strong Diffie–Hellman (t-SDH) assumption rules out; binding rests on t-SDH [Tha22, p. 503; KZG10, §1, p. 2, Def. 2.3, p. 4]. The reference string is a trusted but universal $\mathcal{S}_1$, updatable because it consists only of powers of τ [Tha22, §16.3, pp. 545–547]. Sonic, PLONK and Marlin use KZG. PLONK batches its openings by a random combination, so its proof carries just two opening elements, for its challenge point z and for $z\omega$, and verification still takes two pairings [GWC19, §3.1, pp. 12–13, §8.3, pp. 30–31]. Extractability is typically proved in the algebraic group model, for PLONK under $2d$-DLOG [GWC19, §3, pp. 12–13, Lemma 4.7, p. 17]; for KZG-like schemes this is weak evaluation binding, not standard-model extraction [CHS26, Thm. 2, p. 11, Thm. 6, pp. 17–18]. Belohorec et al. claim the first standard-model extractability proofs for multivariate KZG and many of its variants (our §5.2 covers univariate KZG). They use the falsifiable n-variate ARSDH assumption for an "extended" opening, in which the verifier sends the point one coordinate per round, and a variant, GARSDH, for the canonical one-shot opening [BDH+25, abstract, §2, Thms. 2–3, pp. 597–598, §5.4, Thms. 5–6, p. 613]. Siim and Pal show both assumptions equivalent to the original ARSDH but find two gaps in the proof for canonical PST (the multivariate KZG of Papamanthou et al.), leaving its extractability under a weaker falsifiable assumption open [SP26, Thm. 3, p. 10, Cor. 2, p. 11, §5, pp. 12–15].

Discrete-log-based: IPA, Hyrax-commit, Dory. The inner-product argument of Bootle et al. [BCC+16, abstract], refined in Bulletproofs [BBB+17, §3], gives a transparent polynomial commitment from discrete log alone [Tha22, §14.4]. Each round halves the committed vector, folding it, the generators and the commitment with a random challenge [Tha22, §14.4.1, Protocol 12, pp. 480–482; BBB+17, §3, Protocol 2, pp. 13–16], so proofs have $O(\log s)$ group elements for s coefficients, but the verifier must fold the generators itself, a multi-exponentiation of size $O(s)$ [Tha22, Table 16.1, p. 546; BBB+17, §3.1, pp. 15–17]. The proof length is optimal: Dietz and Giunta show in the generic group model that, for public-coin (Fiat–Shamir) openings over pairing-free prime-order groups, $\Omega(\log s)$ group elements are necessary unless the commitment grows with s or the opening proof has length $s^{1-o(1)}$ [DG26, abstract, §1.1, p. 220, §2.4, p. 230, Cor. 1, p. 246]. KZG and DARK lie outside its model.

The linear verifier is why Bulletproofs is not a zk-SNARK in the strict sense of §3 (Table A.2, §5.4), and why Halo, which uses the same kind of commitment, keeps one linear-time final check (§6). Hyrax-commit commits to the coefficients, arranged as a $\sqrt{s} \times \sqrt{s}$ matrix, row by row, which gives $O(\sqrt{s})$-size commitments and verification [WTS+18, §6.1; Tha22, Table 16.1]. Dory reaches $O(\log s)$ communication and verification with a transparent setup, using pairings, under the standard symmetric external Diffie–Hellman (SXDH) assumption [Lee21, abstract].

Hash-based: how FRI works. FRI, the Fast Reed–Solomon IOP of Proximity [BBHR18b], is an IOP of proximity (§2.1, §4.2) for RS$[\mathbb{F}, \mathcal{L}_0, d]$, where $\mathcal{L}_0$ is a multiplicative subgroup and $|\mathcal{L}_0|$ and d are powers of 2 [BBHR18b, §2.1, p. 14:10]. In the *commit phase*, the prover splits $f_0(X) = f_{0,\mathrm{e}}(X^2) + X f_{0,\mathrm{o}}(X^2)$ into even and odd parts of degree less than $d/2$ and, for a random $c_0 \in \mathbb{F}$, sends the fold $f_1 = f_{0,\mathrm{e}} + c_0 f_{0,\mathrm{o}}$ on $\mathcal{L}_1 = \{u^2 : u \in \mathcal{L}_0\}$, which halves the domain and the degree bound [BBHR18b, p. 14:11; Tha22, §10.4.4, pp. 378–379]. After about $\log_2 d$ rounds the prover sends a constant; in the compiled argument each f_i is a Merkle root and Fiat–Shamir supplies the challenges [BBHR18b, §1.3.2, p. 14:8]. In the

query phase, the verifier picks a random $u_0 \in \mathcal{L}_0$, sets $u_{i+1} = u_i^2$, opens $f_i(\pm u_i)$ at each layer and checks

$$f_{i+1}(u_i^2) \;=\; \frac{f_i(u_i) + f_i(-u_i)}{2} \;+\; c_i \cdot \frac{f_i(u_i) - f_i(-u_i)}{2u_i},$$

whose right side is $f_{i,\mathrm{e}}(u_i^2) + c_i f_{i,\mathrm{o}}(u_i^2)$ for an honest prover [BBHR18b, §2.1, p. 14:11].

The fold is a random combination of two functions on $\mathcal{L}_{i+1}$; if both were δ-close to polynomials of degree less than $d/2$ on a common set of points (correlated agreement), f_i would be δ-close to the code. So if f_i is δ-far, a proximity gap keeps f_{i+1} δ-far except with small probability [Tha22, pp. 382–383; BBHR18b, §2.2, pp. 14:12–14:13]. Each of ℓ query repetitions then rejects with probability at least δ, leaving error $(1-\delta)^\ell$ plus the chance of an unlucky fold [Tha22, Thm. 10.4, p. 383]. Proofs have $O(\lambda \log^2 d)$ hash values, and the prover's main cost is the FFT that evaluates f_0 on $\mathcal{L}_0$ [Tha22, §10.4.3, pp. 376–377, Table 16.1].

As a PCS, the commitment is the Merkle root of f on $\mathcal{L}_0$, and $f(z) = y$ is proven by running FRI on the quotient $(f(X) - y)/(X - z)$, as in KZG; for $\delta < (1-R)/2$, acceptance binds the prover to a polynomial f with $f(z) = y$ [Tha22, §10.4.2, pp. 374–375].

Hash-based: FRI's successors and linear codes. The later hash-based commitments differ in two choices: the code, Reed–Solomon or another linear code that is faster to encode or works over any field, and the proximity test, FRI-style folding or, as in Ligero, a check of a random combination of the rows of the committed matrix [Tha22, §10.5]. The FRI line folds Reed–Solomon codewords, the Ligero line applies the row check to Reed–Solomon or faster codes, and the third group folds other codes; Figure 3 sorts the schemes into these three groups, and Table 7.2 gives their costs.

Reed–Solomon proximity: the FRI line. STIR lowers FRI's query complexity from $O(\lambda \cdot \log d)$ to $O(\log d + \lambda \cdot \log\log d)$ for degree d [ACFY24, abstract]: it reduces the degree by a factor $k \geq 4$ per round but the domain only by 2, so the rate falls and later rounds need fewer queries [ACFY24, §2.1, p. 5]. WHIR handles queries to both multilinear and univariate polynomials [ACFY25, abstract]; it tests *constrained* Reed–Solomon codes, whose codewords, read as multilinear polynomials $\hat{f}$, also satisfy a sum-check constraint such as $\hat{f}(\boldsymbol{z}) = y$, so the proximity test also proves an evaluation [ACFY25, §1.1, Def. 1, p. 5].

Linear codes: the Ligero line. These commitments trade proof size for prover speed, starting from the Ligero commitment's square-root-size proofs [Tha22, Table 16.1, §10.5]. Brakedown makes the prover linear-time and field-agnostic with a new linear-time-encodable code [GLS+23, abstract]; it appears here only as a PCS, since its paper makes it zero-knowledge only by composition with an outer zkSNARK (§5.4). Orion makes Brakedown-style proofs polylogarithmic by code switching [XZS22, abstract], but the revision cited here is unsound and its commitment's verifier is $O(\sqrt{s})$ [DS25, abstract, p. 5] (§9.1). Block et al.'s random expand-accumulate codes have constant rate and distance over any finite field, giving a Brakedown-style PCS with proofs 1.9–2.8× smaller [BFK+24, abstract]. Binius adapts Brakedown to towers of binary fields and keeps square-root verification [DP25, §1, p. 1, Thm. 3.14, p. 20]. Ligerito recurses Ligero's matrix–vector protocol with partial sum-checks, giving a polylogarithmic verifier for tensorizable codes such as Reed–Solomon [NA25, abstract, §6.2, pp. 12–13, §6.6, pp. 18–19].

Foldable and binary codes. BaseFold generalizes FRI from Reed–Solomon codes to any foldable linear code, interleaved with sum-check, giving a field-agnostic multilinear PCS with an $O(s \log s)$ prover [ZCF24, abstract]. The successor of Binius reaches polylogarithmic verification, proven in the unique-decoding regime, by combining a characteristic-2 BaseFold with "ring-switching", which turns a multilinear commitment over a large extension field into one over the ground field [DP26, abstract, §4.4, p. 44, §5.2, p. 47]. Blaze, also over

binary fields, gets a linear-time prover from repeat-accumulate-accumulate (RAA) codes and BaseFold [BCF+25, abstract, Thm. 1, p. 4].

Apart from Ligero, Orion and, through an outer zkSNARK, Brakedown, none of these schemes is zero-knowledge as presented (§3), as is typical of FRI-based commitments [Tha22, §13.3, p. 454]; Blaze "does not guarantee any hiding properties" [BCF+25, Remark 2, p. 5]. Honest-verifier zero knowledge has since been proven at the IOP level, though not for the compiled argument, for variants of WHIR and Ligerito over fields of characteristic other than 2 [CFW26, Thm. 10.2, p. 63], small-field STARKs for a simple AIR [HA24, Thm. 6, p. 13] and binary BaseFold [Dia25, Thm. 4.3, p. 5]. Hash-based schemes are plausibly post-quantum [Tha22, Table 16.1 caption], but their concrete parameters often rest on unproven proximity-gap conjectures for Reed–Solomon codes (§5.3).

Groups of unknown order and lattices. DARK (Diophantine Arguments of Knowledge) commits to polynomials in a group of unknown order [BFS20, §1.1, p. 3]; it is transparent with class groups and trusted with RSA groups (§5.3, Supersonic). DewTwo, also in groups of unknown order, is transparent with doubly logarithmic proofs [BMSS25, abstract], but its prover is estimated at just over 18 hours for $s = 2^{24}$ coefficients on one thread [BMSS25, Remark 3, p. 572]. Greyhound is a transparent lattice-based PCS from the Module Short Integer Solution problem (Module-SIS) in the random oracle model, with polylogarithmic evaluation proofs but an $O(\sqrt{s})$ verifier [NS24, §1.1, p. 245]; its short proofs come from LaBRADOR, a Module-SIS proof of knowledge for quadratic constraints that recurses on its last message, so $O(\log \log n)$ iterations suffice for a witness of size n [BS23b, §1.1, pp. 521–522].

Among other lattice-based commitments, Cini et al. give a transparent PCS from Module-SIS with a knowledge-soundness proof against quantum adversaries (which also assumes LWE), whose concretely efficient variant, with cube-root rather than polylogarithmic proof size, has 403 KB proofs at degree 2^{20} [CMNW24, abstract, §1.1, p. 210, Fig. 2, p. 209]. Several earlier lattice SNARKs rely on the knowledge k-R-ISIS assumption; Wee and Wu's oblivious-sampling attack gives evidence against it, although it does not directly break these schemes' soundness [WW23, abstract, §1, p. 11].

Which commitments give strict zk-SNARKs. The commitment, more than the PIOP, decides whether the final argument meets the strict bound of §3: a holographic PIOP with polylogarithmically many queries inherits the commitment's "efficiency bottlenecks" [Tha22, Ch. 19, p. 608], here its per-evaluation cost. Three classes of evaluation cost give zk-SNARKs in our sense. Constant: KZG. Logarithmic: DARK, Dew, multilinear KZG, and Dory after an $O(\sqrt{s})$-exponentiation preprocessing [Tha22, Table 16.1 caption]. Polylogarithmic: FRI and its successors, Ligerito, Scorpius relative to an outer SNARK (§9.1), and Cini et al.'s lattice commitment, which is not hiding [CMNW24]. Linear (IPA) and square-root (Hyrax-commit, Ligero, Brakedown, Binius, Greyhound) verification do not. Hence Bulletproofs, Hyrax and Spartan$_{\mathrm{DL}}$ (Spartan with Hyrax's commitment, §5.3) are in Table A.2, not Table 7.1, although some of their papers call them zk-SNARKs under sublinear notions of succinctness (§1, §5.4). Table 7.2 (§7) summarizes these families.

4.5 Compiling to a non-interactive argument

From PIOP to interactive argument. Compilation replaces each polynomial message by a commitment and each evaluation query by an evaluation proof [Tha22, §10.2, p. 360], and compiler theorems show that security carries over. For any public-coin AHP and extractable PCS, Marlin obtains a public-coin preprocessing argument with a universal SRS that keeps knowledge soundness, zero knowledge if the PCS is hiding, and updatability [CHM+20, Thm. 1, p. 5, Thms. 8.1, 8.3, 8.4 and Remark 8.2, p. 41]. DARK proves the analogue for

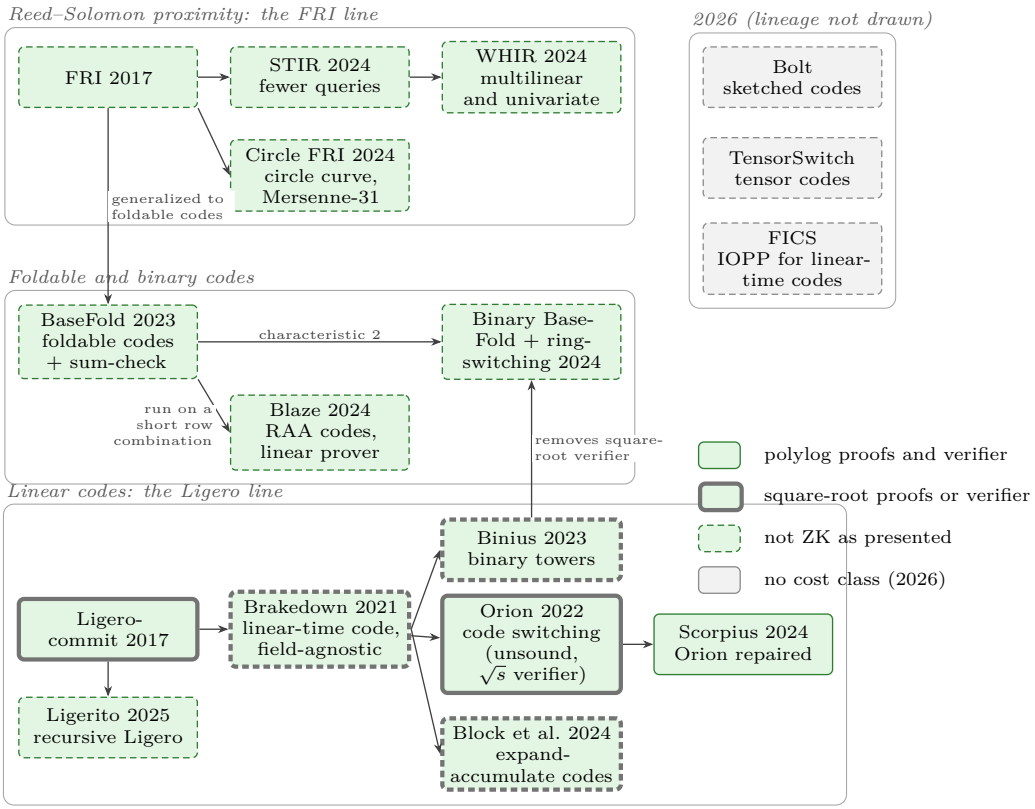


Figure 3: Hash-based polynomial commitments and their lineage. Arrows mark a scheme that adapts or generalizes the earlier one. Green nodes (as in Figure 4) form the FRI line, the Ligero line and the foldable-code group. A thin border marks polylogarithmic evaluation proofs and verification, with which a holographic PIOP can meet the strict bound of §3 (Scorpius relative to an outer SNARK, §9.1; Ligerito for tensorizable codes); a thick grey border, as in Figure 4, marks square-root-size proofs or verification. A dashed border marks schemes that are not zero-knowledge as presented (§3), among them the FRI-based commitments [Tha22, §13.3, p. 454] and circle FRI (§5.4); Brakedown is zero-knowledge only via an outer zkSNARK (§5.4). Only Ligero, Orion and Scorpius are presented with zero knowledge; Orion has a thick border because its commitment's verifier is $O(\sqrt{s})$, and its cited revision is unsound (§9.1). The 2026 schemes, Bolt and TensorSwitch (§9.1) and FICS (§5.3), are grey, without lineage edges or cost class, because their verifier bounds are not in Table 7.2. Sources: [BBHR18b], [ACFY24], [ACFY25], [HLP24], [ZCF24], [DP26], [BCF+25], [Tha22, Table 16.1], [GLS+23], [XZS22], [DS25], [BFK+24, Table 1, p. 278], [DP25], [NA25], [GNR26], [BFRW26], [BMMS26].

polynomial IOPs, given an evaluation protocol with witness-extended emulation, including honest-verifier zero knowledge when the PCS is hiding and its evaluation honest-verifier zero-knowledge [BFS20, §1.1, p. 4]. PLONK proves it directly for KZG in the algebraic group model [GWC19, Lemma 4.7, p. 17]. IP- and MIP-based protocols need a multilinear PCS, univariate PIOPs a univariate one [Tha22, Ch. 19, p. 608].

Knowledge soundness. The extractor $\mathcal{E}$ of §3 is built in two layers. The PCS extractor recovers the polynomials behind the prover's commitments, and the PIOP's knowledge soundness turns them into a witness [CHM+20, Thm. 8.3]. In Thaler's GKR-based argument, the witness is read off the extracted polynomial, and GKR's soundness shows it valid [Tha22, §7.4, pp. 307–309].

How zero knowledge is obtained. Zero knowledge also needs both layers: hiding commitments and evaluation proofs, and revealed evaluations that say nothing about w. In every scheme fresh prover randomness makes each value the verifier sees uniform or fixed by the verification equations, so the simulator of §3 can sample it without w. Groth16 adds random multiples of its setup trapdoor δ_{G} (Groth's δ, subscripted to avoid a clash with the proximity parameter) to two proof elements and corrects the third, giving perfect zero knowledge [Gro16, §3.2, p. 17, Thm. 1, p. 15, Thm. 2, p. 18]. PLONK, whose KZG commitments are not hiding, adds random low-degree multiples of Z_H to the prover's polynomials and draws challenges outside H, so every revealed value is "either completely uniform or determined by verifier equations" [GWC19, §8, p. 26, §8.3, p. 28]. Hash-based IOPs, whose Merkle openings reveal codeword entries, must be zero knowledge against a bounded number of queries: Aurora and STARK mask with random higher-degree extensions and a random codeword [BCR+19, §7.1, pp. 30–31, §8.1, p. 37; BBHR18, App. C.1.3, p. 53, App. D.1, pp. 63–64]. With salted leaves, the BCS compiler (the IOP compilation of §4.2 [BCS16]) turns an honest-verifier zero-knowledge IOP into a statistically zero-knowledge argument [BCS16, §3.2, pp. 15–16, Thm. 7.1, p. 27; COS20, Thm. 10.1, p. 40, §10.2, p. 42]. Sum-check is masked by running it on $g + \alpha p$ for a committed random p [Tha22, §13.3, p. 455; XZZ+19, §4.1, pp. 19–20]. Commit-and-prove can add zero knowledge afterwards, but with Pedersen commitments each verifier multiplication may cost at least nine extra group elements, and the result is not post-quantum sound [Tha22, §13.4, pp. 460–461].

Not zero-knowledge as presented. The commitments that §4.4 and §9.1 mark this way (§3) would need such a step: Blaze proposes an extra proof-composition step [BCF+25, Remark 2, p. 5], and Bolt's authors write that zero knowledge "seems to be at odds with having a systematic encoding" and suggest recursing into a zero-knowledge proof [GNR26, §1.3, p. 407]. Masking a linear code is delicate: Orion salts its hashes and appends a random vector to each masked codeword, and its first mask was not zero-knowledge [XZS22, §2.2, p. 10, §4.3, fn. 4, p. 23].

Fiat–Shamir. The final step is the Fiat–Shamir transform of §2.3, which must hash the statement and the public parameters (strong Fiat–Shamir) to avoid the vulnerabilities of §9.4 [DMWG23]. Its soundness is subtle: some k-round arguments with soundness error ε have Fiat–Shamir error $m^{\Omega(k)} \cdot \varepsilon$ against adversaries making m oracle queries, so the target is not standard soundness but *state-restoration soundness*, which implies Fiat–Shamir security; round-by-round soundness suffices only for interactive proofs, not arguments [CGKY26, §1, p. 190]. Chiesa, Guan, Knabenhans and Yu prove that a state-restoration-sound PIOP compiled with a state-restoration function-binding commitment gives a state-restoration-sound argument [CGKY26, Thm. 1, p. 192, formal version and proof: full version, Thm. 7.1, p. 52]; their framework also covers IOPs with vector commitments and linear PCPs with linear commitments [CGKY26, abstract, pp. 189–192], and re-derives PLONK's security from falsifiable assumptions (§5.2).

Where trust enters. The only trusted component of a PIOP-based zk-SNARK is the setup of its commitment scheme (Figure 1): the trusted portion "is entirely confined to the polynomial commitment scheme's setup" [BFS20, §1, p. 3], and the argument inherits the commitment's "cryptographic and setup assumptions" [Tha22, Ch. 19, p. 608]. KZG gives a universal, updatable but trusted $\mathcal{S}_1$; FRI, an IPA, or DARK with class groups gives a transparent one (Table 7.2; on updatable reference strings see [GKM+18]). The random oracle is a separate assumption that enters chiefly at the Fiat–Shamir step (§3.1).

Two routes outside the recipe. The linear-PCP schemes GGPR, Pinocchio and Groth16 compile a linear PCP whose queries are sampled during setup and hidden "in the exponent" of a pairing group; the route goes back to Ishai, Kushilevitz and Ostrovsky, who showed that a prover can commit to a linear PCP, a proof of exponential length, without writing it out [IKO07; Tha22, §17.1, p. 549]. With the queries hidden, the argument is not public coin and "we cannot apply the Fiat-Shamir transformation" [Tha22, §17.5.1, p. 566]; instead, "a trusted preprocessing step ... preselects the verifier's linear PCP queries" [BFS20, §1.1, p. 3]. This makes $\mathcal{S}_2$ circuit-specific (Remark 1) and gives the smallest linear-PCP proofs, 3 group elements for Groth16 [Tha22, §19.2, p. 613] (§5.1); PIOP-based schemes with pairing-based commitments have since gone below it in group elements (§9.1).

The GKR-based schemes Hyrax, Libra and Virgo commit to the witness's multilinear extension but to no intermediate layer; because their costs grow with circuit depth and their verifier must evaluate the wiring predicates, they meet the strict bound at most for restricted circuits (Table A.2, §5.4). Their Fiat–Shamir versions are open to the attack of §4.2 [KRS25, abstract, §1.1.1, p. 3], Virgo's by our inference (§5.4).

5 Construction families of zk-SNARKs

We group the construction families by setup model (§3 Remark 1), since it fixes how $\mathcal{S}_1$ and $\mathcal{S}_2$ are filled and thus most of what must be trusted. Sections 5.1–5.3 cover the circuit-specific, universal and transparent families, which are the strict zk-SNARKs of Table 7.1. Section 5.4 covers the arguments outside the strict definition (Table A.2). Of the ten later schemes the tables add (§7), Polymath is discussed in §5.1, HyperPlonk, Orion and LaBRADOR with their components (§4.2, §4.4), Plonky2 and HyperNova in §6.4, Garuda, zk-Pari and Dew in §9.1, and Samaritan in §7. Recursion and folding follow in §6. Figure 4 arranges the same schemes by information-theoretic route; that view is independent of the setup-model grouping used here, because the commitment scheme, not the protocol, fixes the setup (§4.5). Figure 7 (Appendix A) orders them by date.

5.1 Circuit-specific setup — GGPR, Pinocchio, Groth16

This family compiles a *linear PCP*, in which the honest proof is a linear function (§4.5) [IKO07; Tha22, §17.1, p. 549], through pairings with a circuit-specific SRS. GGPR, who present their scheme as avoiding PCPs, instantiate it with the quadratic span program (QSP) and the quadratic arithmetic program (QAP) of §4.1, which turns circuit satisfiability into a single divisibility check with the quotient $h(X)$ as evidence [GGPR13, §1]. The QAP approach is what most later implementations refine [Gro16, §1, p. 2].

GGPR and Pinocchio

GGPR's CRS encodes the QAP polynomials and the powers of a secret point τ (GGPR's s; it plays the role of the SRS secret of §4.4) as group elements such as $g^{v_k(\tau)}$. The prover uses them to compute encodings of the three combinations and of $h(\tau)$. The verifier checks the identity at τ with a pairing; in GGPR's simplified description of the QSP version

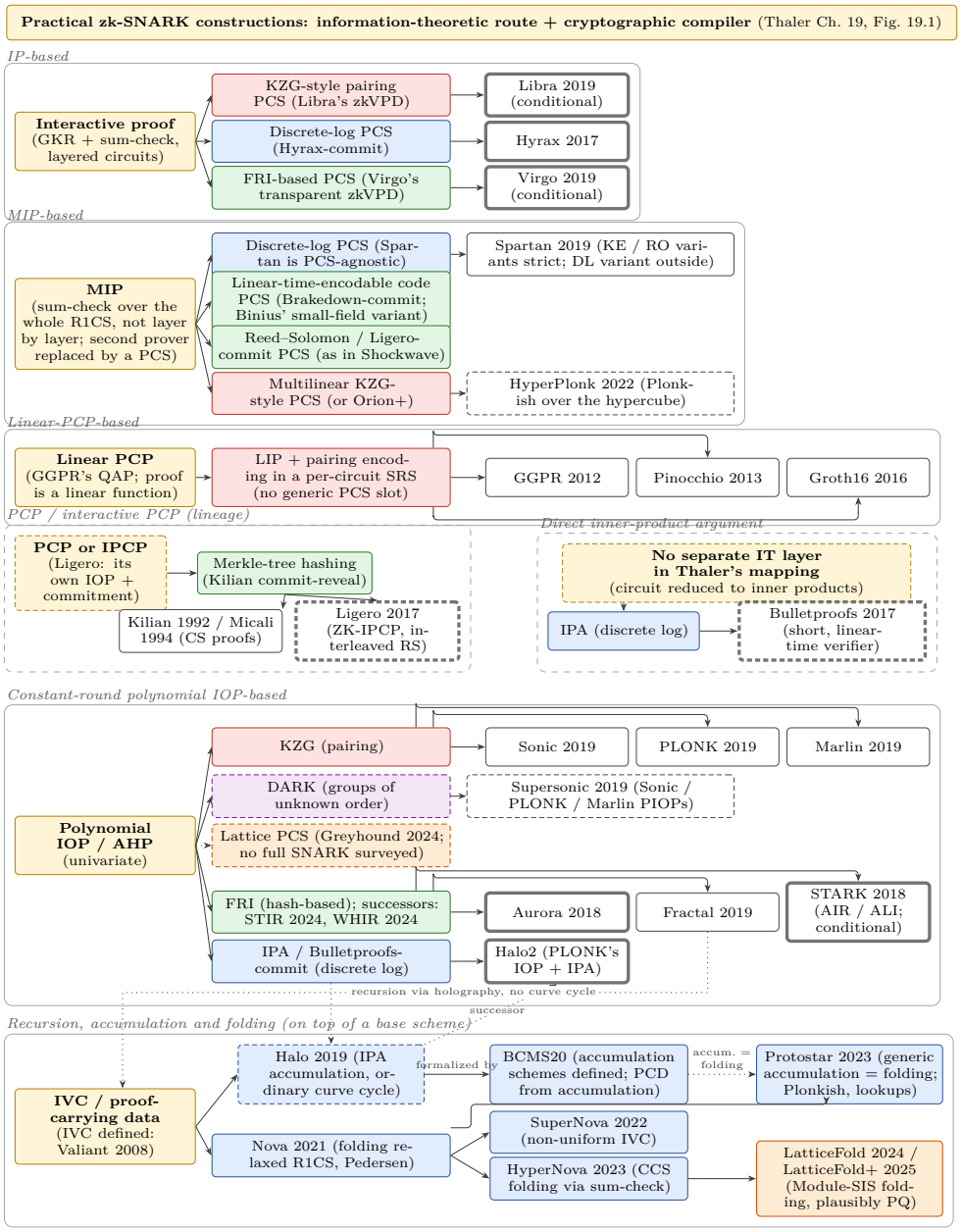


Figure 4: zk-SNARK constructions by information-theoretic route. Practical zk-SNARKs as an information-theoretic protocol plus a cryptographic compiler, following Thaler's Ch. 19 taxonomy [Tha22, §19.1, Fig. 19.1, Halo2: §19.1(f), p. 610]. The IP-, MIP- and constant-round polynomial-IOP-based routes can each be combined with any polynomial commitment scheme (multilinear for IP/MIP, univariate for PIOPs). The commitment, not the IOP, decides whether the result is transparent and whether it is plausibly post-quantum. The linear-PCP route (GGPR, Pinocchio, Groth16) has no generic commitment slot: it compiles directly through pairings with a circuit-specific SRS. Ligero is placed by its own framing as a zero-knowledge interactive PCP [AHIV17]; Thaler lists it as a constant-round polynomial IOP with its own commitment [Tha22, §19.1(g)]. Recursion, accumulation and folding sit on top of a base scheme, not inside the taxonomy (Figure 6). Apart from the lineage route (Kilian, Micali) and the recursion layer, only zero-knowledge schemes are drawn. A thick grey border marks zero-knowledge arguments that miss the strict succinctness bound of §3, or meet it only for restricted circuits (marked "conditional"); see Table A.2 (Halo2, which has no row, misses the bound through its IPA verifier, §4.4). Brakedown (zero-knowledge only via an outer zkSNARK, §5.4), Shockwave and Binius appear only as commitment-scheme leaves. Colours as in Figure 1; a dashed border marks an uncertain placement. In the STARK leaf, ALI is STARK's algebraic linking interactive oracle proof [BBHR18, §2.3].

it confirms $e(g^v, g^w) = e(g^h, g^{t(\tau)})$, and the QAP version adds the third combination y [GGPR13, §1.2, §8]. Knowledge soundness comes from the q-PKE assumption of §3.1 [GGPR13, §5.1]: GGPR prove the QAP-based zk-SNARK secure under the falsifiable q-power Diffie–Hellman assumption (q-PDH) and the non-falsifiable d-PKE (q-PKE at the QAP degree d) [GGPR13, §8.1, Thm. 14]. A proof has 9 group elements (Table A.1).

Pinocchio [PHGR13] turns this scheme into "a built system" [PHGR13, abstract]. It shrinks the proof "from nine group elements to eight" [PHGR13, §3, p. 5], 288 bytes as implemented. Its authors call it "the first general-purpose system to demonstrate verification cheaper than native execution (for some apps)" [PHGR13, abstract].

Groth16

Groth16 [Gro16] redesigns the argument so that a proof is only three group elements (§4.5). Verification takes three pairings [Gro16, abstract, §3.2]. A proof is 128 bytes on BN-256 [CHM+20, Fig. 1]. Groth16 is cheaper than Pinocchio on every cost in Groth's comparison except the verifier's ℓ exponentiations, which are equal [Gro16, Table 2]. Thaler attributes the three elements to two changes: a linear interactive proof with three queries instead of five, and the generic group model [Tha22, pp. 577–578]; linear interactive proofs are defined in [BCI+13, §2.3, Def. 2.5, p. 16]. Groth16's zero knowledge is perfect (§4.5), and its knowledge soundness is statistical against adversaries that use generic bilinear-group operations [Gro16, Thm. 2]. Groth calls relying on this model "an aggressive stance" and justifies it partly by the Gentry–Wichs barrier of §3.1 [Gro16, §1.1].

Groth also proves that linear interactive proofs cannot have a linear decision procedure, answering an open question of Bitansky et al. [Gro16, §4.1, Thm. 3; BCI+13, §1.3.3, pp. 11–12]. It follows that, for relations with hard decisional problems, a pairing-based non-interactive argument with a generic prover and verifier needs proof elements in both $\mathbb{G}_1$ and $\mathbb{G}_2$, and so at least two group elements [Gro16, §4.2, Thm. 4, Cor. 2]. Groth calls this "the first lower bound for pairing-based SNARGs" [Gro16, abstract]; §9.1 discusses a two-element SNARG in an extended model. Polymath shows that the bound counts group elements, not bits. Using a square arithmetic program, KZG and Fiat–Shamir, its proof is $3\,\mathbb{G}_1 + 1\,\mathbb{F}$, 1408 bits on BLS12-381 against Groth16's 1536 [Lip24, abstract, Table 1, p. 175]. It needs a random oracle, the algebraic group model with oblivious sampling, and a circuit-specific SRS [Lip24, §1, p. 174, §3, p. 188]. For the random oracle model, Lipmaa noted that "no non-trivial lower bound is known" [Lip24, §1, p. 170]. Arnon, Dujmovic and Yogev have since proved such a bound in the generic group model with a random oracle, against one-element proofs and under mild restrictions on the verifier (§9.1) [ADY26, abstract, p. 125].

Setup and assumptions

All three schemes have a circuit-specific trusted setup. In the terms of Remark 1, $\mathcal{S}_1$ is empty. $\mathcal{S}_2$ deterministically preprocesses C into its QAP and then samples secret trapdoors, and their encodings form $\mathsf{S_p}$ and $\mathsf{S_v}$. For Groth16 the trapdoors are $\alpha_G, \beta_G, \gamma_G, \delta_G$ (Groth's $\alpha, \beta, \gamma, \delta$, subscripted to keep them apart from the challenges of §4 and the proximity parameter δ) and the point τ (Groth's x) [Gro16, §3.2]. (GGPR's function-specific stage also samples fresh secrets [GGPR13, §3.1].) The parameters have size $O_\lambda(|C|)$ (Table 7.1), and anyone who keeps the trapdoor can forge proofs. Deployments therefore use multi-party ceremonies, which are secure if one participant is honest [MBKM19, §1]. Yet "any modification in the circuit ... requires another cryptographic ceremony" [CHM+20, §1], and the SRS cannot be updated (§5.2) [Tha22, §19.2, pp. 613–614]. The assumptions are non-falsifiable or idealized, as §3.1 explains. The pairing-based compilation also "appears to rely heavily on" the verifier computing a degree-2 function, so it does not extend easily to higher-degree gates [Tha22, §19.3.3, p. 627].

5.2 Universal and updatable setup — Sonic, PLONK, Marlin

The universal family removes the ceremony per circuit. In the terms of Remark 1, $\mathcal{S}_1$ runs once, with secret randomness, and outputs an SRS for all circuits up to size N; $\mathcal{S}_2$ is deterministic. Groth et al. [GKM+18] made the SRS *updatable* as well: security holds "if at least one of the users updating the CRS is honest" [GKM+18, abstract]. A CRS that encodes secret non-monomial polynomials cannot be updatable, because updates reveal the monomials. Groth et al. demonstrate this on Pinocchio and believe it extends to most QSP/QAP-based SNARKs [GKM+18, §1, p. 3, §6, p. 29]. Their own QAP-based zk-SNARK has a monomial-only universal CRS of quadratic size, which is specialized into linear-size CRSs for particular relations [GKM+18, abstract]. The obstruction is therefore the content of the CRS, not the arithmetization.

A quadratic CRS is costly, though: for a Zcash circuit with 2^{17} multiplication gates it "would be on the order of terabytes" [MBKM19, §1, p. 2]. The schemes below use a linear-size SRS of encoded powers of a secret point, as in KZG [KZG10]; Sonic and Marlin also include these powers scaled by a second secret.

Sonic

Sonic [MBKM19] has a linear-size universal and updatable SRS that needs no specialization for a given circuit [MBKM19, abstract, §1.1]. Sonic encodes the constraint system in a bivariate polynomial and commits with a variant of KZG. Its security theorems cover the interactive protocol (witness-extended emulation, with the commitment proven in the algebraic group model under 2d-DLOG); the protocol is made non-interactive by the Fiat–Shamir heuristic in the random oracle model [MBKM19, §3, p. 4, §6, pp. 8–9, Thms. 6.2–6.3]. The verifier needs the value of a sparse bivariate polynomial that describes the circuit. In the *unhelped* (succinct) mode, the prover proves that value correct with new permutation and grand-product arguments. This mode is Sonic's row in Tables 7.1 and A.1, and the one Marlin's authors compare against, since the helped mode "does not achieve succinct verification" [CHM+20, §1.1]. In the *helped* mode, an untrusted helper computes advice for a batch of proofs [MBKM19, §1.1]. Only the helped mode was implemented, with 256-byte proofs [MBKM19, §9, Table 4].

PLONK

PLONK [GWC19] keeps this setup but simplifies the arithmetization to the gate equation and copy constraints of §4.1, with wire values interpolated over a multiplicative subgroup H of roots of unity [GWC19, §6–7]. Its permutation argument for the copy constraints (§4.1) builds on Bayer and Groth [BG12], as Sonic's does, but runs over evaluations on H rather than coefficients of a bivariate polynomial [GWC19, abstract, §1.1]. The commitments are KZG [GWC19, §3.1; KZG10]. So $\mathcal{S}_1$ is the KZG SRS, and $\mathcal{S}_2$ commits to the selector and permutation polynomials (Table 3.1). Knowledge soundness is proven in the algebraic group model under 2d-DLOG [GWC19, Lemma 4.7], with a random oracle for Fiat–Shamir [GWC19, §8]. Proofs have $9\,\mathbb{G}_1 + 6\,\mathbb{F}$ elements ($7\,\mathbb{G}_1 + 6\,\mathbb{F}$ in the small-proof flavour), and verification takes 2 pairings (Table A.1; [GWC19, §1.2, Tables 1–2, p. 4]). Counting group exponentiations only, the prover does about 1.1 to 3 times Groth16's work and 5 to 30 times less than succinct Sonic's [GWC19, §1.2].

PLONK's analysis has since been moved out of the idealized group model. KZG is black-box extractable under ARSDH alone, which gives less efficient PLONK- and Marlin-style variants that are knowledge-sound in the random oracle model "without using any idealized group model" [LPS24, abstract, pp. 6–7, Cor. 1, p. 20, Cor. 3, p. 27]. For PLONK, the variant's argument is about 3.5 times longer, with 46 pairings instead of 2 [LPS25, abstract, §1, p. 3, Table 1, p. 6]. Interactive PLONK itself is computationally special-sound under the falsifiable ARSDH and SplitRSDH assumptions, so its Fiat–Shamir

version is knowledge-sound in the random oracle model without the algebraic group model, although SplitRSDH is justified in the AGM with oblivious sampling [LPS25, abstract]. Chiesa et al. re-derive this modularly, in a proof sketch [CGKY26, Cor. 1, p. 195, full version, §2.5, pp. 18–19]. Fuchsbauer and Sefranek remove the random oracle instead. Under a falsifiable "zero-testing" assumption on the hash and the commitment, together with the (n+2)-power-discrete-log assumption, PLONK is knowledge-sound in the AGM and computationally zero-knowledge in the standard model, which justifies its recursive use in the recursive AGM [FS26b, abstract, Thm. 5, p. 23, Thm. 6, p. 30]. PLONK's multilinear descendant HyperPlonk [CBBZ23] is discussed in §4.2 and §9.1.

Marlin

Marlin [CHM+20] makes the modular structure explicit. Its core is an algebraic holographic proof (AHP) for R1CS (§4.2), and its compiler (§4.5) turns the AHP and KZG into a preprocessing argument with a universal, updatable SRS [CHM+20, §1, Thm. 1]. The indexer is "a deterministic polynomial-time algorithm" [CHM+20, §7]. Marlin is therefore preprocessing but not circuit-specific, which is exactly the distinction drawn in Remark 1. A proof has $13\,\mathbb{G}_1 + 8\,\mathbb{F}$ elements (Table A.1). Marlin reports a prover more than 10 times faster than unhelped Sonic's and a verifier more than 3 times faster, by complexity rather than measured time, since no implementation of unhelped Sonic existed [CHM+20, §1.1, Fig. 2].

All three schemes were first proven secure in the algebraic group model of §3.1 (Marlin can also be based on concrete knowledge assumptions [CHM+20, abstract]). Compared with §5.1, the family gives up proof size for one reusable, updatable setup: proofs have 13 to 36 group and field elements, against Groth16's 3 group elements (Table A.1; [GWC19, Table 1, p. 4]). It is still pairing-based, so it is not post-quantum.

5.3 Transparent setup — Fractal, Spartan, Supersonic (DARK)

A zk-SNARK is *transparent* if neither $\mathcal{S}_1$ nor $\mathcal{S}_2$ uses secret randomness (Remark 1), so there is no trapdoor whose leakage would allow forged proofs. Transparency is decided by the cryptographic compiler, not by the information-theoretic protocol (Figure 4). PLONK's polynomial IOP, for example, needs a universal trusted setup with KZG but is transparent with a class-group commitment (Supersonic, below). This subsection follows the three transparent compilers that give strict zk-SNARKs in Table 7.1: hash-based IOPs (Fractal), sum-check with a transparent multilinear commitment ($\text{Spartan}_{\text{RO}}$), and groups of unknown order (Supersonic with class groups). The subsection then turns to the proven and conjectured soundness of FRI, on which the hash-based route rests, and ends with post-quantum security. Halo and Nova, also transparent, appear in §6.

Hash-based IOPs: FRI and Fractal

The hash-based route compiles an interactive oracle proof (IOP) with Merkle-tree commitments and Fiat–Shamir (§4.2). It needs no algebraic structure, and its non-interactive version rests on the random oracle (Table A.1). Its low-degree test is FRI [BBHR18b] or one of its successors, STIR and WHIR (§4.4), all of which give polylogarithmic proofs and verification.

STARK [BBHR18] has a polylogarithmic verifier only for uniform computations, so we discuss it with the conditional schemes in §5.4.

Fractal [COS20] meets the strict bound for arbitrary R1CS through *holography*, in which the verifier never reads the circuit (§4.2) [COS20, §1.1, p. 2]. Fractal compiles any holographic IOP into a preprocessing SNARG that is "unconditionally secure in the random oracle model" and in the quantum random oracle model [COS20, Thm. 1, p. 2].

In Remark 1's terms, $\mathcal{S}_1$ is empty apart from the oracle. $\mathcal{S}_2$ is the deterministic indexer, whose output, a Merkle root [COS20, §10.1, Constr. 10.3, p. 41], is an $O_\lambda(1)$ verification key: "anyone can publicly preprocess an index" [COS20, Thm. 3, p. 3]. At 128-bit security over a 181-bit prime field, proofs are 80–160 KB for up to millions of constraints, two orders of magnitude larger than pre-quantum preprocessing zk-SNARKs with trusted setup [COS20, p. 3]. Holography also makes Fractal recursion-friendly. Preprocessing simplifies recursive composition [COS20, §2.1], and with no elliptic curves there is no field mismatch for a curve cycle to resolve (§6). Recursing requires replacing the oracle with a concrete hash, which the authors call "a heuristic" [COS20, p. 3, fn. 3].

The sum-check route: Spartan

Spartan [Set20] encodes an R1CS instance as a degree-3 polynomial that decomposes into multilinear polynomials, and runs the sum-check protocol on it [Set20, §1, p. 4]. Checking the final sum-check claim would cost the verifier $O(n)$ for a statement of size n, because it must evaluate polynomials that encode the R1CS matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ (§4.1). Spartan's *computation commitments* move this to a one-time public preprocessing step. The verifier keeps a short commitment to those sparse polynomials, and "creating a computation commitment does not involve any secret trapdoors" [Set20, §1, p. 4]. This is a deterministic $\mathcal{S}_2$ in Remark 1's terms. Spark, a compiler built on offline memory checking (§4.3), then turns any extractable multilinear commitment into one for sparse polynomials without asymptotic overhead, and so keeps a time-optimal prover when the underlying commitment has one [Set20, §1, p. 4, §7.2, p. 25]. Spartan is a family, and the commitment decides where each variant falls [Set20, Fig. 2, p. 6]:

- $\text{Spartan}_{\text{RO}}$ is hash-based, plausibly post-quantum, and strict and transparent ($O_\lambda(\log^2|C|)$ proofs and verification, Table 7.1).
- $\text{Spartan}_{\text{KE}}$ is strict, but its "q-type, knowledge of exponent" commitment needs a universal private setup [Set20, §1, p. 5]. By its setup it belongs with the universal schemes of §5.2.
- $\text{Spartan}_{\text{DL}}$ uses Hyrax's discrete-log commitment and is transparent, but has $O_\lambda(\sqrt{|C|})$ proofs and verification (§5.4).

Groups of unknown order: Supersonic (DARK)

Bünz, Fisch and Szepieniec [BFS20] commit to polynomials by encoding them as integers in a group of unknown order. Their DARK commitment has $O(\mu \log d)$-size evaluation proofs for μ-variate degree-d polynomials, verifiable in $O(\mu \log d)$ time [BFS20, §1.1, p. 3]. Supersonic compiles the polynomial IOPs of Sonic, PLONK and Marlin with it [BFS20, §1.1, p. 4]. Whether the setup is trusted depends on the group. An RSA modulus "cannot be generated in a publicly verifiable way without exposing the order, and thus requires a trusted setup". The discriminant of a class group, by contrast, "can be generated from public coins" [BFS20, §3.1, pp. 8–9]. Supersonic is thus transparent with class groups, and a universal scheme with "constant-size setup parameters" with RSA groups [BFS20, §1.1, p. 3]. Its $O_\lambda(\log|C|)$ proofs and verification are the best asymptotics among the transparent schemes of this subsection (Dew, §9.1, has constant-size proofs but a quadratic prover). The only concrete figures are estimates: 10.1 KB proofs and about 72 ms verification for 2^{20} gates at 120-bit security [BFS20, §6, p. 28, Table 3, Table 4, p. 29].

Two caveats apply. First, the EUROCRYPT 2020 security proof had "a significant gap", found by Block et al., who know of no attack and note that the DARK authors found the same gap independently [BHR+21, abstract, §1.1, p. 6, §2.2, §5.1, Remark 5.3 and Claim 5.4, p. 21]. The revised version (June 2022) shows "the original protocol with a

slightly adjusted parameter is still secure", with extraction relying on the random order assumption [BFS20, abstract, §1.1, pp. 4–5, §3.1, p. 9]. The abstract and Theorem 5 still name the strong RSA and adaptive root assumptions [BFS20, Thm. 5, p. 25]. The revision drops the hiding commitment and the zero-knowledge evaluation, though its abstract and introduction still claim them [BFS20, abstract, §1.1, p. 3], so Supersonic's zero knowledge rests on the 2020 version, whose hiding, without a trusted setup (as with class groups), is only computational, under subgroup indistinguishability and argued informally [BFS20, 2020 ePrint version, §4.7, pp. 19–21]. Later work confirms the original gap: DARK's attempt to avoid range proofs for degree bounds "was shown to have security vulnerabilities", and its successors restored security with range proofs at the cost of an $O(n^3)$ prover, by these authors' count [PTX26, §1.2, fn. 4, p. 465]. Second, Supersonic "is not quantum-secure due to its reliance on groups of unknown order" [BFS20, §1.1, p. 4].

Proven and conjectured soundness of FRI

Concrete FRI parameters depend on which soundness bound one trusts (Figure 5). Theorem 2 of [BBHR18b] proves soundness only up to a proximity parameter $\delta_0 \geq \frac{1}{4}(1-3R)-1/\sqrt{|\mathcal{L}|}$ for rate R and block length $|\mathcal{L}|$. The authors conjecture that it extends to every $\delta \leq 1-R$, i.e. up to capacity [BBHR18b, §1.1.3, Conj. 3, p. 14:5]. Later work proved soundness arbitrarily close to the Johnson radius $1-\sqrt{R}$, over fields at least quadratically larger than $|\mathcal{L}|$ [BCIKS20, §1, p. 3, Thm. 8.3, pp. 40–41; Tha22, Thm. 10.4, p. 383]. Parameters that assume the conjecture need far fewer queries: according to Spartan, the default parameters of Aurora and Fractal "rely on non-standard conjectures related to Reed-Solomon codes" and use about 10× fewer FRI query repetitions than the proven ones [Set20, §9.1, p. 33]. Virgo's "100+ bits" also rests on the conjecture [ZXZS20, §5.2, p. 23].

Crites and Stewart [CS26] have since disproved the up-to-capacity versions of the correlated-agreement conjecture of [BCIKS20, Conj. 8.4, p. 42], of WHIR's mutual-correlated-agreement conjecture and of the list-decoding conjecture for DEEP-FRI (a FRI variant that samples outside the evaluation domain) [CS26, abstract, p. 128, p. 129]. Diamond and Gruen gave further counterexamples for rates tending to 0 [DG26b, abstract, p. 96, Thm. 5, p. 124], and Krachun, Kazanin and Haböck near capacity over the smooth multiplicative subgroups of prime fields that FRI uses, for primes growing with the block length [KKH26, Thm. 1, p. 3]. Since FRI's soundness "is analyzed under the assumption of correlated agreement", these results close one route to FRI's own Conjecture 3 but do not disprove it, and whether "a polynomial-time attack on FRI-based SNARKs" exists is open [CS26, pp. 129, 131].

Over small base fields, however, Fenzi and Sanso find that parameters for which "the capacity conjectures above predict 128 bits of security" admit an attack on a toy protocol that abstracts a step of such systems. The attack is inefficient but succeeds with probability at least about $2^{-116.5}$. They also find that systems deployed with these conjectures, such as SP1 Turbo, RISC Zero and Plonky2, lose 2.8 to 9.0 bits in the query phase; SP1's successor, Hypercube, no longer relies on them (§8). The toy protocol abstracts WHIR-style systems, but their list-size bounds "also affect FRI-based systems" [FS26, abstract, §1.1, §1.2, p. 3, p. 10, Remark 2, p. 12, Table 2, p. 14]. Conjectured security is therefore conditional.

On the proven side, Bordage, Chiesa, Guan and Manzur prove mutual correlated agreement for Reed–Solomon codes up to the Johnson radius for every polynomial generator, leaving open whether it holds "between the list-decoding and capacity bounds" [BCGM26, Thm. 5, p. 24:5, p. 24:6]. Ben-Sasson et al. shrink the exceptional set of Johnson-radius correlated agreement to $O_R(|\mathcal{L}|/\eta^5)$, where η is the gap below the Johnson radius (the source's $O_\rho(n/\eta^5)$, with ρ the rate), and state a bound of the same order for the mutual form, with only a proof sketch [BCHKS26, abstract, Thm. 1.5, p. 9, Thm. 4.6, pp. 28–29].

Going beyond the Johnson radius with a small error probability would require better list decoding [CS26, abstract, p. 128, Thm. 2, p. 146; BCHKS26, Thm. 1.9, p. 11], and the

results past it are restricted. Dao, Kominers and Thaler's claimed bounds need large field characteristic. Against the shipped parameters of a deployed ProveKit proof, at unchanged per-component security allocations, they would save 79.4 KB (11.1%), although optimized Johnson-regime parameters already allow part of this saving [DKT26, abstract, p. 8, §8, pp. 70–77, §8.2, p. 72]. Over binary fields, proximity gaps beyond the Johnson radius fail. On dense additive domains, Binius64's among them, Dao, Kominers, Thaler and Zheng give rate-1/4 pairs of words with common agreement 1/4 whose combinations, for superpolynomially many challenges, agree with a codeword on nearly half the positions, just below the Johnson threshold. This rules out a $\text{poly}(|\mathcal{L}|)/|\mathbb{F}|$ bound there, and weaker counterexamples cover every additive domain [DKTZ26, abstract, §1.1, pp. 4–7, Cor. 5.2, p. 37, Thm. 5.4, p. 38]. None of the systems they examine relies on such gaps, so the result rules out proof-size savings, not the security of current parameters [DKTZ26, p. 4, fn. 2, §7, p. 65].

Goyal and Guruswami's results cover folded Reed–Solomon, multiplicity and randomly evaluated Reed–Solomon codes, not the smooth subgroups FRI uses [GG26, Thms. 1.2–1.3, pp. 2–3; Jo26, p. 4]. For prescribed domains, a preprint claims mutual correlated agreement a fixed number of integer error steps past the Johnson radius, a relative gain of only $O(1/n)$ for block length n [Jo26, abstract, Thm. 1.1, p. 3, p. 5]; another claims it up to capacity with a fixed slack over prime fields, with unquantified constants and an error of n^C/q that is meaningful only for fields much larger than the block length [Jer26, Thm. 1.2, p. 4, p. 32]. For general linear codes, Zeilberger's Khatam proves a relaxed form of the mutual-correlated-agreement property that BaseFold and WHIR rely on, over every finite field, but only for agreement above about a $(1-\Delta)^{1/3}$ fraction of the block (relative distance Δ), well short of capacity [Zei26, abstract, Thm. 1, Remark 1]. Kominers, Thaler and Zheng show it is tight for general linear codes [KTZ26, Thm. 1.1, p. 3]. Even this bound roughly halves BaseFold's verifier repetitions at low rates [Zei26, Table 1, p. 608].

Some new schemes avoid the conjectures altogether: Baweja et al.'s FICS, an IOP of proximity (IOPP, §4.4) for linear-time-encodable codes with an $O(n)$ prover and $O(\lambda \log\log n + \log n)$ queries for block length n, is analyzed within the unique-decoding radius [BMMS26, Thm. 3, pp. 571–572, §2.2, p. 568, §2.7, fn. 10, p. 577].

Post-quantum security separates the three routes. Hash-based schemes are only *plausibly* post-quantum [Set20, Fig. 2 caption, p. 6]: STARK calls its post-quantum claim informal. The stand-in it offers is "reliance only on collision resistant hash functions", which "covers only the interactive setting" [BBHR18, §1.3.1, fn. 12, p. 9], because the non-interactive STARK also assumes a random oracle [BBHR18, §3.3, p. 22]. Fractal's proof in the quantum random oracle model [COS20, Thm. 1, p. 2] says nothing about the concrete hash. Discrete-log schemes ($\text{Spartan}_{\text{DL}}$, Bulletproofs, Halo, Nova) and Supersonic are not post-quantum.

5.4 Zero-knowledge arguments outside the strict definition

Several systems that call themselves zk-SNARKs under the looser notions of succinctness quoted in §1, among them Hyrax and Aurora, are zero-knowledge arguments whose verifier is not polylogarithmic in $|C|$, or is polylogarithmic only for restricted circuits. Table A.2 (Appendix A) lists them with the reason each misses the bound: a linear verifier (Bulletproofs, Aurora), a quasi-linear one (Ligero) or one that is not sublinear (LaBRADOR), sublinear but not polylogarithmic proofs and verification (Hyrax, $\text{Spartan}_{\text{DL}}$), or a polylogarithmic verifier only for uniform computations (STARK) or for low-depth circuits that are log-space uniform (Libra) or regular (Virgo) [BBB+17, abstract; BCR+19, abstract; AHIV17, abstract; BS23b, §1, p. 520; WTS+18, abstract; Set20, Fig. 2, p. 6; BFS20, §1, fn. 1, p. 2; XZZ+19, abstract, Thm. 6, p. 26; ZXZS20, abstract, Thm. 3, p. 20]. Many have what strict schemes lack: very fast provers, short transparent proofs, or plausible post-quantum security. The two paragraphs below cover LaBRADOR's zero knowledge

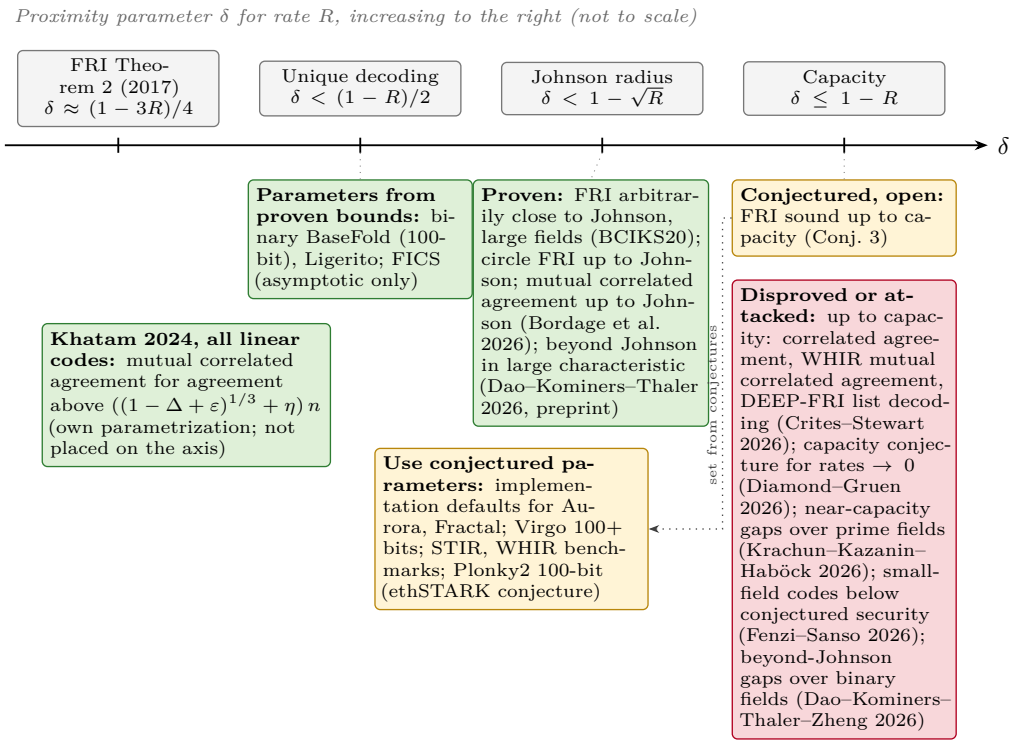


Figure 5: What is proven, conjectured and disproved about Reed–Solomon proximity. The top row orders the proximity parameter δ for rate R, from FRI's original proven bound through the unique-decoding and Johnson radii to capacity. Green results are proven, amber are conjectured and open, and red were disproved or attacked [CS26; DG26b; KKH26; FS26; DKTZ26]. Schemes on the left set concrete parameters from proven bounds (FICS is analyzed only asymptotically, within the unique-decoding radius); the box at the end of the dotted arrow lists schemes that use conjectured parameters, whose conjectures differ in detail. Khatam's bound holds for every linear code, not only Reed–Solomon codes, and is stated in its own parametrization. Sources: [BBHR18b, Thm. 2, Conj. 3], [BCIKS20], [HLP24, Thm. 6], [DP26, §5.2], [NA25, §6.3], [BMMS26], [Zei26, Thm. 1], [BCGM26, Thm. 5], [DKT26, Thms. 1.1–1.2], [CS26], [DG26b, Thm. 5], [KKH26, Thm. 1], [FS26, §1.1], [Set20, §9.1], [ZXZS20, §5.2, p. 23], [ACFY24, §6.2], [ACFY25, §6.2, p. 44, §6.3.2, p. 49], [Pol22, §8].

and the related schemes that have no row, and a Fiat–Shamir attack on the GKR-based rows.

Circle STARKs have no row in Table A.2 because they are not zero-knowledge as presented: they carry STARK's design to Mersenne-31 (§4.1), but their paper gives zero knowledge one sentence, saying it comes "in the usual manner", from randomizing the trace and FRI as in Aurora, with the details left to other work [HLP24, §5.4, p. 27]; no zero-knowledge version is specified (§3). Their FRI analogue is proven sound up to the Johnson radius in the oracle model [HLP24, Thm. 6, p. 38]. LaBRADOR (§4.4) has a row: it misses the bound, with very short proofs but a verification time that is not sublinear [BS23b, §1, p. 520], and its main construction is not zero-knowledge, but the paper specifies a zero-knowledge variant, a masking shim with blinded commitments, that it describes and costs (60.33 KB against 49.02 KB for binary R1CS with 2^{20} constraints) without proving it [BS23b, §6, pp. 539–541].

Brakedown and Shockwave have no row either. Their implementations are not zero-knowledge, and for Shockwave the paper says only that it "can be rendered zero-knowledge using standard techniques" [GLS+23, §1.1, p. 5]. For Brakedown it does specify zero-knowledge versions, by one layer of proof composition with $\text{Spartan}_{\text{RO}}$, Xiphos or Groth16 as the outer zkSNARK [GLS+23, §7, p. 35, Fig. 9, p. 53], but their proof and verifier are the outer scheme's, so they would duplicate existing rows; their codes appear as polynomial commitments in §4.4 (Table 7.2). The GKR-based rows, finally, carry the Fiat–Shamir attack of §4.2: Khovratovich, Rothblum and Soukhanov target the argument used in Hyrax and Libra [KRS25, §1, p. 3 and fn. 3], and Virgo has the same structure, so we infer that it is affected too.

6 Recursion, accumulation and folding

Recursion is orthogonal to the setup model of §5: it is a layer on top of a base argument. Halo and Nova need no trusted setup [BGH19, abstract; KST22, abstract], while pairing-based SNARK recursion and SnarkPack aggregation do (§6.1, §6.4). We include these constructions for two reasons. First, recursion is how zk-SNARKs are applied to long or unbounded computations. Second, although Nova is an IVC scheme rather than a zk-SNARK, its compressed proof in the SXDH variant meets the bound of §3 per step, with the step circuit F in place of C, at a cost independent of the number of steps. Table A.2 lists Nova because it is a folding scheme, not because it misses the bound (§7). Section 6.1 defines IVC and SNARK-based recursion, §6.2 covers Halo and accumulation schemes, §6.3 folding with Nova, and §6.4 the generalizations of folding, recursion without a curve cycle and aggregation.

6.1 Incrementally verifiable computation

Valiant defined *incrementally verifiable computation* (IVC): a long computation periodically outputs its state with a short proof of correctness, the prover stays close to the computation's own time and space, and the verifier's cost is "essentially constant" [Val08, abstract, §1, pp. 1–2]. His construction merges proofs recursively. Proofs of knowledge compose ("A is convinced that B is convinced"), whereas merely sound proofs only assert that a convincing proof exists [Val08, §1.3]. SNARK-based recursion proves "the t-th step ... was executed correctly, and there exists a proof" for $t-1$ steps, with the SNARK verifier inside the circuit [BCMS20, §1, p. 2; BCCT13, §1.4; BCTV14, §1.4.1]. Security rests on recursive knowledge extraction, which Bitansky et al. prove for constant depth [BCCT13, Thm. 6.1, pp. 28–29], using wide proof trees for long computations [BCCT13, Thm. 8.1, p. 46]; a linear-size extractor would extend it to logarithmic depth [BCCT13, Remark 6.3, p. 29]; Ben-Sasson et al. [BCTV14] compose along the chain heuristically. Recursion requires a verifier sublinear

in its statement; otherwise the statement grows exponentially in t. Efficiency also requires verifier arithmetic native to the circuit's field, since emulating a foreign field costs about a $\log q$ factor [BCTV14, §3.1, p. 9]. For pairing-based SNARKs this meant cycles of pairing-friendly curves [BCMS20, §1, pp. 2, 5] (MNT4/MNT6 in [BCTV14, §3.2]), "an expensive algebraic object" [COS20, abstract]. For SNARKs made non-interactive by Fiat–Shamir, the hash must then be computed inside the circuit, which makes security heuristic (§3.1); pairing-based recursion avoids this but rests on knowledge-of-exponent assumptions.

Efficient pairing-friendly cycles remain scarce. At 128-bit security the MNT cycle needs $p \approx q \approx 2^{1000}$ by Guillevic's estimate, and newer cycles from higher-dimensional abelian varieties are suboptimal next to an ideal 2-cycle [CCN24, abstract, §1, pp. 222–225].

The in-circuit hash is also provably hard to avoid. Hall-Andersen and Nielsen prove Valiant's conjecture under mild extra conditions: if collision-resistant hash functions exist, no zero-knowledge, knowledge-sound, succinct non-deterministic IVC exists in the random oracle model, even if its security also uses standard-model assumptions, provided the zero-knowledge simulator only programs the oracle [HN23, Thm. 1, Remark 3, p. 21]. Barbara, Chiesa and Guan show that no succinct argument in the random oracle model can prove computations that themselves query the oracle, so relativized IVC and proof-carrying data (PCD, a generalization of IVC; §6.2) do not exist there either [BCG25, abstract, §1.1]. Chen et al. leave the model: once an idealized low-degree arithmetization of the hash is added, zero-knowledge PCD of any depth follows from Micali's SNARK and collision-resistant hashing, though a concrete hash again makes it heuristic [CCG+23, Thm. 1, p. 4, Thm. 9.4, p. 53]. Whether ordinary IVC exists in the random oracle model without Hall-Andersen and Nielsen's extra conditions is open [BCG25, §1.1, p. 5]. IVC from indistinguishability obfuscation, without a random oracle, is in §9.3 (Figure 6).

6.2 Halo and accumulation schemes

Halo [BGH19] removes both the trusted setup and the pairing-friendly cycle. It uses a Bulletproofs-style inner-product commitment under the discrete-log assumption over a cycle of ordinary 255-bit curves [BGH19, abstract, §1.1]. The commitment's verifier has one linear-time step. *Nested amortization* keeps that step out of the recursive circuit: its input and claimed output become public inputs, and successive claims are merged, so the linear-time check "is thus only performed once at the end of the recursive chain" [BGH19, §1.1, pp. 3–4]. Fully-recursive proofs are 3.5 KiB [BGH19, §1.2, p. 4]. The cost is the final check: Halo is "not fully succinct (in that the verifier's work is linear in the circuit size)" [BGH19, §1.2, p. 4]. By §3's bound, too, a Halo proof is outside the strict definition.

Halo's authors prove their SNARK secure but not its recursive technique: they "do not prove that it satisfies any security property" [BCMS20, abstract, §1, p. 2]. Bünz, Chiesa, Mishra and Spooner fill this gap. They define *accumulation schemes*, with a prover, a verifier and a decider, and show that any SNARK with a sublinear accumulation verifier yields PCD, which generalizes IVC, "even if the argument itself does not have a sublinear-time verifier" [BCMS20, abstract, Thm. 1, p. 3]. Their concrete accumulation schemes are proven in the random oracle model, so the resulting PCD is heuristic [BCMS20, §1.1, Thm. 3, pp. 3–5]. Halo's successor Halo2 pairs PLONK's IOP with an inner-product commitment [Tha22, §19.1] (Figure 4).

6.3 Folding: Nova

Nova [KST22] drops SNARKs from intermediate steps. A *folding scheme* "reduces the task of checking two instances in some relation to the task of checking a single instance" [KST22, abstract]. R1CS is quadratic, so a random combination of two R1CS instances is not an R1CS instance. Nova instead folds *relaxed* R1CS, $(\mathbf{A}Z) \circ (\mathbf{B}Z) = u \cdot (\mathbf{C}Z) + E$,

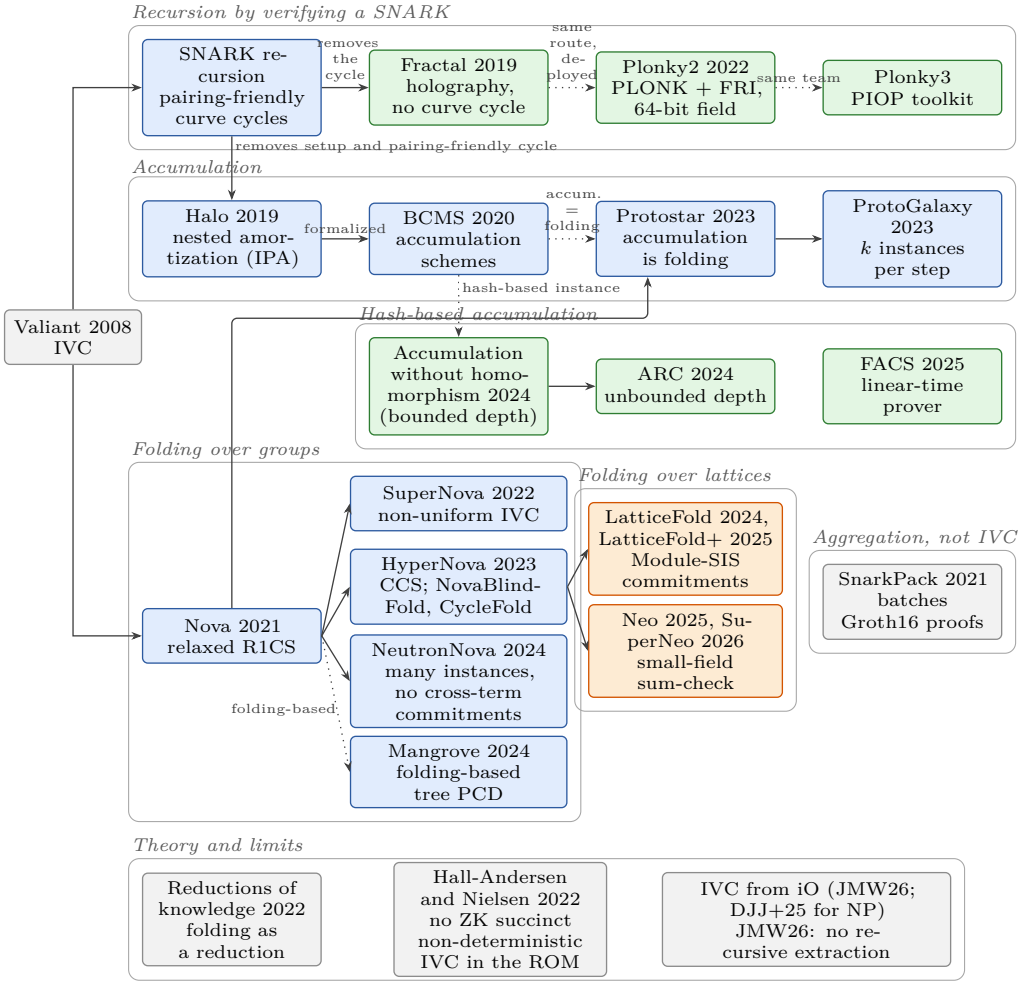


Figure 6: Recursion, accumulation and folding. Arrows mark a construction that generalizes, formalizes or replaces the earlier one; dotted arrows mark a looser relation named on the edge. Blue boxes are constructions over discrete-log or pairing groups (Figures 1 and 4 colour pairings red; here the two share one colour), orange boxes use lattices and green boxes hashes, the last two plausibly post-quantum. Grey boxes are theory or batching rather than IVC constructions, and the iO-based IVC results are discussed in §9.3. Sources: [Val08], [BCTV14], [BCMS20], [COS20], [Pol22], [Plonky3], [BGH19], [BC23], [EG23], [KST22], [KS22], [KS24], [KS26], [NDC+24], [BC25a], [BC25b], [NS26], [BMNW25b], [BMNW25], [BMMS26], [GMN22], [KP23], [HN23], [BCG25], [JMW26], [DJJ+25].

with the R1CS matrices of §4.1 and Nova's assignment vector Z, where the scalar u and the error vector E absorb the cross terms ($u = 1$, $E = 0$ is plain R1CS). The folding scheme is zero-knowledge, with a prover linear in the instance size (Nova's $O_\lambda(N)$ for N-sized instances, not the size bound N of §3) and $O_\lambda(1)$ verifier and communication, from any additively homomorphic commitment such as Pedersen's [KST22, Thm. 1, p. 3]. Each IVC step folds the previous instance into a running one. The recursive circuit is then "dominated by two group scalar multiplications", and the prover's work per step by two multi-exponentiations of size $O(|F|)$ [KST22, abstract]. The IVC proof has size $O(|F|)$. A Spartan-based zk-SNARK compresses it to $O(\log|F|)$, verified in time $O(\log|F|)$ or $O(|F|)$ depending on the commitment [KST22, Thm. 2, p. 4]; the logarithmic case is the SXDH variant [KST22, Fig. 2, p. 7]. Nova is not itself a zero-knowledge IVC scheme: the IVC proof does not hide the step witnesses, and the compressed proof is proven zero-knowledge only for honestly generated IVC proofs [KST22, §5, p. 17, Thm. 4, p. 22]. Nova needs "neither … a trusted setup nor FFTs", and its Fiat–Shamir step is instantiated "(heuristically)" with a concrete hash [KST22, abstract, §1.1, p. 3].

The deployed system diverged from the analyzed one. The paper analyzes a single curve group, but the implementation ran a 2-cycle scheme that was "only described in the code". Nguyen, Boneh and Setty forged a proof of 2^{75} rounds of the MinRoot verifiable delay function in 116 ms, and then gave a proven, more efficient fix that is now in use [NBS23, abstract, §7.3.1, p. 18:18, Thm. 9, p. 18:14]. They also show that Nova's IVC proofs are malleable, so an attacker could swap the payment address in a proven output [NBS23, §8, p. 18:19] (§3, Remark 3).

6.4 Generalizations

The folding line has been generalized in several directions, which Figure 6 arranges by the algebra each construction uses; the first two paragraphs below follow that split. The last two cover related routes: recursion without a curve cycle, and aggregation of independent proofs.

Over groups. SuperNova gives *non-uniform* IVC, in which a step costs only "the size of the circuit representing the instruction invoked" [KS22, abstract]. HyperNova folds CCS and adds zero knowledge through NovaBlindFold, and an efficient instantiation over a curve cycle through CycleFold [KS24, abstract]. Protostar compiles special-sound protocols into non-uniform IVC for PLONK with high-degree gates and lookups, and observes that its accumulation scheme "is also naturally a folding scheme as defined in Nova" [BC23, abstract, Remark 2, p. 20]. ProtoGalaxy folds k instances into an accumulator in one step; it is proven knowledge-sound as an interactive protocol, with its non-interactive version, like Protostar's, justified only in the random oracle model, and it does not treat zero knowledge [EG23, §1.2, pp. 3–4, Table 1, p. 4, §3.1, p. 5, §3.3, p. 7, Thm. 4.3, p. 10]. NeutronNova's recursive verifier performs three group scalar multiplications and a constant number of hashes, independent of the circuit size. It folds many instances at once, at a cost linear in their number, without Nova's cross-term commitments, and folds two 2^{20}-constraint SHA-256 instances in 91 ms, against 891 ms for Nova [KS26, abstract, §1, Fig. 2, p. 528]. Mangrove uses folding for tree-based PCD with a two-pass prover whose memory is $O(1)$ for suitable parameters; its estimated 34 MB proof shrinks to 12 KB after compression with Spartan [NDC+24, abstract, §1, p. 311, §6.3, p. 339]. Kothapalli and Parno's *reductions of knowledge* give the formal language in which folding is a reduction from two instances to one [KP23, Def. 1, p. 671, Thms. 1–2, p. 672, Def. 4, p. 680].

Over lattices and hashes. LatticeFold and LatticeFold+ replace Pedersen with Module-SIS (Ajtai) commitments, for plausibly post-quantum folding over 64-bit fields [BC25a,

abstract; BC25b, abstract, §1, pp. 6–7]; LatticeFold proves knowledge soundness only for the interactive protocol and leaves the Fiat–Shamir version to future work [BC25a, Remark 3.3]. Neo and SuperNeo are lattice-based folding schemes for CCS, the relation HyperNova folds; they run sum-check field-natively, without ring arithmetic, over small fields such as Goldilocks ($p = 2^{64} - 2^{32} + 1$, Table 8.1) [NS26, abstract] (§8). The first hash-based accumulation scheme, from Merkle commitments with spot checks on codewords, "only supports a bounded number of accumulation steps", since a cheating prover can move a bad codeword to a good one over enough steps [BMNW25b, abstract, §2.1 and Remark 3, pp. 23:9–23:10]; ARC removes this bound [BMNW25, abstract, §1.2]. FACS gives an accumulation scheme for NP in the random oracle model with a linear-time prover [BMMS26, Thm. 2]. WARP, concurrent with it, is hash-based with linear prover time, logarithmic verifier time and unbounded depth; its reductions work for any linear code over a sufficiently large field, and the prover is linear when that code is linear-time encodable [BCFW25, abstract, §1.1, pp. 4–5, Remark 1.1, p. 7].

Recursion without a curve cycle. Fractal's cycle-free route (§5.3) was deployed in Plonky2, which compiles an arithmetization based on TurboPLONK (PLONK with custom gates) with FRI over the 64-bit field $p = 2^{64} - 2^{32} + 1$ and reports a recursion threshold of 2^{12} gates, against Fractal's 2^{21} constraints [Pol22, abstract, §1, p. 3, §8, p. 14]. Its 100-bit setting relies on the ethSTARK soundness conjecture for FRI, a conjectured bound beyond the proven ones of §5.3 [Pol22, §8, fn. 8, p. 14]. Plonky2 is now deprecated in favour of the same team's Plonky3 [Plonky2], a toolkit for STARK-based zkVMs (§8). For Supersonic, by contrast, Halo's authors write that "it is not clear ... if recursion can be practically achieved" [BGH19, §1.2, p. 4].

Aggregation. When the goal is only to check many independent proofs at once, aggregation is a lighter alternative. SnarkPack combines n Groth16 proofs for the same verification key into one argument of size $O(\log n)$ using inner pairing product arguments, with keys taken from two existing powers-of-tau ceremonies; its verifier does $O(\log n)$ group and pairing work but still processes every public input. It aggregates 8,192 proofs in 8.7 s into a 40 KB proof [GMN22, abstract, §1, pp. 4, 6, §5, pp. 15–18]. Unlike recursion, this batches a fixed set of proofs rather than giving incrementally verifiable computation.

7 Comparison

This section compares the schemes of §5, Nova from §6, and ten later schemes (HyperPlonk, Orion, LaBRADOR, Polymath, Plonky2, HyperNova, Samaritan, Dew, Garuda and zk-Pari; §5 says where each is discussed). The discussion after the tables draws out four trade-offs: proof size, setup, prover cost, and post-quantum security with its assumptions. Only zero-knowledge schemes are compared, and every cell names its source. Table 7.1 lists the zk-SNARKs that meet the strict definition of §3, with every cost an $O_\lambda(\cdot)$ bound in the circuit size $|C|$, and Table 7.2 compares the polynomial commitment families of §4.4. Appendix A holds two more tables: Table A.2 lists the zero-knowledge arguments that miss the strict bound, or meet it only for restricted circuits, with the reason for each, and Table A.1 gives the exact counts behind Tables 7.1 and A.2 in each paper's own units, with setup models, assumptions and post-quantum status. Orion is in Table A.2 because its theorem gives only a linear verifier, and the revision cited was later shown unsound (§9.1); Plonky2, because its whitepaper states no bounds and it is the one exception to the zero-knowledge rule of §3; HyperNova, like Nova, because it is a folding scheme rather than a zk-SNARK; and zk-Pari, because its knowledge soundness is proven only for circuits that meet a linear-independence condition (§9.1). Measured sizes and times are not tabulated, since the papers use different hardware, curves, security levels and benchmarks.

Table 7.1: **zk-SNARKs meeting the strict definition (costs in $|C|$).** Every entry is $O_\lambda(\cdot)$ in $|C|$, the number of gates or constraints of C, hiding poly(λ) factors; $|x|$, $|w|$, d and G are the statement size, witness size, circuit depth and width (Table 7.1's cost columns use only $|x|$; the others appear in Table A.2, and where Table A.1 or a Notes cell quotes a paper's own d, as for Supersonic, HyperPlonk and HyperNova, it keeps that paper's meaning). "Setup params" counts what setup generates with fresh randomness (gp plus any randomized part of $\mathsf{S_p}$); deterministic encodings of C do not count, so a scheme is fully succinct exactly when this column is $O_\lambda(1)$. "Setup model; assumptions; PQ" gives the setup model of §5, the main hardness or idealized-model assumptions, and whether the scheme is plausibly post-quantum; Table A.1 states each in full with its source. "Derived" means computed from the paper's own counts; blank means the source gives no bound. For the universal schemes the SRS has size $O_\lambda(N)$ for the size bound $N \geq |C|$ of §3, written $O_\lambda(|C|)$ for $N = \Theta(|C|)$.

Scheme	Setup model; assumptions; PQ	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
GGPR	circuit-specific (trusted); q-PDH + d-PKE; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\|)$	verifier $O_\lambda(1)$ if the statement is hashed	[GGPR13, abstract, §2.4, p. 16]; [Set20, Fig. 2]
Pinocchio	circuit-specific (trusted); d-PKE, q-PDH, 2q-SDH; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log^2 \|C\|)$	$O_\lambda(\|x\|)$	prover from the interpolation for $h(X)$ (p. 4; §4.2.1, p. 8; see Prover cost); exact counts in Table A.1	[Gro16, Table 2]; [PHGR13, p. 4]
Groth16	circuit-specific (trusted); generic group model; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\|)$	exact counts in Table A.1	[Gro16, Table 2]; [CHM+20, Fig. 1]
Sonic	universal, updatable; AGM + ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	unhelped (succinct) mode; the deterministic index proving key is not counted	[MBKM19, Table 2]; [GWC19, Table 1]; [CHM+20, Fig. 1]
PLONK	universal, updatable; AGM + q-DLOG + ROM, or falsifiable ARSDH + SplitRSDH in the ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	verifier **derived**: $Z_H(z) = z^n - 1$ costs $O(\log n)$ and PI(z) sums ℓ terms	[GWC19, §1, Table 1, §8]; [LPS25]
Marlin	universal, updatable; AGM + ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	universal SRS for the size bound $N \geq \|C\|$ of §3; the deterministic index proving key is not counted	[CHM+20, abstract, Fig. 1, Thm. 2]
Fractal	transparent; ROM; plausibly PQ	$O_\lambda(1)$	$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	verifier bound after a transparent, deterministic preprocessing of C	[COS20, Thm. 3]
Spartan$_{\mathrm{KE}}$	universal (trusted); q-type knowledge of exponent + ROM; not PQ		$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	universal *private* setup (q-type, knowledge-of-exponent PCS); its size is not stated	[Set20, Fig. 2]
Spartan$_{\mathrm{RO}}$	transparent; ROM (hash-based PCS); plausibly PQ	$O_\lambda(1)$	$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	setup **derived**: Spartan's only hash-based commitment option, Virgo-VPD, needs only the hash function	[Set20, Thm. 5.1, Figs. 2, 4–6]

continued on next page

Table 7.1, continued

Scheme	Setup model; assumptions; PQ	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
Supersonic (DARK)	transparent with class groups, trusted with RSA; strong RSA + adaptive root; not PQ	$O_\lambda(1)$	$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|x\|+\log\|C\|)$	class groups transparent, RSA trusted; univariate PIOPs ($\mu=1$) in the $O(\mu\log d)$ bounds	[BFS20, §1, §4.3, Thm. 5]; [Set20, Fig. 2]
Dew	transparent with class groups; generic group model + ROM; not PQ	$O_\lambda(1)$	$O_\lambda(1)$	$O_\lambda(\|C\|^2)$	$O_\lambda(\|x\|+\log\|C\|)$	prover as in Cors. 1–2 (Table A.1); verifier **derived** (plus the public input); zero knowledge via the [CHM+20] compiler	[AGL+22, §7.1, Cors. 1–2, Fig. 14]
HyperPlonk (multilinear KZG)	universal (trusted); those of the PCS + ROM; not PQ		$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+\log\|C\|)$	universal SRS, size not stated; prover linear for a fixed gate degree; verifier **derived** (sum-check messages in the clear, one batched KZG opening)	[CBBZ23, §1, Thm. 4.1, Remark 4.2, Table 2, §3.8, §6.2, p. 41, Cor. A.4]
Polymath	circuit-specific (trusted); AGM with oblivious sampling, PDL + TOFR, ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|x\|\log\|x\|)$	per-circuit SRS; prover **derived** from a $\Theta(n\log n)$ KZG prover (§1, p. 173)	[Lip24, §1, pp. 173, 175, Table 1, §5, pp. 199–202]
Samaritan	universal, updatable; q-DLOG, AGM + ROM; not PQ		$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+\log\|C\|)$	verifier **derived** (Thm. 5.1 plus the public input); zero knowledge from cited compilers (Lemma 5.3), not proven	[GPS25, Table 1, p. 5, Thm. 5.1, §5.4]
Garuda	circuit-specific (trusted); AGM + ROM; not PQ		$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+\log\|C\|)$	verifier **derived** (plus the public input); zero knowledge specified, not proven	[DMS26, §1, p. 5792, §5, p. 5798]

Table 7.2: **Polynomial commitment families (§4.4).** Costs are for one evaluation proof for a polynomial with s coefficients; in a compiled zk-SNARK, $s = O(|C|)$. Rows from Thaler's Table 16.1 are univariate; "Strict" says whether a holographic PIOP compiled with the scheme meets the bound of §3. Blank = the source gives no such bound.

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Pairing	KZG (Sonic, PLONK, Marlin)	trusted, universal, updatable SRS of powers g^{τ^i}	1 group element	$O(1)$ group elements	$O(1)$ group ops + 2 pairings	t-SDH (binding); AGM for extraction	no	yes	[KZG10, §1, p. 2]; [Tha22, p. 545]; [GWC19, §3.1, p. 12, Lemma 4.7]

continued on next page

Table 7.2, continued

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Pairing, multilinear	multilinear KZG (HyperPlonk)	universal SRS	constant	$O(\log s)$ $\mathbb{G}_1$ elements; 0.8 KB at $s = 2^{25}$ (128-bit security)	$O(\log s)$ pairings	bilinear groups (no assumption named in the table)	no	yes	[CBBZ23, Table 2, p. 15]; [Tha22, p. 545]
Discrete log	Bulletproofs IPA (Bulletproofs, Halo)	transparent: $O(s)$ random group elements	1 group element	$O(\log s)$ group elements	multi-exp. of size $O(s)$	discrete log	no	no (linear verifier)	[Tha22, Table 16.1]; [BBB+17, §3]
Discrete log	Hyrax-commit (Hyrax)	transparent: $O(\sqrt{s})$ random group elements	$O(\sqrt{s})$ group elements	$O(\sqrt{s})$, or $O(\log s)$ with Bulletproofs	multi-exp. of size $O(\sqrt{s})$	discrete log	no	no ($\sqrt{s}$)	[Tha22, Table 16.1]; [WTS+18, §6.1]
Pairing, transparent	Dory	transparent: $O(\sqrt{s})$ random group elements; $O(\sqrt{s})$-exp. preprocessing	1 group element	$O(\log s)$ group elements	multi-exp. of size $O(\log s) + O(1)$ pairings	SXDH (standard model, interactive); ROM after Fiat–Shamir	no	yes, in principle	[Lee21, abstract]; [Tha22, Table 16.1]
Hash, RS proximity	FRI; STIR, WHIR (STARK, Aurora, Fractal, Virgo)	transparent: hash function only	1 hash	$O(\lambda \log^2 s)$ hashes (FRI's bound; STIR and WHIR need fewer queries)	$O(\lambda \log^2 s)$ hash evaluations	collision-resistant hash / ROM	yes	yes (STARK: uniform only)	[Tha22, Table 16.1]; [ACFY24]; [ACFY25]
Hash, linear codes	Ligero-commit, Brakedown, Binius	transparent: hash function (Brakedown: also a code sampled from a public seed)	1 hash	$O(\sqrt{s\lambda})$ field elements (Binius: $\tilde{O}(\sqrt{\lambda s})$ bits)	$O(\sqrt{s\lambda})$ field ops (Binius: $\tilde{O}(\sqrt{\lambda s})$ bit ops)	collision-resistant hash / ROM	yes	no ($\sqrt{s}$)	[Tha22, Table 16.1, §10.5]; [GLS+23]; [DP25, Thm. 3.14]
Hash, linear codes	Orion-PC (current revision unsound [DS25]); repair: Scorpius	transparent		$O(\log^2 s)$ (Scorpius)	$O(\log^2 s)$ (Scorpius); Orion-PC: $O(\sqrt{s})$, from a $\sqrt{s}$-size challenge	collision-resistant hash / ROM, plus a knowledge-sound, zero-knowledge outer commit-and-prove SNARK (Scorpius)	yes, with a plausibly post-quantum outer SNARK	Orion: no (unsound); Scorpius: commitment proven relative to the outer SNARK, full argument unproven	[DS25, abstract, p. 5, Thm. 4, §4.6, p. 18]; [XZS22, Table 1, p. 3, §4.2, p. 22]

continued on next page

Table 7.2, continued

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Hash, foldable codes	BaseFold (used with HyperPlonk in [ZCF24])	transparent: hash function; random foldable code from public randomness	1 hash	polylog(s)	$O(\log^2 s)$	collision-resistant hash / ROM (soundness is proven for the interactive protocol; round-by-round soundness, which would suffice for Fiat–Shamir, is not proven)	yes	yes (ZK not discussed)	[ZCF24, abstract, §5.1, p. 155]
Hash, binary fields	binary BaseFold with ring-switching (informally "FRI-Binius")	transparent: hash function only	1 hash	polylog(s)	$O(\lambda \log s)$ field + $O(\lambda \log^2 s)$ hash ops	collision-resistant hash / ROM	yes	yes (ZK not discussed)	[DP26, §4.4, p. 44]
Hash, RAA codes	Blaze	transparent: hash function; randomized code parameters, which miss the target distance with a one-time probability, e.g. $2^{-27.4}$	1 hash	$O_\lambda(\log^2 s)$	$O_\lambda(\log^2 s)$	collision-resistant hash / ROM	yes	yes (not hiding)	[BCF+25, Thm. 1, p. 4, §1.1, p. 4]
Hash, recursive Ligero	Ligerito	transparent: hash function only	1 hash	about $\log^2 s / \log\log s$	polylog for tensorizable codes	collision-resistant hash / ROM	yes	yes for tensorizable codes (not ZK)	[NA25, §6.2, p. 13, §6.4, p. 17, §6.6, pp. 18–19]
Unknown order	DARK (Supersonic)	class groups: transparent; RSA: trusted; constant size	1 group element	$O(\mu \log d)$ for μ variables of degree d; $O(\log s)$ if $\mu = 1$	$O(\mu \log d)$	strong RSA + adaptive root (Thm. 5); revised 2022 proof: random order + adaptive root	no	yes	[BFS20, §1.1, §3.1, Thm. 5]; Table A.1

continued on next page

Table 7.2, continued

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Unknown order	DewTwo	transparent: hyperelliptic Jacobians, $O(\log s)$ group elements (class groups fail its strong RSA assumption)	1 group element	$O(\log\log s)$ group elements; $\leq$ 4.5 KB for $s \leq 2^{30}$	$O(\log s)$ field + group ops	falsifiable group-of-unknown-order (GUO) assumptions (interactive version)	no	yes	[BMSS25, Table 1, p. 554, §3, p. 558]
Unknown order	Dew	transparent: class groups	1 group element	$O(1)$: 66 group elements	$O(\log s)$ field ops + $O(1)$ group ops	generic group model	no	yes	[AGL+23, §1.1, p. 544]; [AGL+22, §5.2.5, p. 46]
Unknown order	Behemoth	transparent: class groups; $O(s)$ group elements, from $O(s^2 \log p)$ sequential squarings	1 group element	$O(1)$: 47 group + 19 field elements	$O(1)$ group ops	generic group + random oracle models	no	yes	[SB25, Table 1, p. 26, Table 2, p. 27]
Unknown order	Mist	transparent: class groups	1 group element	$O(1)$: 3 group + 14 field elements (1312 B) at a point sampled independently of the commitment; 1472 B at arbitrary points (sketched)	$O(\log s)$ field + $O(1)$ group ops	falsifiable class-group assumptions (dyadic modular consistency (DMC), DLOG; interactive version)	no	yes	[Sri26, Table 1, p. 3, Cor. 7.3, pp. 23–24]
Unknown order	Dew++	transparent: class groups	1 group element	$O(1)$: 10 group elements + 24 λ-bit integers (about 2.7 KB)	$O(s)$ integer mults and quasi-linear group ops by default; $O(\log s)$ variant with a larger proof	generic group model	no	yes (logarithmic variant)	[Fu25, p. 3, Thm. 4, p. 19, §4.3, pp. 22–23]
Lattice	Greyhound	transparent (ROM)	$O_\lambda(1)$ ring elements	polylog(s)	$O(\sqrt{s})$	Module-SIS (hiding variant: also Module-LWE)	yes	no ($\sqrt{s}$)	[NS24, §1.1, p. 245, Table 3, p. 264, §4.5, p. 266]
Lattice	Cini–Malavolta–Nguyen–Wee	transparent (ROM)	$O(\lambda^2 \log s)$ bits	polylog(s)	polylog(s)	Module-SIS	yes	yes (not hiding)	[CMNW24, abstract, §1.1, §2.1, p. 212, Table 3, p. 235]

Discussion: what the tables show

Constant-size versus polylogarithmic proofs. Table 7.1 splits by proof size, with five exceptions. Every row built on a univariate pairing-based commitment or a linear PCP (GGPR, Pinocchio, Groth16, Sonic, PLONK, Marlin, Polymath) has an $O_\lambda(1)$ proof. HyperPlonk, Samaritan and Garuda are pairing-based too, but their sum-check-based multilinear PIOPs give logarithmic proofs [CBBZ23, §1, p. 6, Table 2; GPS25, Table 1, p. 5; DMS26, §1, p. 5792], and $\text{Spartan}_{\text{KE}}$ has a trusted setup but $O_\lambda(\log^2|C|)$ proofs [Set20, Fig. 2]. The transparent strict rows (Fractal, $\text{Spartan}_{\text{RO}}$, Supersonic) have proofs that grow polylogarithmically with the circuit, and verifiers that are polylogarithmic apart from an $O_\lambda(|x|)$ term [COS20, Thm. 3; Set20, Fig. 2; BFS20, §1]. Dew is the transparent exception: its class-group commitment gives constant-size proofs, 77 group elements with PLONK, but its corollaries state an $O(n^2)$ prover [AGL+22, Cors. 1–2, pp. 54–55] (§9.1). Among the constant-proof rows, Groth16's verifier needs three pairings and one exponentiation per public input [Gro16, Table 2], Polymath replaces these exponentiations by an $O(|x|\log|x|)$ interpolation and needs two pairings [Lip24, Table 1], and the univariate universal-SRS schemes (Sonic, PLONK, Marlin) add a $\log|C|$ term, which in PLONK comes from evaluating the vanishing polynomial $Z_H(z) = z^n - 1$ (derived from [GWC19, §8]). A short proof is not enough for the strict definition, though: most schemes in Table A.2 miss it through the verifier (§5.4), as Bulletproofs, Aurora and Orion do despite (poly)logarithmic proofs [BBB+17, §3–4; BCR+19, Thm. 1.2; XZS22, Thm. 5], and STARK's verifier is polylogarithmic only for uniform (AIR) computations [BBHR18, §2.2, Thm. 3.5].

Trusted setup versus transparency. The constant proof size is paid for in the setup: every pairing-based row of Table 7.1 with an $O_\lambda(1)$ proof has setup parameters of size $O_\lambda(|C|)$ generated from secret randomness, a per-circuit CRS or SRS for GGPR, Pinocchio, Groth16 and Polymath [Lip24, Table 1, §3, p. 188], and a universal, updatable SRS for Sonic, PLONK and Marlin (Table A.1). Only Fractal, $\text{Spartan}_{\text{RO}}$, Supersonic and Dew have $O_\lambda(1)$ setup parameters, i.e. are fully succinct: the first two need nothing beyond the random oracle [COS20, Thm. 3; Set20, Fig. 4, p. 21], Supersonic needs only a group description and a generator, and is transparent only with class groups [BFS20, §1, §4.3], and Dew uses class groups only [AGL+22, Fig. 14, p. 55]. The setup sizes of $\text{Spartan}_{\text{KE}}$, HyperPlonk, Samaritan and Garuda are not stated. Only Dew combines an $O_\lambda(1)$ proof with a transparent setup, and its prover, quadratic in its own corollaries, means that no practical zk-SNARK does yet (§9.1, Table 7.2).

Prover cost. Most strict rows have quasi-linear provers, $O_\lambda(|C|\log|C|)$: the logarithmic factor comes from the field arithmetic of the polynomial operations, and the pairing-based rows add a linear number of group exponentiations, as Table A.1's counts for Groth16 and PLONK show [Gro16, Table 2; CHM+20, Fig. 1; GWC19, Table 1]. Pinocchio's entry has a second logarithmic factor, from the $O(|C|\log^2|C|)$ field work for computing $h(X)$ by interpolation [PHGR13, p. 4, §4.2.1, p. 8]; with roots of unity this drops to $O(|C|\log|C|)$, the bound in GGPR's row [GGPR13, §2.4, p. 16]. Four strict rows have linear-time provers, and all four pay for this with a trusted setup, universal for $\text{Spartan}_{\text{KE}}$, HyperPlonk and Samaritan and circuit-specific for Garuda [Set20, §1, Fig. 2; CBBZ23, Table 2; GPS25, abstract, Table 1, p. 5; DMS26, §1 and Remark 1.1, p. 5792]. Samaritan builds its multilinear commitment from KZG, and its zero knowledge comes from cited compilers rather than a proof of its own [GPS25, §5.4, Lemma 5.3, p. 37]. With multilinear KZG or Orion+, both needing a universal setup, HyperPlonk's prover is linear; with its transparent FRI-based commitment it is quasi-linear. The paper does not instantiate or analyze it with the transparent linear-time commitments that its Table 2 lists: Dory (additive, with linear commit-and-evaluate time and logarithmic verification), Bulletproofs and Orion [CBBZ23, §1, p. 5, Table 2]. So no row of Table 7.1 has both a linear-time

prover and a transparent setup; §9.1 discusses the schemes that come closest, among them Orion (unsound as revised, §9.1). Polymath moves the other way: to shorten Groth16's proof in bits, its prover does about $40.0n$ $\mathbb{G}_1$ scalar multiplications against Groth16's $8.5n$, by its author's estimate [Lip24, §5, p. 200].

Linear-time provers are more common in Table A.2, but each gives up something else: Libra a transparent setup and depth-independent costs, Hyrax arbitrary circuits (its prover is linear only with enough data parallelism), Spartan$_{\mathrm{DL}}$ and Bulletproofs a polylogarithmic proof or verifier, and Orion a proven polylogarithmic verifier and, in its current revision, soundness itself [XZZ+19, Table 1; WTS+18, §1, §6; Set20, Fig. 2; BBB+17, §3–4; XZS22, Thm. 5; DS25, abstract]. Since O_λ counts a group exponentiation and a field multiplication alike, rows with the same asymptotic prover can differ greatly in practice, as Table A.1's exponentiation counts for unhelped Sonic and PLONK show [GWC19, Table 1]. Section 9.1 returns to linear-time provers as an open problem.

Post-quantum security and assumptions. The assumptions in Table A.1 follow the dichotomy of §3.1; among the circuit-specific rows, only Polymath, Garuda and zk-Pari also use Fiat–Shamir in the random oracle model, with their commitments analysed in the algebraic group model (§5.1, §9.1). Only the hash-based rows and LaBRADOR, a lattice-based row of Table A.2, are plausibly post-quantum (§5.3, §9.2); the hash-based ones pay with proofs that grow with the circuit; Plonky2's security is only conjectured [Pol22, §8, fn. 8]. Choosing a row is therefore also choosing an assumption: §9.4 shows that replacing the random oracle with a concrete hash can fail, and §9.5 discusses the search for standard assumptions.

8 Developments since 2022

The ten later rows of Tables 7.1 and A.2 (§7) fit the existing families, because most work since 2022 has changed the components (§4.1, §4.3, §4.4, §5.3, §6.4) rather than added families. This section draws those threads together around five questions.

Why have fields become small, and what does it cost? Group-based commitments fix the field to the group's scalar field, of about 256 bits (§2.1, Table 8.1). Hash-based commitments (§4.4) lift this constraint (§4.1 gives the cost of a fixed field). Diamond and Posen credit small fields, fields with fast reduction such as Goldilocks ($p = 2^{64} - 2^{32} + 1$, Table 8.1), and fast hashes for the speed of FRI- and Brakedown-based SNARKs. With Goldilocks and Keccak-256, FRI commits 1 MiB about 7-fold faster than KZG over BN254 [DP25, §1, p. 2]. The cost is soundness. By Schwartz–Zippel (§2.1), a random check misses a false identity with probability at most its degree over the field size. In a circle STARK, for instance, the two algebraic error terms scale with the constraint count and with the degree times the trace length, each divided by the challenge field's size [HLP24, §5.2], so over a 31-bit field one of them exceeds 2^{-31}. The small-field systems of Table 8.1 therefore keep the witness in $\mathbb{F}$ and draw challenges from an extension $\mathbb{K} \supset \mathbb{F}$, or repeat a check over $\mathbb{F}$, as Plonky2 does for some challenges. Part of the prover's work, such as Binius's sum-check, then runs in $\mathbb{K}$ [DP25, §1, p. 3]. Binary towers go furthest: Binius commits to $\mathbb{F}_2$-valued data with no embedding overhead [DP25, abstract], and ring-switching moves the evaluation claim to the large field (§4.4).

How are zkVMs built? A zkVM (§1) proves that a program for a fixed instruction set ran correctly, and the two designs in use differ in how they arithmetize a CPU step. The *lookup-centric* design reduces almost everything a CPU step does to memory operations [STZ26, §1, p. 268]. Jolt's circuits "primarily perform lookups into a gigantic lookup table, of size more than 2^{128}, that depends only on the ISA", which is structured and never

Table 8.1: Fields used by recent systems. Each row has one source. The "Challenges" column gives the field that the source's configuration samples random challenges from.

Field	Order	Why it is chosen	Challenges	Source
Scalar field of bn256 / bls381	about 256-bit prime	fixed by the pairing group (KZG, Groth16)	the field itself	[BS23, §15.4, p. 626]
Goldilocks	$p = 2^{64} - 2^{32} + 1$	an element fits in a 64-bit word, and reduction is fast	$\mathbb{F}_p[X]/(X^2 - 7)$ for the evaluation point and FRI; $\mathbb{F}_p$, with r-fold repetition, for the permutation and combining challenges (Plonky2)	[Pol22, §2–2.1, pp. 3–4, §7.2–7.3, pp. 12–13]
BabyBear	$p = 2^{31} - 2^{27} + 1$	31-bit prime used in Plonky3's FRI commitment	$\mathbb{F}_p[X]/(X^4 - 11)$ (Plonky3)	[DP26, §5.2, p. 47]
KoalaBear	$p = 2^{31} - 2^{24} + 1$	31-bit prime of SP1 Hypercube, hashed with Poseidon2	not stated in the source	[Suc26]
Mersenne-31	$p = 2^{31} - 1$	"exceptionally efficient arithmetic"; $p + 1$ is a power of two, hence circle STARKs (§4.1)	any finite extension of $\mathbb{F}_p$, sized for soundness	[HLP24, §2, §5.2]
Binary tower	$\mathbb{F}_2 \subset \mathbb{F}_{2^2} \subset \cdots \subset \mathbb{F}_{2^{128}}$	addition is XOR, and bits are committed with no embedding overhead; DP26's benchmarks use the GHASH field $\mathbb{F}_{2^{128}}$ instead [DP26, §5.2, p. 47]	$\mathbb{F}_{2^{128}}$	[DP25, §1, pp. 2–3]

materialized [AST24, abstract]. In the 2024 Jolt paper, decomposable-table lookups and offline memory checking carry almost all of the computation, and the rest is a uniform R1CS with several hundred constraints per CPU step, proved with Spartan [AST24, §1.4, p. 6, §7, p. 28]; Twist and Shout have since replaced both with one-hot arguments (§4.3). The *STARK-based* design instead proves an AIR trace (§4.1) over a small field with a hash-based commitment, as in Plonky3, "mainly used to power STARK-based zkVMs" [Plonky3]. The line between the two is blurring: SP1 Hypercube's jagged polynomial commitment stacks the whole multi-table trace into one multilinear polynomial, instead of committing to each column as a univariate polynomial with batch FRI [HJR+26, §7, p. 22]. Both designs end alike: a large zkVM proof "can be generically shrunk via SNARK composition", so the bottleneck is the prover, still "hundreds of thousands of times slower than native execution" when Twist and Shout was written [STZ26, §1, p. 268]. By December 2025, the Ethereum Foundation reported that zkVMs "prove 99% of all Ethereum blocks in under 10 seconds on target hardware", and it set provable security as the next requirement: 100 bits by May 2026 and 128 bits by the end of 2026 [EF25]. The elliptic-curve Jolt has been zero-knowledge since March 2026, through NovaBlindFold (§6.4), without wrapping and with proofs about 3 KB larger [DGT+26].

Why do recent IVC schemes fold instead of verifying recursively? Classical recursion (§6.1) proves the previous verifier inside the next circuit; folding (§6.3) proves no SNARK verifier. Each step folds the new instance into a running one, which is proven once at the end, so the circuit holds only the cheap folding verifier. Nova's verifier circuit has about 10,000 R1CS constraints, which by its authors' count is over 10× below SNARK-based recursion with a per-circuit trusted setup and over 100× below a transparent one [KST22, §1.3, p. 5]. Later schemes fold CCS, high-degree gates, lookups and many instances,

NeutronNova with a constant-size recursive verifier (§6.4, Figure 6). Two limits remain: the folded proof must still be compressed by a SNARK (§6.3), and the in-circuit hash leaves IVC security heuristic (§6.1).

How much of the security is proven? A hash-based commitment sets its query count, and so its proof size, from a proximity-gap bound (§5.3). The error terms of these bounds grow with the problem size and shrink only with the field, so WHIR samples the relevant challenges from "a sufficiently large extension of the base field" and adds proof-of-work [ACFY25, §6.2, p. 44]. Even 128 bits can be too few: Diamond and Posen note that over a 128-bit field the best proximity-gap bound then proven in the list-decoding regime gives an error of at least 2^{-100} on any reasonably large instance, so 100 bits of provable security need the unique-decoding regime [DP26, §5.2, p. 47]. Proven bounds cost proof size: for 24 variables at rate 1/2 and 128-bit security, over Goldilocks with cubic-extension challenges, WHIR's arguments are 621 KiB with the proven unique-decoding bound, 299 KiB under its Johnson-bound conjecture and 156 KiB under its up-to-capacity conjecture [ACFY25, §6.2, p. 44, §6.3.4, p. 53]. Bordage et al. have since proven mutual correlated agreement up to the Johnson radius, answering a question WHIR posed, though whether their error bound supports WHIR's 299 KiB parameters is not stated (§2.1); the up-to-capacity conjecture has been disproved (§5.3), and Crites and Stewart propose weaker replacement conjectures, which "should serve only as benchmarks" [CS26, abstract, §1, p. 130]. Newer commitments set parameters from proven bounds alone: binary BaseFold and Ligerito in the unique-decoding regime, and circle FRI up to the Johnson radius [DP26, §5.2, p. 47; NA25, §6.3, p. 14; HLP24, Thm. 6, p. 38] (Figure 5); among deployed zkVMs, SP1 Hypercube operates "in the unique decoding regime" [Suc26]. Khatam extends the proven regime to every linear code, and Dao, Kominers and Thaler report the first proof-size savings in existing implementations from bounds proven beyond the Johnson radius (§5.3).

Which post-quantum options are not hash-based? So far the answer is lattices, both for folding and for polynomial commitments. Hash-based accumulation recurses expensively: Nguyen and Setty put ARC's verifier circuit at about 1,600,000 R1CS constraints with Poseidon at code rate 1/2 [NS26, §1.1, p. 495]. ARC's authors estimate roughly 200,000 gates under different parameters, for accumulating R1CS claims of 2^{20} constraints at rate 1/16 (32 Merkle openings) and without standard optimizations [BMNW25, §1.2, p. 132]. Group-based folding schemes rely on discrete-log commitments, so they are not post-quantum and need a large (256-bit) field [BC25a, abstract]. LatticeFold (§6.4) uses Ajtai commitments under Module-SIS instead, and keeps extracted witnesses of low norm through many folds with sum-check [BC25a, abstract]. LatticeFold's rings split completely over fields such as Goldilocks, which forces an extension and a matching multiplicative overhead [NS26, §1.1, p. 495]. Neo and SuperNeo instead run one sum-check over a small-field extension, with commitment costs that scale with the witness's bit-width [NS26, abstract]. A constant-size lattice verifier circuit like Nova's "remains an open problem" [NS26, §1.1, p. 495]. Among lattice commitments, Greyhound's verifier is only sublinear, RoKoko's polylogarithmic verifier rests on the less standard vanishing-SIS (vSIS) assumption [KLN+26, p. 5] (§9.2), and several schemes rest on the contested knowledge k-R-ISIS assumption (§4.4). Section 9.2 takes up what a complete lattice-based zk-SNARK still lacks.

9 Open problems and future directions

Tables 7.1, 7.2, A.1 and A.2 show what current zk-SNARKs achieve and at what cost, and §8 shows where recent work has moved. This section collects the open problems that remain, together with recent work on them, in five areas: prover efficiency and proof size

(§9.1), post-quantum zk-SNARKs (§9.2), recursion overhead and the security of folding (§9.3), Fiat–Shamir in practice (§9.4), and SNARGs from standard assumptions (§9.5).

9.1 Prover efficiency and proof size

Prover efficiency. The prover is a key practical bottleneck, one that has limited SNARKs to relatively small statements [GLS+23, §1, p. 1; YLZ+26, abstract]. One line of work removes the FFT, as HyperPlonk does by moving PLONK's PIOP to the Boolean hypercube and reducing every check to sum-check (§4.2). HyperPlonk is made zero-knowledge by masking, for any additive, spanning commitment scheme [CBBZ23, App. A, Cor. A.4].

A second line uses linear-time-encodable codes, starting with Brakedown, which this survey treats only as a commitment (§4.4, §5.4). Orion is a zero-knowledge argument with a linear-time prover and $O_\lambda(\log^2|C|)$ proofs, but its theorem, which the paper states without proof, gives a linear verifier; the polylogarithmic one is claimed only after preprocessing and without a theorem [XZS22, abstract, Thm. 5, p. 23, §1, pp. 3–4]. Moreover, den Hollander and Slamanig show with an implemented attack that the revision cited here "is still unsound", and note that its commitment's verifier is $O(\sqrt{s})$ for s committed coefficients [DS25, abstract, p. 5, §3.1, p. 15]; the revision's own complexity analysis also states that verifier, whereas the CRYPTO 2022 version claims $O(\log^2 s)$ [XZS22, §4.2, p. 22 of the ePrint revision, LNCS version, p. 318]. Den Hollander and Slamanig's repair, Scorpius, is proven only as a zero-knowledge commitment relative to an outer commit-and-prove SNARK (one that proves statements about already-committed values), has no implementation, and leaves the Fiat–Shamir analysis of a full zk-SNARK to future work [DS25, Thm. 4, p. 18, §5.1, p. 27].

A third line works over small and binary fields (§8). The successor of Binius (§4.4) is parametrized for 100 bits of provable security in the unique-decoding regime [DP26, §5.2, p. 47], and Blaze, over a 128-bit binary field, has a linear-time prover but proofs larger than BaseFold's, which was run over a 64-bit prime field, with queries set for 100 bits of security [BCF+25, §8, p. 39, §8.2, p. 40, Fig. 4, p. 41].

The 2026 commitment schemes push the prover's constant factors further: Bolt, in its theory-oriented instantiation over binary fields, commits with about $(3 + \varepsilon)$ field additions per element plus a Merkle hash [GNR26, abstract, p. 396, Thm. 1, pp. 397–398], TensorSwitch opens with about six field multiplications per coefficient [BFRW26, abstract, p. 297], and Chopin has constant-size pairing-based proofs and a single large multi-scalar multiplication [BHK+26, abstract, p. 428]. None of the three is zero-knowledge as presented (§3): Bolt "does not provide hiding" [GNR26, §1.3, p. 407], and Chopin's zero-knowledge variant is only outlined, with its standard-model knowledge soundness left to future work [BHK+26, p. 436].

An open problem, then, is to combine a linear-time prover, zero knowledge, a polylogarithmic verifier and a transparent setup in one fully proven non-interactive scheme; for arbitrary circuits the verifier bound can hold only after a transparent preprocessing of the kind §3 allows, since otherwise the verifier must read the circuit [XZS22, §1, pp. 3–4]. Bootle, Chiesa and Liu reach all four properties, but only in an interactive argument with constant soundness error and no non-interactive version [BCL22, Thms. 1–2, pp. 2–3, §1, p. 3]. Bootle, Chiesa and Marty prove round-by-round knowledge soundness, hence Fiat–Shamir security, for a related linear-time argument from tensor codes, but its verifier is linear, it does not treat zero knowledge, and they leave Bootle–Chiesa–Liu, Orion and Blaze to future work [BCM26, Thm. 1, p. 2, Cor. 1, p. 3, §7, p. 29]. Interactive arguments and IOPs for Boolean circuits with linear or near-linear provers (Ron-Zewi–Rothblum, Holmgren–Rothblum, Athamnah–Ron-Zewi–Rothblum) do not prove zero knowledge; Ron-Zewi–Rothblum and Holmgren–Rothblum only remark that the standard transformation of Ben-Or et al. would add it, in the first case with a non-black-box use of the hash [RZR22, Thm. 3.2, p. 18, p. 2, fn. 3; HR22, Thm. 1.1, p. 475, p. 486; ARZR25, Thm. 1.1,

p. 4, §1.3, p. 13]. Among non-interactive schemes, Setty and Lee's Xiphos already claims all four, with $O_\lambda(n)$ group exponentiations, $O_\lambda(\log n)$ proofs and, after preprocessing, $O_\lambda(\log n)$ verification, under SXDH in the random oracle model [SL20, abstract, §1.2, p. 4, Fig. 1, p. 3]. However, it is an unrefereed preprint with no end-to-end security theorem, its prover is linear only in group operations, and it is not post-quantum. Pappas and Papadopoulos's refereed Hobbit also claims all four, with a linear number of field operations, from a Reed–Solomon–Spielman tensor code and proof composition with WHIR [PP25, Thm. 2, p. 22, Thm. 3, p. 32]; but its theorem states only an argument of knowledge, its zero-knowledge proofs are sketched or omitted (partly by reference to Orion), and non-interactive knowledge soundness is left to future work [PP25, §4.3.3, p. 26, fn. 6, p. 26, §5.2, p. 34]. Before Hobbit, Orion aimed closest (above), and no row of Table 7.1 has both a linear-time prover and a transparent setup (§7).

Distributed proving splits the prover across machines; HyperFond does so transparently, plausibly post-quantum and with polylogarithmic communication, but its proof size and verification time grow linearly with the number of machines, an overhead left open, and its PIOPs are presented without zero knowledge [YLZ+26, abstract, §6, p. 880, Remark 3, p. 876].

Proof size. How small can a proof get? Arnon, Dujmovic and Yogev construct a publicly verifiable SNARG with imperfect completeness, not claimed to be zero-knowledge, whose proof is two $\mathbb{G}_1$ elements (768 bits on BLS12-381), and prove, assuming one-way functions, that no SNARG in the (Maurer) generic bilinear group model plus random oracle has a one-element proof, under mild restrictions on the verifier and the setup [ADY26, abstract, p. 125, §1.1, pp. 128–130]. Pari has two $\mathbb{G}_1$ and two field elements (1280 bits on BLS12-381) with a circuit-specific trusted setup, but no zero knowledge [DMS26, abstract, Remark 1.1, p. 5792, §9.1, p. 5803]. Its companion Garuda goes the other way, with a linear-time prover, $O(\log n)$ proofs and verification (5–8 kB in its benchmarks) and a zero-knowledge version specified from PLONK-style randomization, a hiding commitment and sum-check masking, though not proven (Table 7.1) [DMS26, §1, p. 5792, §5, p. 5798, §8.1.2, p. 5802]. Khurana, Padhye and Sahu add zero knowledge to Pari with the same proof and verifier, at the cost of 13 more $\mathbb{G}_1$ elements in the proving key. The result, zk-Pari, is a zero-knowledge argument with fewer group elements than Groth16's three, proven knowledge-sound in the algebraic group model with random oracles under SDH and a Witness-Column Independence condition on the circuit, though it is not implemented [KPS26, abstract, §1.1, pp. 2–3, Thm. 3, p. 21, §7, pp. 22–23]. Because its knowledge-soundness theorem assumes that condition, it has a conditional row in Table A.2 [KPS26, Thm. 4, p. 22]. Polymath (§5.1) is a zk-SNARK shorter than Groth16 in bits, though not in group elements [Lip24]. In the designated-verifier setting, one generic-group element plus $O(\tau)$ bits suffices for $2^{-\tau}$ soundness; adding one random-oracle output and a CRS quadratic in the circuit size gives 695 bits for 2^{-80} soundness at the 128-bit level [ADI25, §1.1, Thms. 1–2, pp. 194–196, Cor. 3, p. 221], though without reusable soundness [ADY26, p. 127].

Transparent constant-size proofs. These have been reached only in groups of unknown order, and not yet at practical cost (Table 7.2 gives sizes and verifiers). Dew has constant-size class-group evaluation proofs. Its prover is $O(n^2)$ in its authors' corollaries (its Figure 14 gives $\tilde{O}(n)$ after preprocessing) and cubic in later papers. Corollaries 1–2 state the compiled arguments only as arguments of knowledge, but the full version proves a hiding commitment and an honest-verifier zero-knowledge evaluation, and derives zero knowledge for the arguments from the compiler theorem it cites, so Dew has a row in Table 7.1 [AGL+23, abstract; AGL+22, §1.1, p. 4, §7.1, pp. 50–52, §7.3, p. 54, Cors. 1–2, pp. 54–55, Fig. 14, p. 55; SB25, Table 2, p. 27; Sri26, §2, p. 4]. Behemoth, also constant-size, has a cubic prover and a transparent setup estimated at 63.3 years of sequential squarings

for degree 2^{20} [SB25, abstract, Table 1, p. 26]; Dew++ has a quasi-quadratic prover, but a linear default verifier and only sketched zero knowledge [Fu25, p. 3, §4.2, p. 21, §4.3, pp. 21–23]; DewTwo trades constant size for a quasi-linear prover and falsifiable assumptions, with hiding and zero-knowledge evaluation only sketched and deferred to the full version [BMSS25, abstract, §5.3, p. 573]; and Mist, a 2026 preprint, reaches a quasi-linear prover under falsifiable class-group assumptions, but is analyzed only interactively and without hiding [Sri26, abstract, §3.2, p. 7, Cor. 6.5, p. 19, §7.1, pp. 20–21]. The other four have no row in Tables 7.1 and A.2 (§7). What remains open is a transparent zk-SNARK with constant-size proofs, a practical prover and proven zero knowledge, and a post-quantum one, since groups of unknown order are not quantum-secure (§5.3).

Hash-based proofs. Haitner, Nukrai and Yogev prove a lower bound for hash-based schemes under the randomized exponential-time hypothesis, for deterministic non-adaptive verifiers and under the stronger "salted" soundness, restrictions that all known constructions satisfy. Under them, (t, ε)-sound SNARGs in the random oracle model need $\tilde{\Omega}(\log(t/\varepsilon) \cdot \log t)$-bit proofs, with the instance size entering only through a $1/\log q_P$ factor (q_P the honest prover's oracle queries). By our reading, Thm. 5.1 gives at least $2^{-14}\lambda^2/\log q_P$ bits for $t = 1/\varepsilon = 2^\lambda$ [HNY22, abstract, Thm. 1.1, p. 2, Thm. 5.1, p. 20].

9.2 Post-quantum zk-SNARKs

Table A.1 marks the hash-based schemes (Plonky2 aside) and one lattice-based argument, LaBRADOR, as plausibly post-quantum. This subsection first qualifies the hash-based mark and then turns to lattice-based constructions. For the hash-based schemes, two points need care. First, STARK's post-quantum claim is informal, and its hash-only basis covers the interactive protocol only; the non-interactive version also assumes a random oracle [BBHR18, §1.3.1, fn. 12, p. 9, §3.3, p. 22]. For the BCS transform that compiles such IOPs into non-interactive arguments, Chiesa, Manohar and Spooner prove knowledge soundness in the quantum random oracle model, for IOPs with round-by-round knowledge soundness [CMS19, Thm. 8.6, p. 40; RT25, §1, p. 100]. Second, concrete security often rests on Reed–Solomon proximity-gap conjectures, whose status §5.3 and Figure 5 summarize. Some deployed systems now set parameters from proven bounds alone, at the proof-size cost that §8 measures for WHIR, while others still rely on conjectures [FS26, §1.2, Table 1, p. 3]; what remains open is to narrow the gap between proven and conjectured bounds, which is now actively pursued: the Ethereum Foundation has made provable security a requirement for its zkVM milestones [EF25], and bounds claimed beyond the Johnson radius, in a 2026 preprint, have given the first proof-size reductions in implemented systems [DKT26, abstract, §8].

Plain Fiat–Shamir applied to other many-round protocols, such as folding and sum-check-based arguments, still has a quantum-specific gap: the known quantum-random-oracle analysis for $(2s+1)$-round protocols loses a factor of order q^{2s+1} for q queries, which gives no guarantee for super-constantly many rounds [RT25, §1, p. 99]. Rotem and Tessaro's multi-round transform with straight-line extraction avoids this loss, using Merkle-tree commitments to transcript trees [RT25, abstract, §1]; its prover builds a whole tree of transcripts, which stays polynomial for $O(\log \lambda)$ rounds, and its QROM theorem gives no concrete bound, with the proof deferred to the full version [RT25, §1, p. 99, Thm. 5.1, p. 122].

Lattices are the other post-quantum route. LatticeFold and LatticeFold+ give plausibly post-quantum folding (§6); by its authors' estimates and a Nethermind implementation, LatticeFold's prover is about as fast as HyperNova's [BC25a, abstract, §1, p. 6, §7, p. 56; BC25b, §1, p. 4]. Neo and SuperNeo move lattice folding to small fields (§6.4, §8). Greyhound (§4.4) has 53 KB evaluation proofs at degree 2^{30}, but a verifier that is sublinear rather than polylogarithmic [NS24, abstract]. Albrecht et al.'s earlier publicly verifiable,

tentatively post-quantum lattice SNARK has a logarithmic-time verifier after preprocessing, but a trusted setup, zero knowledge only sketched and a knowledge assumption from its new k-R-ISIS family (formally knowledge k-M-ISIS) against which §4.4 records evidence [ACL+22, abstract, §1.2, p. 2, p. 7, p. 17, Thm. 4, p. 31, App. A, p. 48; WW23, abstract]. Debris-Alazard, Fallahpour and Stehlé's quantum oblivious LWE sampler invalidates the security analyses, though not the constructions, of several standard-model lattice SNARKs; the random-oracle-model one it names is unaffected [DFS24, abstract, §1, p. 3, §1.2, p. 10].

Two recent constructions come close on succinctness. Symphony is a SNARK from high-arity lattice folding with polylogarithmic proofs, whose verification is linear in the number of proven statements and whose CP-SNARK verifier takes the roughly 1 MB of Fiat–Shamir randomness (mostly the random projection matrix) as public input, a cost the paper leaves open [Che25, abstract, §1.1, p. 5, §7, pp. 42–44]. RoKoko is a lattice polynomial commitment with a linear-time prover, polylogarithmic verification and 112 KB proofs [KLN+26, Table 1, p. 6]. Neither treats zero knowledge, and none of the works cited here builds and analyzes a complete lattice-based zk-SNARK, zero knowledge included, that meets the strict bound of §3 (cf. Figure 4); even Cini et al.'s commitment, with polylogarithmic proofs and verification (Table 7.2), is not hiding [CMNW24, §2.1, p. 212]. Building a complete scheme is an open direction.

9.3 Recursion overhead and the security of folding

Recursion raises two problems: its cost, measured by the size of the verifier circuit and the recursion threshold (§6.4, §8), and its security, which rests on the in-circuit hash heuristic (§3.1, §6.1). We take security first and return to cost in the last paragraph. Nova, for example, instantiates its random oracle with a concrete hash function and *assumes* that the result remains knowledge-sound [NBS23, §2.3, p. 18:5]. The circuits in Khovratovich–Rothblum–Soukhanov's attack (§4.2) compute the Fiat–Shamir hash and the commitment internally, which the authors note is common in recursive proof composition [KRS25, §1.1.2, p. 5]. The gap between paper and code is a second security problem, as the forged proofs against Nova's deployed 2-cycle variant show (§6.3) [NBS23, abstract]. Nguyen, Boneh and Setty's analysis also shows a limit of recursive extraction: if the extractor is slower than the prover by a constant factor $c > 1$, iterating it works only for $O(\log \lambda)$ steps [NBS23, §2.1, Remark 2, p. 18:3]. Lee and Seo show that this bounded-depth definition is genuinely too weak: an artificial IVC scheme can satisfy it yet be forgeable at polynomial depth, while Nova itself has polynomial-depth knowledge soundness in an extended algebraic group model under the discrete-logarithm assumption, with a zero-testing hash in place of the random oracle [LS24, abstract, Thm. 5, p. 30].

On the theory side, Jain, Mathialagan and Waters build fully succinct IVC for deterministic computations from subexponential indistinguishability obfuscation (iO) and one-way functions. Unlike SNARK-based recursion, whose SNARKs need non-standard assumptions or models, their construction uses no recursive extraction; it is not claimed to be zero-knowledge [JMW26, abstract, p. 3, §1, p. 4]. IVC for deterministic computations already followed from LWE alone, through rate-1 batch arguments [PP22; DGKV22], but theirs is the first such IVC without algebraic assumptions [JMW26, abstract, p. 3, §1.1, p. 5]. Datta et al. obtain zero-knowledge IVC for NP, sound but not knowledge-sound, from subexponential iO with LWE or bilinear maps, with proofs polynomial in the configuration size and $\log T$ for T steps [DJJ+25, abstract, Thm. 1.1, Cor. 1.2, p. 614, Remark 3.6, p. 634].

Recursion overhead also separates group-based from post-quantum folding. NeutronNova's recursive verifier is constant-size (§6.4), but "achieving constant recursion overheads in the post-quantum setting remains an open problem" [KS26, §1, p. 527], and Neo and SuperNeo aim for logarithmic recursion overhead, like HyperNova's [NS26, §1.1, p. 495]. ProtogaLattice replaces sum-check in its folding backbone by ProtoGalaxy-style folding

under Module-SIS. Its authors argue that this makes the verifier circuit smaller, from fewer rounds and random-oracle calls, though not constant, since the range proof it attaches is still a sum-check with a logarithmic number of rounds, and they leave a zero-knowledge variant open [BNP26, abstract, §1.2, p. 4, §1.4, p. 6]. Recursion that is efficient, zero-knowledge and proven secure with the concrete hash it uses remains open; for PLONK, a falsifiable zero-testing assumption on the hash justifies recursive use, but only in the recursive AGM (§5.2) [FS26b, abstract].

9.4 Fiat–Shamir in practice

The Fiat–Shamir step of the pipeline in §4 is where many deployed failures occur. *Weak* Fiat–Shamir (§2.3, §3.1) breaks the adaptive knowledge soundness of Bulletproofs, PLONK and Spartan [DMWG23, abstract], and Chaliasos et al. record this "Frozen Heart" class and the related "Last Challenge" attack among the proof-system flaws in their dataset [CET+24, App. A.2–A.3, p. 20]. The class recurs: in a 2026 report, auditors described public-claim data left out of the Fiat–Shamir transcripts of six zkVMs, among them Jolt and Binius64; the issues were reported to the teams between September and December 2025 and have all since been fixed [SW26].

Strong Fiat–Shamir can fail too: the attack on GKR-based arguments (§4.2, §5.4) works for every choice of hash [KRS25, Thm. 1], so soundness depends on how the circuit is implemented, not just on what it computes [KRS25, §1.1.3]. Its authors know of no such attack on protocols that commit to the full computation trace, as most PIOP-based schemes do, but do not rule them out [KRS25, §5, p. 16]. Fenzi has since broken adaptations of Spartan- and Aurora-style IOPs, which do commit to the witness, when the R1CS instance is generated by a program and the Fiat–Shamir hash binds only the program, for any hash that is cheap relative to the generator's time budget. Hashing the generated statement instead provably reduces soundness to that of the base protocol's Fiat–Shamir compilation with the same hash, for the non-generated relation [Fen26, abstract, p. 5, Prop. 5.5, p. 21].

Hardened transforms are only a partial answer. Arnon and Yogev's extended Fiat–Shamir (XFS), which adds a proof of work against attacks like Khovratovich, Rothblum and Soukhanov's (§4.2), is proven only for three-message protocols in a relativized random oracle model [AY25, abstract, p. 3], which, Khovratovich et al. note, "fails to capture" their attack; Arnon and Yogev argue informally that XFS circumvents it [AY25, p. 2], and whether XFS resists such attacks is open [KRS25, §5, p. 16]. Brzuska, Hubáček and Kalsta give a contrived counterexample to it for concrete hashes: a three-message protocol whose XFS transform is unsound for every uniform hash function, assuming SNARGs for RAM computations with a pseudorandom CRS, known from LWE. They also show that contrived but secure commitments sharing the Fiat–Shamir hash can make even PLONK's gate check unsound, though no attack on a concrete commitment and hash is given [BHK26, abstract, p. 3, Thm. 1, p. 15, Thm. 2, p. 17, §6.2, pp. 24–26, §7, pp. 26–31, Thm. 3, p. 30].

On the positive side, Chiesa et al. prove that the PIOP-plus-commitment recipe gives state-restoration-sound arguments, as Fiat–Shamir security in the random oracle model needs, when the PIOP is state-restoration sound and the commitment has a state-restoration form of function binding (§4.5) [CGKY26, abstract, p. 189].

Extraction is a separate issue. Fiat–Shamir extraction rewinds the prover, which is often at odds with concurrent and universally composable security; Rotem and Tessaro's generic straight-line-extractable transforms for multi-round protocols use only the random oracle and apply to Bulletproofs [RT25, abstract, §1, pp. 96–98, 100]. Faonio and Russo show that the multivariate PST commitment of Papamanthou et al. is malleable only through its linear homomorphism, and that this suffices for sum-check-based zk-SNARKs compiled with PST, after their zero-knowledge and batching compilers, to be simulation-extractable in the algebraic group model, provided every oracle is queried in some sum-check. Because

PST proofs are not unique, only the weak notion (no proof for a new statement) is possible [FR26, abstract, p. 35, §1.2, p. 38, §1.3, pp. 41–42, Thm. 2, p. 55]. Without the AGM, though still with random oracles, Lipmaa proves fully optimized PLONK simulation-extractable under ARSDH and SplitRSDH [Lip25, abstract, Thm. 7, p. 31]. Two problems remain open: characterizing which deployed protocols are Fiat–Shamir-secure with a concrete hash, and proving simulation extractability of deployed schemes such as PLONK and Marlin without idealized models, the random oracle included.

9.5 Foundations: SNARGs from standard assumptions

The Gentry–Wichs barrier (§3.1) explains why every zk-SNARK compared in this survey uses a knowledge assumption, an idealized group model or a random oracle [GW11, Thm. 5.1]. Recent theory works around it for SNARGs, which are sound but not claimed to be knowledge-sound. Waters and Wu build an adaptively sound SNARG for NP from subexponential iO and one-way functions, with a CRS whose size grows with the relation's circuit; the SNARG is also perfectly zero-knowledge [WW25, abstract, Thm. 1.1, p. 3, Remark 4.16, p. 26]. Freitag and Wichs explain why this does not contradict the barrier: such constructions use subexponential hardness and a CRS longer than the statement [FW26, abstract, p. 190]. Restricting the language also helps. For UP, where each statement has at most one witness, Mathialagan et al. obtain designated-verifier zk-SNARKs with adaptive soundness, though with only a non-adaptive black-box knowledge extractor, from subexponential LWE and evasive LWE, a new assumption that is not falsifiable on its face [MPV24, abstract, §1, p. 42, Cor. 4, p. 67]. Applied to the Sahai–Waters SNARG, the same method replaces evasive LWE by LWE, subexponential one-way functions and subexponential iO [MPV24, abstract, Cor. 5, p. 67]. For all of NP, two 2026 results give non-adaptive SNARGs with non-black-box proofs: from LWE or bilinear maps under a non-cryptographic Nullstellensatz conjecture [DHK+26, abstract, Thm. 1.1, p. 2], and from LWE, SXDH and derandomization under a new, non-falsifiable proof-complexity assumption [HJLM26, Thm. 1.1, p. 3, p. 40].

Knowledge soundness is harder still, since black-box extraction already forces proofs of size $\Omega(|w|)$ for some NP relation under discrete log (§3.1) [CGKS23], but that bound relies on instances with multiple witnesses [JLP26, §2.1, p. 1815]. For UP languages whose witness uniqueness has polynomial-size Extended Frege proofs, Jin, Lu and Peng build SNARKs from polynomially hard LWE, with a non-black-box soundness reduction and a guarantee "weaker than a standard infinitely-often soundness"; zero knowledge is not claimed [JLP26, abstract, Thm. 2.1, p. 1813, §2, p. 1814, §2.2, p. 1815].

Batch arguments (BARGs) relax the goal differently. Proofs may grow with a single witness, and security is somewhere extractability, so BARGs "evade the known impossibility result for SNARKs" and are realizable from falsifiable assumptions [BFN26, §1, p. 363]. None of these results yields a practical zk-SNARK, and whether zk-SNARKs for NP can be built from standard falsifiable assumptions without idealized models remains open.

10 Conclusion

Under one strict definition, the zk-SNARK families of 2013–2020 reduce to a few trade-offs. Pairing-based schemes built on a linear PCP or on univariate KZG have constant-size proofs; HyperPlonk's multilinear KZG gives logarithmic ones (§7, Table 7.1). With a circuit-specific setup (GGPR, Pinocchio, Groth16), Groth16's three group elements remain the deployed benchmark, although Polymath is shorter in bits, and Pari, with Khurana et al.'s zero-knowledge extension, is shorter in group elements too, for restricted circuits (§5.1, §9.1). With a universal, updatable setup (Sonic, PLONK, Marlin), one ceremony serves all circuits up to a size bound. Either way the price is a trusted setup with parameters

linear in the circuit size, and assumptions that are non-falsifiable or idealized (§§5.1–5.2). Transparent schemes (Fractal, Spartan$_{\mathrm{RO}}$, Supersonic with class groups) remove the trapdoor at the cost of logarithmic or polylogarithmic proofs, or, in Dew, a superlinear prover, and among the strict rows only the hash-based ones are plausibly post-quantum (§5.3, §7). Bulletproofs, Ligero, Aurora and Hyrax miss the strict bound, Bulletproofs and Aurora because of the verifier alone (§5.4). In the pipeline of §4, the polynomial commitment largely decides the setup, the proof size and the post-quantum status, and Fiat–Shamir adds a random-oracle heuristic that has failed in practice (§3.1, §9.4).

Since 2022 the components have changed more than the families: Tables 7.1 and A.2 add ten later schemes, from HyperPlonk to zk-Pari, of which Orion has since been broken (§9.1), and most new work changes one pipeline component (§7, §8). Hash-based commitments lift the 256-bit field of group-based ones, so recent systems run over small prime fields or binary towers with extension-field challenges (§4.1, §8). Lookups have moved from plookup's sorted grand product to log-derivative identities and sum-check, and Twist and Shout, which Jolt builds on, prove memory reads and writes without grand products (§4.3, §8). zkVMs build either on such lookups, as in Jolt, or on AIR traces over small fields with hash-based commitments (§8). GKR has returned because its prover commits only to the input, the property the Fiat–Shamir attack of §4.2 exploits. FRI and Ligero have grown into a family of hash-based commitments (STIR, WHIR, BaseFold and its binary successor, Brakedown, Blaze, Ligerito) with linear-time provers or polylogarithmic verification, none zero-knowledge as presented (§3; Brakedown only via an outer zkSNARK). Honest-verifier zero knowledge was proven in 2026 for variants of WHIR and Ligerito, but only at the IOP level (§4.4). Folding, the main alternative to recursion by verification, now covers CCS, high-degree gates, lookups, many instances and, with LatticeFold and Neo, lattices (§6.4, §8). The security of hash-based commitments is less settled than their deployment suggests: Crites and Stewart disproved the up-to-capacity correlated-agreement conjecture and WHIR's mutual version of it, and two further counterexamples followed, while beyond-Johnson gaps were shown to fail over binary fields (§5.3). Mutual correlated agreement is proven up to the Johnson radius, but FRI's own conjecture remains open, and proven unique-decoding parameters make WHIR's arguments about four times larger than under its since-disproved up-to-capacity conjecture (§5.3, §8).

The open problems of §9 fall into five areas. In prover efficiency and proof size (§9.1), two schemes are still missing. One is a transparent zk-SNARK with constant-size proofs, a practical prover and proven zero knowledge, ideally post-quantum; Dew already has constant size and proven zero knowledge, but not a practical prover. The other is a fully proven non-interactive scheme with a linear-time prover, zero knowledge, a polylogarithmic verifier and transparent preprocessing; an interactive argument with constant soundness error has all four, and an unrefereed pairing-based preprint and the refereed Hobbit claim them, without complete proofs. For post-quantum security (§9.2), narrowing the proof-size gap between proven and conjectured parameters remains open, and no complete, publicly verifiable lattice-based zk-SNARK yet meets the strict bound. Post-quantum folding still lacks constant recursion overhead (§9.3), and Fiat–Shamir and recursion are proven secure with the concrete hash they use only in idealized or restricted models, as is the simulation extractability of PLONK and Marlin (§§9.3–9.4). Finally, zk-SNARKs for NP from standard falsifiable assumptions without idealized models remain out of reach (§9.5). Once succinctness is fixed by one strict definition, the schemes that meet it differ mainly in what they trust: a setup, an idealized model, or a hash function assumed to behave like a random oracle.

Acknowledgments

This survey is an independent project, conceived and directed by the author without a faculty advisor. AI tools were used to assist in writing this paper.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "STIR: Reed–Solomon Proximity Testing with Fewer Queries". In: *Journal of the ACM* (2026). Just Accepted (online 2026-10-16); Preliminary version in CRYPTO 2024, LNCS, pp. 380–413, doi:10.1007/978-3-031-68403-6_12; IACR ePrint 2024/390. DOI: 10.1145/3858414.

[ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification". In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Full version: IACR ePrint 2024/1586. Springer, 2025, pp. 214–243. DOI: 10.1007/978-3-031-91134-7_8.

[ACL+22] Martin R. Albrecht, Valerio Cini, Russell W. F. Lai, Giulio Malavolta, and Sri AravindaKrishnan Thyagarajan. "Lattice-Based SNARKs: Publicly Verifiable, Preprocessing, and Recursively Composable". In: *Advances in Cryptology – CRYPTO 2022, Part II*. Vol. 13508. LNCS. Full version: IACR ePrint 2022/941 (PDF dated 2023-02-08). Springer, 2022, pp. 102–132. DOI: 10.1007/978-3-031-15979-4_4.

[ADI25] Gal Arnon, Jesko Dujmovic, and Yuval Ishai. "Designated-Verifier SNARGs with One Group Element". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 192–224. DOI: 10.1007/978-3-032-01907-3_7.

[ADY26] Gal Arnon, Jesko Dujmovic, and Eylon Yogev. "Pairing-Based SNARGs with Two Group Elements". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Full version: IACR ePrint 2025/2160. Springer, 2026, pp. 125–156. DOI: 10.1007/978-3-032-35424-2_5.

[AGL+22] Arasu Arun, Chaya Ganesh, Satya V. Lokam, Tushar Mopuri, and Sriram Sridhar. *Dew: Transparent Constant-sized zkSNARKs*. Cryptology ePrint Archive, Paper 2022/419. Full version of the PKC 2023 paper; revision of 2022-07-01. 2022.

[AGL+23] Arasu Arun, Chaya Ganesh, Satya V. Lokam, Tushar Mopuri, and Sriram Sridhar. "Dew: A Transparent Constant-Sized Polynomial Commitment Scheme". In: *Public-Key Cryptography – PKC 2023, Part II*. Vol. 13941. LNCS. IACR ePrint 2022/419, titled "Dew: Transparent Constant-sized zkSNARKs". Springer, 2023, pp. 542–571. DOI: 10.1007/978-3-031-31371-4_19.

[AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. Extended version: IACR ePrint 2022/1608. ACM, 2017, pp. 2087–2104. DOI: 10.1145/3133956.3134104.

[ARZR25] Noor Athamnah, Noga Ron-Zewi, and Ron D. Rothblum. "Linear Prover IOPs in Log Star Rounds". In: *Theory of Cryptography – TCC 2025, Part I*. Vol. 16268. LNCS. Cited version: IACR ePrint 2025/1269 (version 2025-09-29). Springer, 2025, pp. 335–368. DOI: 10.1007/978-3-032-12287-2_12.

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. "Jolt: SNARKs for Virtual Machines via Lookups". In: *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Vol. 14656. LNCS. Full version: IACR ePrint 2023/1217. Springer, 2024, pp. 3–33. DOI: 10.1007/978-3-031-58751-1_1.

[AY25] Gal Arnon and Eylon Yogev. "Towards a White-Box Secure Fiat-Shamir Transformation". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Full version: IACR ePrint 2025/329 (version of 2025-06-30). Springer, 2025, pp. 27–56. DOI: 10.1007/978-3-032-01887-8_2.

[BBB+17] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. "Bulletproofs: Short Proofs for Confidential Transactions and More". In: *2018 IEEE Symposium on Security and Privacy (SP)*. Full version: IACR ePrint 2017/1066, https://eprint.iacr.org/2017/1066. IEEE, 2018, pp. 315–334. DOI: 10.1109/SP.2018.00020.

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. IACR Cryptology ePrint Archive, Report 2018/046. 2018.

[BBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. "Fast Reed-Solomon Interactive Oracle Proofs of Proximity". In: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Vol. 107. LIPIcs. Schloss Dagstuhl, 2018, 14:1–14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14.

[BC23] Benedikt Bünz and Binyi Chen. "Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols". In: *Advances in Cryptology – ASIACRYPT 2023*. LNCS. Full version: IACR ePrint 2023/620. Springer, 2023, pp. 77–110. DOI: 10.1007/978-981-99-8724-5_3.

[BC25a] Dan Boneh and Binyi Chen. "LatticeFold: A Lattice-Based Folding Scheme and Its Applications to Succinct Proof Systems". In: *Advances in Cryptology – ASIACRYPT 2025*. LNCS. Full version: IACR ePrint 2024/257. Springer, 2025, pp. 330–362. DOI: 10.1007/978-981-95-5099-9_11.

[BC25b] Dan Boneh and Binyi Chen. "LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: IACR ePrint 2025/247. Springer, 2025, pp. 327–361. DOI: 10.1007/978-3-032-01907-3_11.

[BCC+16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. "Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting". In: *Advances in Cryptology – EUROCRYPT 2016*. Vol. 9666. LNCS. Springer, 2016, pp. 327–357. DOI: 10.1007/978-3-662-49896-5_12.

[BCC88] Gilles Brassard, David Chaum, and Claude Crépeau. "Minimum Disclosure Proofs of Knowledge". In: *Journal of Computer and System Sciences* 37.2 (1988), pp. 156–189. DOI: 10.1016/0022-0000(88)90005-0.

[BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. "From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again". In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. Full version (cited sections): IACR ePrint 2011/443. ACM, 2012, pp. 326–349. DOI: 10.1145/2090236.2090263.

[BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data". In:

Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC 2013). Full version: IACR ePrint 2012/095. 2013, pp. 111–120. DOI: 10.1145/2488608.2488623.

[BCF+25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. "Blaze: Fast SNARKs from Interleaved RAA Codes". In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Cited version: IACR ePrint 2024/1609 (version of 2024-10-09). Springer, 2025, pp. 123–152. DOI: 10.1007/978-3-031-91134-7_5.

[BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. "Linear-Time Accumulation Schemes". In: *Theory of Cryptography – TCC 2025, Part I*. LNCS. IACR ePrint 2025/753, version of 2026-06-17. Springer, 2025, pp. 369–399. DOI: 10.1007/978-3-032-12287-2_13.

[BCG+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. "Zerocash: Decentralized Anonymous Payments from Bitcoin". In: *2014 IEEE Symposium on Security and Privacy (S&P)*. Full version: IACR ePrint 2014/349. 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.

[BCG25] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. "Relativized Succinct Arguments in the ROM Do Not Exist". In: *Theory of Cryptography – TCC 2025, Part I*. Vol. 16268. LNCS. Cited version: IACR ePrint 2024/728 (version 2025-09-23); TCC 2025 Part I, LNCS 16268. Springer, 2025, pp. 400–416. DOI: 10.1007/978-3-032-12287-2_14.

[BCGM26] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. "All Polynomial Generators Preserve Distance with Mutual Correlated Agreement". In: *41st Computational Complexity Conference (CCC 2026)*. LIPIcs. Full version: IACR ePrint 2025/2051. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 24:1–24:18. DOI: 10.4230/LIPIcs.CCC.2026.24.

[BCHKS26] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. "On Proximity Gaps of Reed-Solomon Codes". In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26)*. Full version: IACR ePrint 2025/2055 (2025-11-06); also ECCC TR25-169. ACM, 2026, pp. 1157–1167. DOI: 10.1145/3798129.3800827.

[BCI+13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. "Succinct Non-interactive Arguments via Linear Interactive Proofs". In: *Theory of Cryptography (TCC 2013)*. LNCS. Full version: IACR ePrint 2012/718; Journal version: J. Cryptology 35 (2022), doi:10.1007/s00145-022-09424-4. Springer, 2013, pp. 315–333. DOI: 10.1007/978-3-642-36594-2_18.

[BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. "Proximity Gaps for Reed–Solomon Codes". In: *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS 2020)*. Full version: ECCC TR20-083; Journal version: J. ACM 70(5), 2023, doi:10.1145/3614423. 2020, pp. 900–909. DOI: 10.1109/FOCS46700.2020.00088.

[BCL22] Jonathan Bootle, Alessandro Chiesa, and Siqi Liu. "Zero-Knowledge IOPs with Linear-Time Prover and Polylogarithmic-Time Verifier". In: *Advances in Cryptology – EUROCRYPT 2022*. LNCS. Full version: IACR ePrint 2020/1527 (dated 2021-10-14). Springer, 2022, pp. 275–304. DOI: 10.1007/978-3-031-07085-3_10.

[BCM26] Jonathan Bootle, Alessandro Chiesa, and Mathias Marty. "Round-by-Round Knowledge Soundness for Linear-Time IOPs". In: *IACR Communications in Cryptology* 3.3 (2026). DOI: 10.62056/a36chevtw.

[BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. "Recursive Proof Composition from Accumulation Schemes". In: *Theory of Cryptography – 18th International Conference, TCC 2020, Part II*. Vol. 12551. LNCS. Full version: "Proof-Carrying Data from Accumulation Schemes", IACR ePrint 2020/499. Springer, 2020, pp. 1–18. DOI: 10.1007/978-3-030-64378-2_1.

[BCP+26] Joshua Brakensiek, Yeyuan Chen, Aaron Putterman, Zihan Zhang, and Kai Zhe Zheng. *Algorithmic List Decoding of Reed–Solomon Codes up to Capacity*. Tech. rep. TR26-164. Revision 1; preprint. Electronic Colloquium on Computational Complexity (ECCC), 2026.

[BCR+19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. "Aurora: Transparent Succinct Arguments for R1CS". In: *Advances in Cryptology – EUROCRYPT 2019, Part I*. Vol. 11476. LNCS. Full version: IACR ePrint 2018/828. Springer, 2019, pp. 103–128. DOI: 10.1007/978-3-030-17653-2_4.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Theory of Cryptography (TCC 2016-B)*. Vol. 9986. LNCS. Full version: IACR ePrint 2016/116. Springer, 2016, pp. 31–60. DOI: 10.1007/978-3-662-53644-5_2.

[BCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: *Advances in Cryptology – CRYPTO 2014, Part II*. Vol. 8617. LNCS. Full version: IACR ePrint 2014/595. Springer, 2014, pp. 276–294. DOI: 10.1007/978-3-662-44381-1_16.

[BDH+25] Juraj Belohorec, Pavel Dvořák, Charlotte Hoffmann, Pavel Hubáček, Kristýna Mašková, and Martin Pastyřík. "On Extractability of the KZG Family of Polynomial Commitment Schemes". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Springer, 2025, pp. 584–616. DOI: 10.1007/978-3-032-01887-8_19.

[BFK+24] Alexander R. Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. "Field-Agnostic SNARKs from Expand-Accumulate Codes". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 276–307. DOI: 10.1007/978-3-031-68403-6_9.

[BFL90] László Babai, Lance Fortnow, and Carsten Lund. "Non-Deterministic Exponential Time Has Two-Prover Interactive Protocols". In: *Proceedings of the 31st Annual Symposium on Foundations of Computer Science (FOCS 1990)*. Journal version: Computational Complexity 1 (1991), 3–40. IEEE Computer Society, 1990, pp. 16–25. DOI: 10.1109/FSCS.1990.89520.

[BFN26] David Balbás, Dario Fiore, and Duy Nguyen. "Splitting Bilinear Groups: New Translations from Composite- to Prime-Order with Applications to Batch Arguments for NP". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 362–395. DOI: 10.1007/978-3-032-35424-2_13.

[BFRW26] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. "TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 297–329. DOI: 10.1007/978-3-032-35424-2_11.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Advances in Cryptology – EUROCRYPT 2020,*

Part I. Vol. 12105. LNCS. IACR ePrint 2019/1229; revised security proof after a gap found by Block et al. (CRYPTO 2021). Springer, 2020, pp. 677–706. DOI: 10.1007/978-3-030-45721-1_24.

[BG12] Stephanie Bayer and Jens Groth. "Efficient Zero-Knowledge Argument for Correctness of a Shuffle". In: *Advances in Cryptology – EUROCRYPT 2012*. Vol. 7237. LNCS. Springer, 2012, pp. 263–280. DOI: 10.1007/978-3-642-29011-4_17.

[BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. *Recursive Proof Composition without a Trusted Setup*. IACR Cryptology ePrint Archive, Report 2019/1021. 2019.

[BGKW88] Michael Ben-Or, Shafi Goldwasser, Joe Kilian, and Avi Wigderson. "Multi-Prover Interactive Proofs: How to Remove Intractability Assumptions". In: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC '88)*. ACM, 1988, pp. 113–131. DOI: 10.1145/62212.62223.

[BHK+26] Juraj Belohorec, Pavel Hubáček, Aleksi Kalsta, and Kristýna Mašková. "Chopin: Optimal Pairing-Based Multilinear Polynomial Commitments from Bivariate KZG". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Full version: IACR ePrint 2026/480. Springer, 2026, pp. 428–459. DOI: 10.1007/978-3-032-35424-2_15.

[BHK26] Chris Brzuska, Pavel Hubáček, and Aleksi Kalsta. "Simple Attacks Against (Extended) Fiat-Shamir". In: *Public-Key Cryptography – PKC 2026*. Vol. 16551. LNCS. Read the Aalto electronic reprint (accepted author manuscript, own pagination 1–34). Springer, 2026, pp. 510–543. DOI: 10.1007/978-3-032-26731-3_17.

[BHR+21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. "Time- and Space-Efficient Arguments from Groups of Unknown Order". In: *Advances in Cryptology – CRYPTO 2021, Part IV*. Vol. 12828. LNCS. Full version: IACR ePrint 2021/358 (version of 2021-09-21). Springer, 2021, pp. 123–152. DOI: 10.1007/978-3-030-84259-8_5.

[BMMS26] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. "FICS and FACS: Fast IOPPs and Accumulation via Code-Switching". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 557–591. DOI: 10.1007/978-3-032-35424-2_19.

[BMNW25] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. "ARC: Accumulation for Reed–Solomon Codes". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 128–160. DOI: 10.1007/978-3-032-01907-3_5.

[BMNW25b] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. "Accumulation Without Homomorphism". In: *16th Innovations in Theoretical Computer Science Conference (ITCS 2025)*. Vol. 325. LIPIcs. Full version: IACR ePrint 2024/474. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 23:1–23:25. DOI: 10.4230/LIPIcs.ITCS.2025.23.

[BMSS25] Benedikt Bünz, Tushar Mopuri, Alireza Shirzad, and Sriram Sridhar. "DewTwo: A Transparent PCS with Quasi-Linear Prover, Logarithmic Verifier and 4.5KB Proofs from Falsifiable Assumptions". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Springer, 2025, pp. 549–583. DOI: 10.1007/978-3-032-01887-8_18.

[BNP26] David Balbás, Anca Nitulescu, and Maxime Plançon. *ProtogaLattice: Lattice-based Algebraic Folding*. Cryptology ePrint Archive, Paper 2026/1317. Version dated 2026-09-18. 2026.

[BR93] Mihir Bellare and Phillip Rogaway. "Random Oracles are Practical: A Paradigm for Designing Efficient Protocols". In: *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS 1993)*. 1993, pp. 62–73. DOI: 10.1145/168588.168596.

[BS23] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Free online draft, Version 0.6, January 2023. 2023.

[BS23b] Ward Beullens and Gregor Seiler. "LaBRADOR: Compact Proofs for R1CS from Module-SIS". In: *Advances in Cryptology – CRYPTO 2023, Part V*. Vol. 14085. LNCS. Springer, 2023, pp. 518–548. DOI: 10.1007/978-3-031-38554-4_17.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Advances in Cryptology – EUROCRYPT 2023, Part II*. Vol. 14005. LNCS. Full version: IACR ePrint 2022/1355. Springer, 2023, pp. 499–530. DOI: 10.1007/978-3-031-30617-4_17.

[CCG+23] Megan Chen, Alessandro Chiesa, Tom Gur, Jack O'Connor, and Nicholas Spooner. "Proof-Carrying Data from Arithmetized Random Oracles". In: *Advances in Cryptology – EUROCRYPT 2023, Part II*. Vol. 14005. LNCS. Full version: IACR ePrint 2023/587. Springer, 2023, pp. 379–404. DOI: 10.1007/978-3-031-30617-4_13.

[CCH+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. "Fiat-Shamir: From Practice to Theory". In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*. 2019, pp. 1082–1090. DOI: 10.1145/3313276.3316380.

[CCN24] Maria Corte-Real Santos, Craig Costello, and Michael Naehrig. "On Cycles of Pairing-Friendly Abelian Varieties". In: *Advances in Cryptology – CRYPTO 2024, Part IX*. Vol. 14928. LNCS. Springer, 2024, pp. 221–253. DOI: 10.1007/978-3-031-68400-5_7.

[CET+24] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. "SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs". In: *33rd USENIX Security Symposium (USENIX Security 24)*. Full version: arXiv:2402.15293v4. USENIX Association, 2024, pp. 3855–3872.

[CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. *Zero-Knowledge IOPPs for Constrained Interleaved Codes*. Cryptology ePrint Archive, Paper 2026/391. Version dated 2026-02-25. 2026.

[CGH04] Ran Canetti, Oded Goldreich, and Shai Halevi. "The Random Oracle Methodology, Revisited". In: *Journal of the ACM* 51.4 (2004), pp. 557–594. DOI: 10.1145/1008731.1008734.

[CGKS23] Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. "Impossibilities in Succinct Arguments: Black-Box Extraction and More". In: *Progress in Cryptology – AFRICACRYPT 2023*. Vol. 14064. LNCS. Cited version: revised full version, IACR ePrint 2022/638 (2024). Springer, 2023, pp. 465–489. DOI: 10.1007/978-3-031-37679-5_20.

[CGKY26] Alessandro Chiesa, Ziyi Guan, Christian Knabenhans, and Zihan Yu. "On the Fiat–Shamir Security of Succinct Arguments from Functional Commitments". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Full version: IACR ePrint 2025/902, version of 2026-06-02. Springer, 2026, pp. 189–213. DOI: 10.1007/978-3-032-35424-2_7.

[Che25] Binyi Chen. *Symphony: Scalable SNARKs in the Random Oracle Model from Lattice-Based High-Arity Folding*. Cryptology ePrint Archive, Paper 2025/1905. To appear at ASIACRYPT 2026; version of 2026-05-06. 2025.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Advances in Cryptology – EUROCRYPT 2020, Part I*. Vol. 12105. LNCS. IACR ePrint 2019/1047. Springer, 2020, pp. 738–768. DOI: 10.1007/978-3-030-45721-1_26.

[CHS26] Petr Chmel, Pavel Hubáček, and Dominik Stejskal. *Knowledge Soundness of Polynomial Commitments in the Algebraic Group Model Does Not Guarantee Extractability*. Cryptology ePrint Archive, Paper 2026/284. Version of 2026-10-29. 2026.

[CMNW24] Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. "Polynomial Commitments from Lattices: Post-quantum Security, Fast Verification and Transparent Setup". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 207–242. DOI: 10.1007/978-3-031-68403-6_7.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. "Succinct Arguments in the Quantum Random Oracle Model". In: *Theory of Cryptography – TCC 2019, Part II*. Vol. 11892. LNCS. Cited version: IACR ePrint 2019/834 (version of 2020-01-14). Springer, 2019, pp. 1–29. DOI: 10.1007/978-3-030-36033-7_1.

[COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. "Fractal: Post-Quantum and Transparent Recursive Proofs from Holography". In: *Advances in Cryptology – EUROCRYPT 2020, Part I*. Vol. 12105. LNCS. IACR ePrint 2019/1076. Springer, 2020, pp. 769–793. DOI: 10.1007/978-3-030-45721-1_27.

[CS26] Elizabeth Crites and Alistair Stewart. "On Reed–Solomon Proximity Gaps Conjectures". In: *Advances in Cryptology – CRYPTO 2026, Part IV*. Vol. 16803. LNCS. Springer, 2026, pp. 128–156. DOI: 10.1007/978-3-032-35398-6_5.

[DFS24] Thomas Debris-Alazard, Pouria Fallahpour, and Damien Stehlé. "Quantum Oblivious LWE Sampling and Insecurity of Standard Model Lattice-Based SNARKs". In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC '24)*. Full version: IACR ePrint 2024/030 (PDF dated 2024-05-14). ACM, 2024, pp. 423–434. DOI: 10.1145/3618260.3649766.

[DG26] Marian Dietz and Emanuele Giunta. "Bulletproofs are Optimal: Lower Bounds for Vector Commitments from Fiat-Shamir in Pairing-Free Groups". In: *Advances in Cryptology – CRYPTO 2026, Part I*. Vol. 16800. LNCS. Springer, 2026, pp. 218–249. DOI: 10.1007/978-3-032-35367-2_8.

[DG26b] Benjamin E. Diamond and Angus Gruen. "On the Distribution of the Distances of Random Words". In: *Advances in Cryptology – CRYPTO 2026, Part IV*. Vol. 16803. LNCS. ePrint 2025/2010. Springer, 2026, pp. 96–127. DOI: 10.1007/978-3-032-35398-6_4.

[DGKV22] Lalita Devadas, Rishab Goyal, Yael Tauman Kalai, and Vinod Vaikuntanathan. "Rate-1 Non-Interactive Arguments for Batch-NP and Applications". In: *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. Cited from the abstract; ePrint 2022/1236. IEEE, 2022, pp. 1057–1068. DOI: 10.1109/FOCS54457.2022.00103.

[DGT+26] Sagar Dhawan, Markos Georghiades, Justin Thaler, Andrew Tretyakov, and Michael Zhu. *Putting the zk in zkVM: Jolt Now Supports Zero Knowl-*

edge. https://a16zfrontier.com/posts/article/zkvm-jolt-zero-knowledge. 2026.

[DHK+26] Lalita Devadas, Samuel B. Hopkins, Yael Tauman Kalai, Pravesh K. Kothari, Alex Lombardi, and Surya Mathialagan. "SNARGs for NP and Non-Signaling PCPs, Revisited". In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26)*. Full version: IACR ePrint 2026/006 (dated 2026-01-08). ACM, 2026, pp. 665–674. DOI: 10.1145/3798129.3800782.

[Dia25] Benjamin E. Diamond. *Zero-Knowledge Polynomial Commitment in Binary Fields*. Cryptology ePrint Archive, Paper 2025/1015. Version of 2025-06-02. 2025.

[DJJ+25] Pratish Datta, Abhishek Jain, Zhengzhong Jin, Alexis Korb, Surya Mathialagan, and Amit Sahai. "Incrementally Verifiable Computation for NP from Standard Assumptions". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 611–642. DOI: 10.1007/978-3-032-01907-3_20.

[DKT26] Quang Dao, Scott Duke Kominers, and Justin Thaler. *Reed–Solomon Codes Beyond Johnson: Efficient Decoding and Smaller Cryptographic Proofs*. Cryptology ePrint Archive, Paper 2026/2056. Version dated September 2026. 2026.

[DKTZ26] Quang Dao, Scott Duke Kominers, Justin Thaler, and Kai Zhe Zheng. *Counterexamples to Beyond-Johnson Proximity Gaps over Binary Fields*. Cryptology ePrint Archive, Paper 2026/2424. Version dated 6 October 2026. 2026.

[DMS26] Michel Dellepere, Pratyush Mishra, and Alireza Shirzad. "Garuda and Pari: Faster and Smaller SNARKs via Equifficient Polynomial Commitments". In: *35th USENIX Security Symposium (USENIX Security 26)*. Full version: IACR ePrint 2024/1245. USENIX Association, 2026, pp. 5791–5810.

[DMWG23] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. "Weak Fiat-Shamir Attacks on Modern Proof Systems". In: *2023 IEEE Symposium on Security and Privacy (SP)*. IACR ePrint 2023/691. IEEE, 2023, pp. 199–216. DOI: 10.1109/SP46215.2023.10179408.

[DP25] Benjamin E. Diamond and Jim Posen. "Succinct Arguments over Towers of Binary Fields". In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Full version: IACR ePrint 2023/1784. Springer, 2025, pp. 93–122. DOI: 10.1007/978-3-031-91134-7_4.

[DP26] Benjamin E. Diamond and Jim Posen. "Polylogarithmic Proofs for Multilinears over Binary Towers". In: *Advances in Cryptology – EUROCRYPT 2026*. LNCS. Cited version: IACR ePrint 2024/504 (version of 2025-09-22). Springer, 2026, pp. 3–32. DOI: 10.1007/978-3-032-25336-1_1.

[DS25] Thomas den Hollander and Daniel Slamanig. "A Crack in the Firmament: Restoring Soundness of the Orion Proof System and More". In: *Advances in Cryptology – ASIACRYPT 2025, Part V*. Vol. 16249. LNCS. Cited version: IACR ePrint 2024/1164 (version 2025-02-14). Springer, 2025, pp. 396–426. DOI: 10.1007/978-981-95-5116-3_13.

[EF25] Ethereum Foundation. *Shipping an L1 zkEVM #2: The Security Foundations*. https://blog.ethereum.org/2025/12/18/zkevm-security-foundations. 2025.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. *cq: Cached quotients for fast lookups*. Cryptology ePrint Archive, Paper 2022/1763. Version of 2024-07-05. 2022.

[EG23] Liam Eagen and Ariel Gabizon. *Protogalaxy: Efficient Protostar-style folding of multiple instances*. Cryptology ePrint Archive, Paper 2023/1106. Version of January 29, 2024. 2023.

[Fen26] Giacomo Fenzi. *How to Prove More False Statements: Fiat–Shamir Limitations on (Generated) R1CS*. Cryptology ePrint Archive, Paper 2026/1838. Version dated 2026-09-09. 2026.

[FR26] Antonio Faonio and Luigi Russo. "Sumcheck-Based zkSNARKs are Non-malleable". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 35–61. DOI: 10.1007/978-3-032-35424-2_2.

[FS26] Giacomo Fenzi and Antonio Sanso. "Small-field hash-based SNARGs are less sound than conjectured". In: *IACR Communications in Cryptology* 3.3 (2026). Preprint: IACR ePrint 2025/2197. DOI: 10.62056/akdkmpxqi.

[FS26b] Georg Fuchsbauer and Marek Sefranek. *Plonk Without Random Oracles*. Cryptology ePrint Archive, Paper 2026/200. Version of 2026-02-07; preprint. 2026.

[FS86] Amos Fiat and Adi Shamir. "How To Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology – CRYPTO 1986*. Vol. 263. LNCS. Springer, 1986, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[Fu25] Shihui Fu. "Improved Constant-Sized Polynomial Commitment Schemes Without Trusted Setup". In: *IACR Communications in Cryptology* 2.2 (2025). The scheme is called Dew++. DOI: 10.62056/anudy76bm.

[FW26] Cody Freitag and Daniel Wichs. "Unique SNARGs with Adaptive Security: Constructions and Black-Box Separations". In: *Advances in Cryptology – CRYPTO 2026, Part I*. Vol. 16800. LNCS. Springer, 2026, pp. 190–217. DOI: 10.1007/978-3-032-35367-2_7.

[GG26] Rohan Goyal and Venkatesan Guruswami. "Optimal Proximity Gaps for Subspace-Design Codes and (Random) Reed-Solomon Codes". In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26)*. Full version: IACR ePrint 2025/2054 (revision of March 2026). ACM, 2026, pp. 1558–1568. DOI: 10.1145/3798129.3800864.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. "Quadratic Span Programs and Succinct NIZKs without PCPs". In: *Advances in Cryptology – EUROCRYPT 2013*. Vol. 7881. LNCS. Full version: IACR ePrint 2012/215. Springer, 2013, pp. 626–645. DOI: 10.1007/978-3-642-38348-9_37.

[GKM+18] Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. "Updatable and Universal Common Reference Strings with Applications to zk-SNARKs". In: *Advances in Cryptology – CRYPTO 2018, Part III*. Vol. 10993. LNCS. Full version: IACR ePrint 2018/280. Springer, 2018, pp. 698–728. DOI: 10.1007/978-3-319-96878-0_24.

[GKRRS21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems". In: *30th USENIX Security Symposium (USENIX Security 21)*. Cited version: updated version of IACR ePrint 2019/458 (version of 2023-07-05). USENIX Association, 2021, pp. 519–535.

[GLS+23] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS". In: *Advances in Cryptology – CRYPTO 2023, Part II*. Vol. 14082.

LNCS. Full version: IACR ePrint 2021/1043. Springer, 2023, pp. 193–226. DOI: 10.1007/978-3-031-38545-2_7.

[GMN22] Nicolas Gailly, Mary Maller, and Anca Nitulescu. "SnarkPack: Practical SNARK Aggregation". In: *Financial Cryptography and Data Security (FC 2022)*. LNCS. Cited version: IACR ePrint 2021/529 (version of 2021-09-06). Springer, 2022, pp. 203–229. DOI: 10.1007/978-3-031-18283-9_10.

[GMR85] S. Goldwasser, S. Micali, and C. Rackoff. "The Knowledge Complexity of Interactive Proof-Systems". In: *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing (STOC '85)*. ACM, 1985, pp. 291–304. DOI: 10.1145/22145.22178.

[GNR26] Kobi Gurkan, Andrija Novakovic, and Ron D. Rothblum. "Bolt: Faster SNARKs from Sketched Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 396–427. DOI: 10.1007/978-3-032-35424-2_14.

[GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. "Samaritan: Linear-Time Prover SNARK from New Multilinear Polynomial Commitments". In: *Advances in Cryptology – ASIACRYPT 2025, Part V*. Vol. 16249. LNCS. Cited version: IACR ePrint 2025/419 (version 2025-11-04). Springer, 2025, pp. 456–489. DOI: 10.1007/978-981-95-5116-3_15.

[Gro16] Jens Groth. "On the Size of Pairing-based Non-interactive Arguments". In: *Advances in Cryptology – EUROCRYPT 2016, Part II*. Vol. 9666. LNCS. Full version: IACR ePrint 2016/260. Springer, 2016, pp. 305–326. DOI: 10.1007/978-3-662-49896-5_11.

[GW11] Craig Gentry and Daniel Wichs. "Separating Succinct Non-Interactive Arguments from All Falsifiable Assumptions". In: *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC '11)*. Full version: IACR ePrint 2010/610, version of 2013-06-06. ACM, 2011, pp. 99–108. DOI: 10.1145/1993636.1993651.

[GW20] Ariel Gabizon and Zachary J. Williamson. *plookup: A simplified polynomial protocol for lookup tables*. Cryptology ePrint Archive, Paper 2020/315. Version of November 20, 2020. 2020.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PlonK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. IACR Cryptology ePrint Archive, Report 2019/953. 2019.

[GZL+26] Yonghui Guan, Rihe Zhang, Bin Liu, Tianyu Zhao, Jialu Hao, and Antonis Michalas. "SoK: PIOP-based SNARKs for General Computation". In: *ACM ASIA Public-Key Cryptography Workshop (APKC '26)*. Full version: IACR ePrint 2026/1133. ACM, 2026, pp. 31–40. DOI: 10.1145/3803627.3805818.

[HA24] Ulrich Haböck and Al Kindi. *A note on adding zero-knowledge to STARKs*. Cryptology ePrint Archive, Paper 2024/1037. Version dated 2025-02-20 (last revision 2025-02-22). 2024.

[Hab22] Ulrich Haböck. *Multivariate lookups based on logarithmic derivatives*. Cryptology ePrint Archive, Paper 2022/1530. Version of 2023-03-31. 2022.

[HJLM26] Yao-Ching Hsieh, Abhishek Jain, Jiatu Li, and Surya Mathialagan. "SNARGs for NP from Unprovability of Mathematical Theorems (Or: How to Use the Simplicity of Cryptographic Reasoning)". In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26)*. Full version: IACR ePrint 2026/1180. ACM, 2026, pp. 31–41. DOI: 10.1145/3798129.3800724.

[HJR+26] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. "Jagged Polynomial Commitments (or: How to Stack Multilin-

ears)". In: *Advances in Cryptology – EUROCRYPT 2026*. LNCS. IACR ePrint 2025/917 (version dated 2026-04-10). Springer, 2026, pp. 121–147. DOI: 10.1007/978-3-032-25336-1_5.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. *Circle STARKs*. Cryptology ePrint Archive, Paper 2024/278. Version dated 2024-12-17. 2024.

[HN23] Mathias Hall-Andersen and Jesper Buus Nielsen. "On Valiant's Conjecture: Impossibility of Incrementally Verifiable Computation from Random Oracles". In: *Advances in Cryptology – EUROCRYPT 2023, Part II*. Vol. 14005. LNCS. Cited version: IACR ePrint 2022/542. Springer, 2023, pp. 438–469. DOI: 10.1007/978-3-031-30617-4_15.

[HNY22] Iftach Haitner, Daniel Nukrai, and Eylon Yogev. "Lower Bound on SNARGs in the Random Oracle Model". In: *Advances in Cryptology – CRYPTO 2022*. LNCS. Full version: IACR ePrint 2022/178 (dated 2022-11-09). Springer, 2022, pp. 97–127. DOI: 10.1007/978-3-031-15982-4_4.

[HR22] Justin Holmgren and Ron D. Rothblum. "Faster Sounder Succinct Arguments and IOPs". In: *Advances in Cryptology – CRYPTO 2022, Part I*. Vol. 13507. LNCS. Springer, 2022, pp. 474–503. DOI: 10.1007/978-3-031-15802-5_17.

[HWC26] Christoph Hochrainer, Valentin Wüstholz, and Maria Christakis. "Arguzz: Testing zkVMs for Soundness and Completeness Bugs". In: *35th USENIX Security Symposium (USENIX Security 26)*. arXiv:2509.10819. USENIX Association, 2026, pp. 1807–1826.

[IKO07] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. "Efficient Arguments without Short PCPs". In: *Twenty-Second Annual IEEE Conference on Computational Complexity (CCC 2007)*. IEEE Computer Society, 2007, pp. 278–291. DOI: 10.1109/CCC.2007.10.

[Jer26] Fernando Granha Jeronimo. *Algorithmic List Decoding at Capacity and Optimal Proximity Gaps for Reed–Solomon Codes*. Tech. rep. TR26-169. Version of 2026-09-05; preprint. Electronic Colloquium on Computational Complexity (ECCC), 2026.

[JLP26] Zhengzhong Jin, Mingqi Lu, and Bo Peng. "SNARKs from LWE via Non-black-Box Reductions". In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26)*. ACM, 2026, pp. 1812–1823. DOI: 10.1145/3798129.3800888.

[JMW26] Abhishek Jain, Surya Mathialagan, and Brent Waters. "Incrementally Verifiable Computation Without Extraction". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 3–34. DOI: 10.1007/978-3-032-35424-2_1.

[Jo26] Sunghyeon Jo. *Reed–Solomon Mutual Correlated Agreement Beyond the Johnson Radius*. Cryptology ePrint Archive, Paper 2026/1432. Version of 2026-08-19; preprint. 2026.

[Kil92] Joe Kilian. "A Note on Efficient Zero-Knowledge Proofs and Arguments". In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC '92)*. ACM, 1992, pp. 723–732. DOI: 10.1145/129712.129782.

[KKH26] Dmitry Krachun, Stepan Kazanin, and Ulrich Haböck. *Failure of proximity gaps close to capacity*. Cryptology ePrint Archive, Paper 2026/782. 2026.

[KLN+26] Michael Klooß, Russell W. F. Lai, Ngoc Khanh Nguyen, Michał Osadnik, and Lorenzo Tucci. *RoKoko: Lattice-based Succinct Arguments, a Committed Refinement*. Cryptology ePrint Archive, Paper 2026/575. Revision of 2026-09-09. 2026.

[KP23] Abhiram Kothapalli and Bryan Parno. "Algebraic Reductions of Knowledge". In: *Advances in Cryptology – CRYPTO 2023, Part IV*. Vol. 14084. LNCS. Springer, 2023, pp. 669–701. DOI: 10.1007/978-3-031-38551-3_21.

[KPS26] Shubham Khurana, Sahadeo Padhye, and Rajeev Anand Sahu. "Zero-Knowledge Extension of PARI". In: *IACR Communications in Cryptology* 3.2 (2026). DOI: 10.62056/ahjb0l2hd.

[KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. "How to Prove False Statements: Practical Attacks on Fiat-Shamir". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Full version: IACR ePrint 2025/118. Springer, 2025, pp. 3–26. DOI: 10.1007/978-3-032-01887-8_1.

[KS22] Abhiram Kothapalli and Srinath Setty. *SuperNova: Proving Universal Machine Executions Without Universal Circuits*. IACR Cryptology ePrint Archive, Paper 2022/1758. 2022.

[KS24] Abhiram Kothapalli and Srinath Setty. "HyperNova: Recursive Arguments for Customizable Constraint Systems". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Full version: IACR ePrint 2023/573. Springer, 2024, pp. 345–379. DOI: 10.1007/978-3-031-68403-6_11.

[KS26] Abhiram Kothapalli and Srinath Setty. "NeutronNova: Group-Based Folding Done Right". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Extended version: IACR ePrint 2024/1606. Springer, 2026, pp. 524–556. DOI: 10.1007/978-3-032-35424-2_18.

[KSHC26] Arman Kolozyan, Tom Sorger, Alexander Hicks, and Stefanos Chaliasos. *ZKP Security Tools and Verification: Coverage, Effectiveness, Adoption, and Challenges*. arXiv:2607.23752. v1, 26 Jul 2026. 2026.

[KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes". In: *Advances in Cryptology – CRYPTO 2022, Part IV*. Vol. 13510. LNCS. IACR ePrint 2021/370. Springer, 2022, pp. 359–388. DOI: 10.1007/978-3-031-15985-5_13.

[KTZ26] Scott Duke Kominers, Justin Thaler, and Kai Zhe Zheng. *The One-and-a-Half Johnson Bound Is Tight for Proximity Gaps of General Linear Codes*. Cryptology ePrint Archive, Paper 2026/1894. Preprint dated August 2026. 2026.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. *Polynomial Commitments*. Tech. rep. 2010-10. Extended version of the ASIACRYPT 2010 paper. Centre for Applied Cryptographic Research (CACR), 2010.

[Lee21] Jonathan Lee. "Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments". In: *Theory of Cryptography Conference (TCC 2021), Part II*. Vol. 13043. LNCS. IACR ePrint 2020/1274. Springer, 2021, pp. 1–34. DOI: 10.1007/978-3-030-90453-1_1.

[LHW+25] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. "SoK: Understanding zk-SNARKs: The Gap Between Research and Practice". In: *34th USENIX Security Symposium (USENIX Security 25)*. Full version: IACR ePrint 2025/172; arXiv:2502.02387. USENIX Association, 2025, pp. 2085–2104.

[Lip24] Helger Lipmaa. "Polymath: Groth16 Is Not the Limit". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 170–206. DOI: 10.1007/978-3-031-68403-6_6.

[Lip25] Helger Lipmaa. "Plonk is Simulation Extractable in ROM Under Falsifiable Assumptions". In: *Theory of Cryptography – TCC 2025, Part IV*. Vol. 16271.

LNCS. Full version: IACR ePrint 2025/1759. Springer, 2025, pp. 3–36. DOI: 10.1007/978-3-032-12290-2_1.

[LPS24] Helger Lipmaa, Roberto Parisella, and Janno Siim. "Constant-Size zk-SNARKs in ROM from Falsifiable Assumptions". In: *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Vol. 14656. LNCS. Full version: IACR ePrint 2024/173 (revision of 2025-06-04). Springer, 2024, pp. 34–64. DOI: 10.1007/978-3-031-58751-1_2.

[LPS25] Helger Lipmaa, Roberto Parisella, and Janno Siim. "On Knowledge-Soundness of Plonk in ROM from Falsifiable Assumptions". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: IACR ePrint 2024/994. Springer, 2025, pp. 362–395. DOI: 10.1007/978-3-032-01907-3_12.

[LS24] Hyeonbum Lee and Jae Hong Seo. *On the Security of Nova Recursive Proof System: Limitations of and Alternatives to Bounded-Depth Analysis*. Cryptology ePrint Archive, Paper 2024/232. Revision of 2026-02-13. 2024.

[MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. "Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. Full version: IACR ePrint 2019/099. ACM, 2019, pp. 2111–2128. DOI: 10.1145/3319535.3339817.

[Mer89] Ralph C. Merkle. "A Certified Digital Signature". In: *Advances in Cryptology – CRYPTO 1989*. Vol. 435. LNCS. Springer, 1989, pp. 218–238. DOI: 10.1007/0-387-34805-0_21.

[Mic94] Silvio Micali. "CS Proofs". In: *Proceedings of the 35th Annual IEEE Symposium on Foundations of Computer Science (FOCS '94)*. IEEE, 1994, pp. 436–453. DOI: 10.1109/SFCS.1994.365746.

[MPV24] Surya Mathialagan, Spencer Peters, and Vinod Vaikuntanathan. "Adaptively Sound Zero-Knowledge SNARKs for UP". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 38–71. DOI: 10.1007/978-3-031-68403-6_2.

[NA25] Andrija Novakovic and Guillermo Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. Cryptology ePrint Archive, Paper 2025/1187. Version of May 2025. 2025.

[NBS23] Wilson Nguyen, Dan Boneh, and Srinath Setty. "Revisiting the Nova Proof System on a Cycle of Curves". In: *5th Conference on Advances in Financial Technologies (AFT 2023)*. Vol. 282. LIPIcs. Full version: IACR ePrint 2023/969. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 18:1–18:22. DOI: 10.4230/LIPIcs.AFT.2023.18.

[NDC+24] Wilson Nguyen, Trisha Datta, Binyi Chen, Nirvan Tyagi, and Dan Boneh. "Mangrove: A Scalable Framework for Folding-Based SNARKs". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 308–344. DOI: 10.1007/978-3-031-68403-6_10.

[Nit20] Anca Nitulescu. *zk-SNARKs: A Gentle Introduction*. Manuscript. 2020.

[NS24] Ngoc Khanh Nguyen and Gregor Seiler. "Greyhound: Fast Polynomial Commitments from Lattices". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Full version: IACR ePrint 2024/1293. Springer, 2024, pp. 243–275. DOI: 10.1007/978-3-031-68403-6_8.

[NS26] Wilson Nguyen and Srinath Setty. "Neo and SuperNeo: Post-quantum Folding with Pay-per-Bit Costs over Small Fields". In: *Advances in Cryptology –*

CRYPTO 2026, Part IX. Vol. 16808. LNCS. Springer, 2026, pp. 492–523. DOI: 10.1007/978-3-032-35424-2_17.

[Ped91] Torben Pryds Pedersen. "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing". In: *Advances in Cryptology – CRYPTO 1991*. Vol. 576. LNCS. Springer, 1991, pp. 129–140. DOI: 10.1007/3-540-46766-1_9.

[PH23] Shahar Papini and Ulrich Haböck. *Improving logarithmic derivative lookups using GKR*. Cryptology ePrint Archive, Paper 2023/1284. Version of 2023-09-18. 2023.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. "Pinocchio: Nearly Practical Verifiable Computation". In: *2013 IEEE Symposium on Security and Privacy*. Full version: IACR ePrint 2013/279. IEEE, 2013, pp. 238–252. DOI: 10.1109/SP.2013.47.

[Plonky2] Polygon Zero Team. *Plonky2 and More (README)*. https://github.com/0xPolygonZero/plonky2. Software repository, archived 2026-10-23. 2026.

[Plonky3] Polygon Zero Team. *Plonky3: A Toolkit for Polynomial IOPs*. https://github.com/Plonky3/Plonky3. Software repository.

[Pol22] Polygon Zero Team. *Plonky2: Fast Recursive Arguments with PLONK and FRI*. Whitepaper (draft), https://github.com/0xPolygonZero/plonky2. Dated 2022-09-07; not peer-reviewed. 2022.

[Pol26] Polygon Labs. *Polygon zkEVM: Mainnet Beta Sunset & Fund Claims*. https://polygon.technology/polygon-zkevm. Accessed 2026-10-28. 2026.

[PP22] Omer Paneth and Rafael Pass. "Incrementally Verifiable Computation via Rate-1 Batch Arguments". In: *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. Cited from the abstract; ePrint 2023/1394. IEEE, 2022, pp. 1045–1056. DOI: 10.1109/FOCS54457.2022.00102.

[PP25] Christodoulos Pappas and Dimitrios Papadopoulos. "HOBBIT: Space-Efficient zkSNARK with Optimal Prover Time". In: *34th USENIX Security Symposium (USENIX Security 25)*. Full version (cited pages): IACR ePrint 2025/1214, version of 2025-06-30. USENIX Association, 2025.

[PTX26] Dimitrios Papadopoulos, Qiang Tang, and Jiajun Xin. "Morphic Accumulators and Applications". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 460–491. DOI: 10.1007/978-3-032-35424-2_16.

[QGX+26] Wenjie Qu, Yanpei Guo, Zhen Xuan, Xuanming Liu, and Jiaheng Zhang. "Celer: A Lookup Argument for Large-Scale Queries". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 243–266. DOI: 10.1007/978-3-032-35424-2_9.

[RRR16] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. "Constant-Round Interactive Proofs for Delegating Computation". In: *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC 2016)*. 2016, pp. 49–62. DOI: 10.1145/2897518.2897652.

[RT25] Lior Rotem and Stefano Tessaro. "Straight-Line Knowledge Extraction for Multi-Round Protocols". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 95–127. DOI: 10.1007/978-3-032-01907-3_4.

[RZR22] Noga Ron-Zewi and Ron D. Rothblum. "Proving as Fast as Computing: Succinct Arguments with Constant Prover Overhead". In: *Proceedings of the*

54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022). Cited version: IACR ePrint 2021/1673 (version 2021-12-20); journal version J. ACM 72(2), 2025, doi 10.1145/3721477. ACM, 2022, pp. 1353–1363. DOI: 10.1145/3519935.3519956.

[SAKK25] Nojan Sheybani, Anees Ahmed, Michel Kinsy, and Farinaz Koushanfar. *Zero-Knowledge Proof Frameworks: A Systematic Survey*. arXiv:2502.07063. 2025.

[SB25] István András Seres and Péter Burcsi. "Behemoth: Transparent Polynomial Commitment Scheme with Constant Opening Proof Size and Verifier Time". In: *Progress in Cryptology – AFRICACRYPT 2025*. Vol. 15651. LNCS. IACR ePrint 2023/670. Springer, 2025, pp. 343–369. DOI: 10.1007/978-3-031-97260-7_16.

[Sch80] J. T. Schwartz. "Fast Probabilistic Algorithms for Verification of Polynomial Identities". In: *Journal of the ACM* 27.4 (1980), pp. 701–717. DOI: 10.1145/322217.322225.

[Set20] Srinath Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: *Advances in Cryptology – CRYPTO 2020, Part III*. Vol. 12172. LNCS. IACR ePrint 2019/550. Springer, 2020, pp. 704–737. DOI: 10.1007/978-3-030-56877-1_25.

[Ska26] Christos Skatharoudis. *SoK: Hash-Based Polynomial Commitments and Low-Degree Tests: From FRI to Basefold, STIR, and WHIR*. Cryptology ePrint Archive, Paper 2026/1367. Preprint; read version of 2026-07-02. 2026.

[SL20] Srinath Setty and Jonathan Lee. *Quarks: Quadruple-efficient transparent zkSNARKs*. Cryptology ePrint Archive, Paper 2020/1275. Preprint, last revised 2020-10-24. 2020.

[SP26] Janno Siim and Pritam Pal. "Note on Extractability of the PST Polynomial Commitment Scheme". In: *IACR Communications in Cryptology* 3.3 (2026). Read the author copy dated 2026-10-07 (ePrint 2026/2398). DOI: 10.62056/avzogy10k.

[Sri26] Sriram Sridhar. *Mist: Transparent Polynomial Commitments from Class Group Subset-Product Accumulators*. Cryptology ePrint Archive, Paper 2026/2112. Version dated 2026-09-19. 2026.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable Constraint Systems for Succinct Arguments*. Cryptology ePrint Archive, Paper 2023/552. Preprint. 2023.

[STW24] Srinath Setty, Justin Thaler, and Riad Wahby. "Unlocking the Lookup Singularity with Lasso". In: *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Vol. 14656. LNCS. Full version: IACR ePrint 2023/1216. Springer, 2024, pp. 180–209. DOI: 10.1007/978-3-031-58751-1_7.

[STZ26] Srinath Setty, Justin Thaler, and Michael Zhu. "Twist and Shout: Faster Memory Checking Arguments via One-Hot Addressing and Increments". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 267–296. DOI: 10.1007/978-3-032-35424-2_10.

[Suc26] Succinct. *SP1 Security Model*. https://docs.succinct.xyz/docs/sp1/security/security-model. SP1 Hypercube documentation. 2026.

[SW26] Himanshu Sheoran and Valter Wik. *Unfaithful Claims: Breaking 6 zkVMs*. https://osec.io/blog/2026-03-03-zkvms-unfaithful-claims/. 2026.

[Tha22] Justin Thaler. "Proofs, Arguments, and Zero-Knowledge". In: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), pp. 117–660. DOI: 10.1561/3300000030.

[Val08] Paul Valiant. "Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency". In: *Theory of Cryptography – Fifth Theory of Cryptography Conference, TCC 2008*. Vol. 4948. LNCS. Springer, 2008, pp. 1–18. DOI: 10.1007/978-3-540-78524-8_1.

[WTS+18] Riad S. Wahby, Ioanna Tzialla, abhi shelat abhi, Justin Thaler, and Michael Walfish. "Doubly-Efficient zkSNARKs Without Trusted Setup". In: *2018 IEEE Symposium on Security and Privacy (SP)*. IACR ePrint 2017/1132. IEEE, 2018, pp. 926–943. DOI: 10.1109/SP.2018.00060.

[WW23] Hoeteck Wee and David J. Wu. "Lattice-Based Functional Commitments: Fast Verification and Cryptanalysis". In: *Advances in Cryptology – ASIACRYPT 2023*. Vol. 14442. LNCS. Cited version: IACR ePrint 2024/028. Springer, 2023, pp. 201–235. DOI: 10.1007/978-981-99-8733-7_7.

[WW25] Brent Waters and David J. Wu. "A Pure Indistinguishability Obfuscation Approach to Adaptively-Sound SNARGs for NP". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: IACR ePrint 2024/933. Springer, 2025, pp. 292–326. DOI: 10.1007/978-3-032-01907-3_10.

[XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Orion: Zero Knowledge Proof with Linear Prover Time". In: *Advances in Cryptology – CRYPTO 2022, Part IV*. Vol. 13510. LNCS. Revised full version with errata: IACR ePrint 2022/1010 (major revision, last revised 2023-11-02). Springer, 2022, pp. 299–328. DOI: 10.1007/978-3-031-15985-5_11.

[XZZ+19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. "Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation". In: *Advances in Cryptology – CRYPTO 2019*. Vol. 11694. LNCS. Full version: IACR ePrint 2019/317. Springer, 2019, pp. 733–764. DOI: 10.1007/978-3-030-26954-8_24.

[YLZ+26] Yuanzhuo Yu, Mengling Liu, Yuncong Zhang, Tianyi Ma, Shi-Feng Sun, Man Ho Au, and Dawu Gu. "HyperFond: A Transparent and Post-Quantum Distributed SNARK with Polylogarithmic Communication". In: *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26)*. ACM, 2026, pp. 869–885. DOI: 10.1145/3779208.3805973.

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 138–169. DOI: 10.1007/978-3-031-68403-6_5.

[Zei26] Hadas Zeilberger. "Khatam: Proximity Gaps for Multilinear Evaluation for all Linear Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 592–611. DOI: 10.1007/978-3-032-35424-2_20.

[Zip79] Richard Zippel. "Probabilistic Algorithms for Sparse Polynomials". In: *Symbolic and Algebraic Computation (EUROSAM 1979)*. Vol. 72. LNCS. Springer, 1979, pp. 216–226. DOI: 10.1007/3-540-09519-5_73.

[ZXZS20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof". In: *2020 IEEE Symposium on Security and Privacy (SP)*. Full version: IACR ePrint 2019/1482. IEEE, 2020, pp. 859–876. DOI: 10.1109/SP40000.2020.00052.

A Supplementary tables and timeline

This appendix collects the material behind §7: Figure 7 gives a timeline of selected papers this survey draws on, Table A.1 the exact counts each paper reports, and Table A.2 the zero-knowledge arguments that miss the strict bound or meet it only conditionally (§5.4).

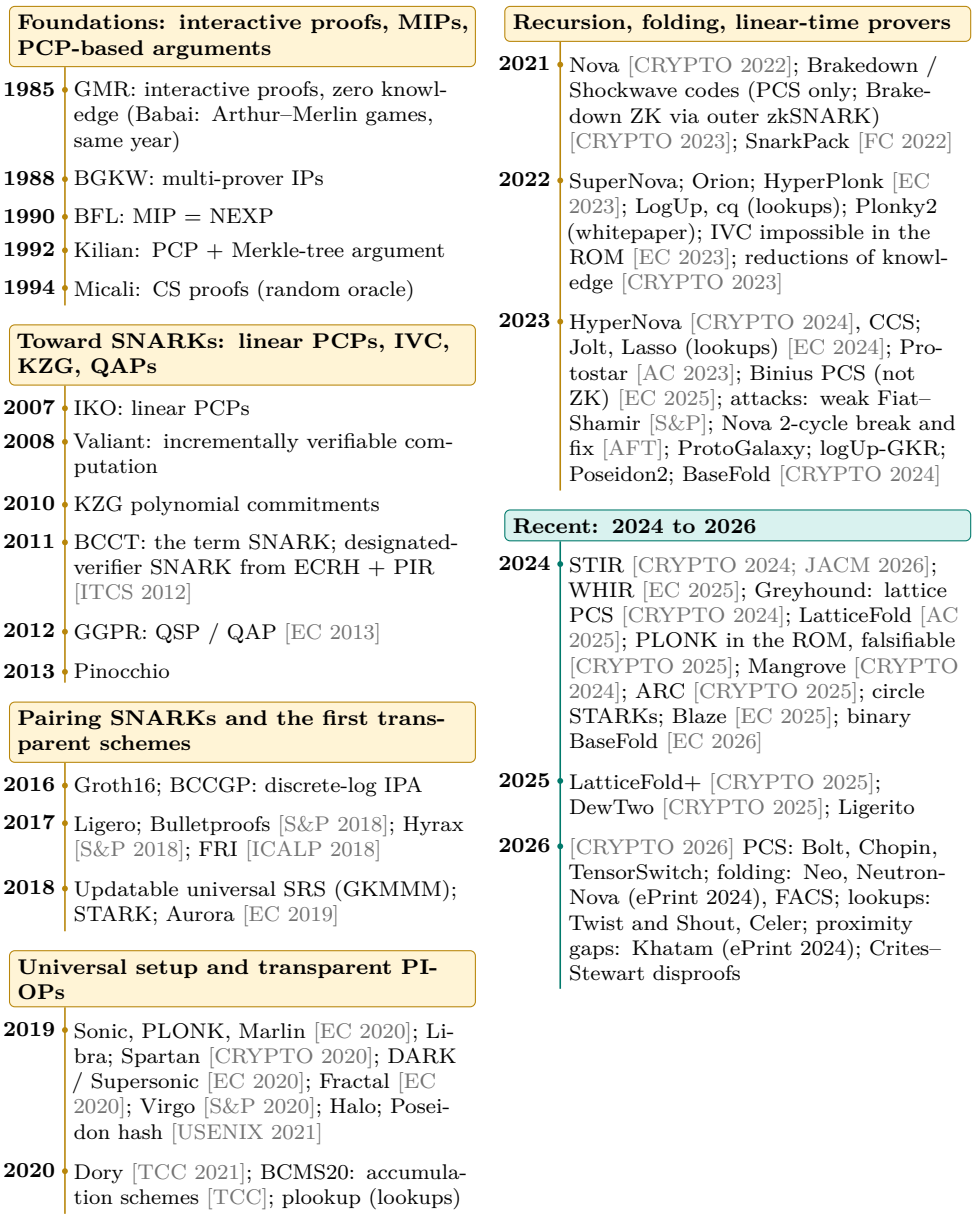


Figure 7: Construction timeline. A selection of the papers this survey draws on, by year, from interactive proofs and zero knowledge (1985) through PCP-based arguments, the pairing/QAP era, the 2019 wave of universal and transparent SNARKs, and recursion, folding, lookups and linear-time provers, to a 2024–2026 band of recent work. Years are preprint years where these differ from the venue year (except that the CRYPTO 2026 papers are grouped under 2026); the venue is then given in brackets, and some same-year venues are bracketed too (EC = EUROCRYPT, AC = ASIACRYPT). Attacks and disproofs are included because §§5.3, 6.3 and 9.4 rely on them.

Table A.1: **Setup, assumptions and the costs as each paper reports them.** It gives the exact counts behind Tables 7.1 and A.2, in each paper's own units and symbols. E and P count exponentiations and pairings (with a subscript for the group), PPE counts pairing-product equations, F mul counts field multiplications, and MUL (Supersonic row) counts scalar multiplications of λ-bit numbers in the group of unknown order; unless a row says otherwise, m is the number of wires, n of multiplication gates, a of addition gates, ℓ of public inputs, d the circuit depth and N the number of constraints (ι: Hyrax's commitment parameter; $\mathbb{G}_U$: the group of unknown order; for Samaritan, $\mu = \log m$ and K the number of nonzero R1CS entries). Some rows keep their paper's own symbols without restating them: d is the QSP or QAP degree for GGPR and Pinocchio and the polynomial degree for Supersonic (with μ variables, and n its arithmetic complexity, the total gate count); Marlin's m counts all gates, Fractal's m and Aurora's N the nonzero entries of the R1CS matrices; Spartan's n is the size of the statement; HyperNova's m, n and N count CCS constraints, witness variables and nonzero matrix entries, and its d, t and q are the constraint degree and the numbers of CCS matrices and monomials. Spartan's single row covers all variants (DL in Table A.2; KE and RO in Table 7.1) and is placed with the strict rows.

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Strict zk-SNARKs (Table 7.1 rows)								
GGPR	QSP: 7 $\mathbb{G}$; QAP: 9 $\mathbb{G}$ (symmetric)	quasi-linear (linear group ops)	linear in statement size (computing $E(v_{in}(s))$), otherwise constant; constant apart from hashing if the statement is hashed	per-circuit, CRS linear in circuit; two stages: generic powers-of-s Gen, then per-function Regen_f (§3.1)	q-PDH + d-PKE, $q \geq \max\{2d-1, d+2\}$ (Thms. 7, 14; §5.1 calls it q-PKE; d-PKE non-falsifiable)	no	QSP/QAP / pairing	[GGPR13, abstract, §1.2, §3.1, §3.3, §5.1, §8]; [PHGR13, p. 4]
Pinocchio	8 $\mathbb{G}$ (symmetric); 288 B as implemented (asymmetric BN-256)	$7m + n - 2\ell$ E (Groth16's count; derived from Protocol 2: $7(m-N)+d$, N = number of I/O elements)	ℓ E (Groth16's count; derived from Protocol 2: about $3N$), 11P, 5 PPE	per-circuit	d-PKE, q-PDH, 2q-SDH ($q = 4d+4$)	no	QAP / pairing	[Gro16, Table 2] (symmetric); [PHGR13, Thm. 1, Protocol 2, Fig. 8]
Groth16	3 elts ($2\,\mathbb{G}_1 + 1\,\mathbb{G}_2$)	$m + 3n - \ell$ E_1, n E_2	ℓ E_1, 3P, 1 PPE	per-circuit	generic bilinear group model	no	QAP / pairing	[Gro16, Table 2] (asymmetric)
Sonic	succinct: $20\,\mathbb{G}_1 + 16\,\mathbb{F}$; helped: $4\,\mathbb{G}_1 + 2\,\mathbb{F}$ (PLONK) or $7\,\mathbb{G} + 5\,\mathbb{F}$ (Sonic Table 2)	succinct $273n$ $\mathbb{G}_1$ E; helped $18n$	13P (10P helped), counted unbatched as in Sonic Table 2; 7P with standard pairing batching (Marlin Fig. 1)	universal, updatable, linear SRS	AGM + ROM (interactive, made non-interactive by Fiat–Shamir in the ROM, §3 p. 4)	no	own constraint system / KZG variant (univariate Laurent, §6.2)	[GWC19, Tables 1–2]; [MBKM19, Table 2, §1, §3, §6.2]; [CHM+20, Fig. 1] (batched count)

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
PLONK	$9\,\mathbb{G}_1 + 6\,\mathbb{F}$ (fast prover)	$9n + 9a$ $\mathbb{G}_1$ E $+ \approx 54(n+a)\log(n+a)$ F mul	2P, 18 $\mathbb{G}_1$ E (fast prover; 16 for the small variant)	universal and updatable (PLONK's "universal" includes updatability, §1); SRS degree = #gates	original: AGM + q-DLOG (knowledge soundness under 2d-DLOG), ROM for Fiat–Shamir; alternatively, knowledge-sound in the ROM under the falsifiable ARSDH + SplitRSDH (Lipmaa–Parisella–Siim, CRYPTO 2025, via special soundness of interactive PLONK; re-derived modularly (sketch) by Chiesa et al., CRYPTO 2026, Cor. 1)	no	Plonkish / KZG	[GWC19, Tables 1–2, §2.2, Lemma 4.7, §8]; [LPS25]; [CGKY26]
Marlin	$13\,\mathbb{G}_1 + 8\,\mathbb{F}$	22 v-MSM(m) (variable-base multi-scalar multiplications, MSMs, of size m), $O(m \log m)$	2P, $O(\lvert x\rvert + \log m)$	universal, updatable (Remark 7.1)	AGM (fallback: knowledge assumptions) + ROM (Fiat–Shamir, §1.1, §7)	no	R1CS / AHP + KZG	[CHM+20, Fig. 1, §7]
Fractal	$O_\lambda(\log^2 m)$	$O_\lambda(m \log m)$	$O_\lambda(\lvert x\rvert + \log^2 m)$ after preprocessing	transparent	ROM	plausibly (security proven in the QROM, Thm. 3)	R1CS / holographic IOP	[COS20, Thm. 3]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Spartan	$O(\log^2 n)$ to $O(\sqrt{n})$ by PCS	$O(n)$, time-optimal via Spark (DL, KE); $O(n \log n)$ (RO, CL; Fig. 2)	$O(\log^2 n)$ to $O(\sqrt{n})$ by PCS	transparent (DL, RO, and CL, which uses DARK's commitment scheme after an adaptation the paper does not give, Fig. 2 caption; §5.1, p. 21); universal trusted setup for KE ("private⋆", Fig. 2)	DL + ROM ($\text{Spartan}_{\text{DL}}$, implemented, §8); q-type knowledge-of-exponent PCS + ROM ($\text{Spartan}_{\text{KE}}$, §1; Cor. 1.1); ROM only ($\text{Spartan}_{\text{RO}}$)	no (DL, KE, CL); plausibly (RO, Fig. 2 caption)	R1CS / sum-check + Spark	[Set20, abstract, §1, Fig. 2, Thm. 5.1, p. 18, Fig. 5, p. 21, Lemma 7.6, Fig. 6, p. 30, §8]
Supersonic (DARK)	$O(\mu \log d)$ (Table 4: $2 \log n$ $\mathbb{G}_U$)	$O(n \log n)$ MUL (quasi-linear)	$O(\mu \log d)$ (Table 4: $3 \log n$ MUL)	transparent with class groups (trusted with RSA)	strong RSA + adaptive root (Thm. 5); revised 2022 proof: random order + adaptive root; zero knowledge only in the 2020 version (this survey's §5.3)	no	Sonic / PLONK / Marlin polynomial IOPs (10.1 KB headline = PLONK) / DARK PCS	[BFS20, §1.1, §3.1, Thm. 5, §6, Table 4]
Dew	77 group elements with PLONK (Cor. 1), 87 with Marlin (Cor. 2), plus $O(1)$ F (p. 55)	$O(n^2)$ (Cors. 1–2); $\tilde{O}(n)$ after preprocessing (Fig. 14)	$O(\log n)$ (Cors. 1–2)	transparent: class groups; $\lvert pp \rvert = O(1)$ (Fig. 14)	generic group model (Cors. 1–2); ROM for Fiat–Shamir (§7.3); hiding statistical under a condition on the generator that may fail transparently, else computational under subgroup indistinguishability (Thms. 7.3–7.4); honest-verifier zero-knowledge evaluation (Thm. 7.5), zero knowledge via the [CHM+20] compiler (Thm. 7.7)	no	PLONK / Marlin AHP / Dew PCS	[AGL+22, §7.1, pp. 50–52, §7.3, pp. 53–54, Cors. 1–2, pp. 54–55, Fig. 14, p. 55]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
HyperPlonk	$\mu + 5\ \mathbb{G}_1 + 4\mu + 29\ \mathbb{F}$ for $n = 2^\mu$ degree-2 gates (multilinear KZG, optimized as in App. C); $176\mu + 1168$ bytes (App. C; §6.2 quotes it for the implementation, which does not yet commit to the sum-check round polynomials and so is slightly larger)	PIOP $O(nd \log^2 d)$ F (Thm. 4.1); KZG commitment + evaluation $n\ \mathbb{G}_1$ (Table 2); $O(n + d \log n)$ exponentiations for n gates of total degree d (§1, p. 5, which writes s for the gate count)	PIOP $O(\mu + \ell)$, ℓ = public-input length (Thm. 4.1); KZG opening $\log(n)$ P (Table 2); batching k evaluations $O(k\mu)$ (§3.8)	universal with multilinear KZG or Orion+; transparent with the FRI-based PCS (Table 2)	those of the multilinear PCS, through PIOP compilation with witness-extended emulation (Thm. 2.4), not restated; ROM for Fiat–Shamir (§2.3); zero knowledge needs an additive, spanning PCS (Cor. A.4)	no (KZG); "quantum-resistant" with the FRI-based PCS (§1, p. 5)	Plonkish on the Boolean hypercube / multilinear KZG (also Orion+ or FRI-based)	[CBBZ23, §1, §2.3, Table 2, Thm. 2.4, §3.8, Thm. 4.1, §6.2, Cor. A.4]
Polymath	$3\,\mathbb{G}_1 + 1\,\mathbb{F}$	$(\tilde{m} + 13\tilde{n})\ \mathbb{G}_1$ scalar mult. (SAP, $\tilde{m} \approx 2m$, $\tilde{n} \approx 2n$)	2P + 2 $\mathbb{G}_1$ + 1 $\mathbb{G}_2$ scalar mult. + $O(m_0 \log m_0)$ F, m_0 = input length	per-circuit SRS: $(\tilde{m} + 12\tilde{n})\ \mathbb{G}_1$ (Table 1)	special soundness of the interactive argument in the AGM with oblivious sampling, under the power-discrete-log (PDL) and Tensor Oracle FindRep (TOFR) assumptions (Thm. 3); knowledge soundness after Fiat–Shamir via Attema et al. (§4.7); perfect zero knowledge (Thm. 2)	no	SAP / KZG	[Lip24, Table 1, p. 175, Thm. 2, p. 188, Thm. 3, p. 193, §4.7, p. 199]
Samaritan	$(11 \log m + 8)\,\mathbb{F} + 25\,\mathbb{G}_1$, 8.3 KB at $\mu = 20$ on BLS12-381, $m = \max(n, K)$ (Thm. 5.1); 6.2 KB with the §5.3 PIOP (abstract)	$10m$ v-MSM + $7m$ short v-MSM + $O(n)$ F (Thm. 5.1)	$O(\log m)$ F + $O(1)$ $\mathbb{G}_1$ + $O(1)$ P (Thm. 5.1)	universal, updatable KZG SRS (abstract; §1, p. 4)	q-DLOG in the AGM (Thm. 4.2); ROM for Fiat–Shamir (§3.4); zero knowledge derived from cited compilers by Lemma 5.3, not proven (§5.4)	no	R1CS / multilinear PCS built from KZG	[GPS25, abstract, §1, p. 4, §3.4, p. 17, Thm. 4.2, p. 29, Thm. 5.1, p. 36, §5.4, p. 37]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Garuda	$O(\log n)$; 5–8 kB in benchmarks (§8.1.2)	$O(n)$ F + $O(n)$ group ops (§1)	$O(\log n)$ (§1); about 3 ms (§8.1.2)	circuit-specific trusted setup (Remark 1.1)	bounds stated in an informal theorem (Thm. 1, p. 5798); interactive version proven in the AGM, with PST and the Kiltz–Wee linear-subspace SNARK (§1; §5, p. 5798); Fiat–Shamir in the ROM (§4, p. 5797); zero knowledge specified from PLONK-style random constraints, a hiding commitment to a zero polynomial and sum-check masking as in Libra, not proven (§5, p. 5798)	no	GR1CS / PST + linear-subspace SNARK	[DMS26, §1, p. 5792, Remark 1.1, p. 5792, §4, p. 5797, §5, p. 5798, §8.1.2, p. 5802]
Outside the strict definition (Table A.2 rows)								
STARK	$O(\log^2\|C\|)$	$O(\|C\|\log^2\|C\|)$	$O(\log^2\|C\|)$ for uniform computations (AIR; DARK §1.1); $O(\|C\|)$ for an arbitrary circuit (Aurora Fig. 1)	transparent	collision-resistant hashing (interactive protocol); ROM once Fiat–Shamir'd	plausibly (informal post-quantum claim; its hash-only basis covers the interactive setting only, §1.3.1 fn. 12)	AIR / FRI	[ZXZS20, Table 2]; [BBHR18, §2.2, Thm. 3.5, §1.3.1]; [BFS20, §1.1]; [BCR+19, Fig. 1]
Libra	$O(d\log\|C\|)$ for d layers	$O(\|C\|)$ (any circuit)	$O(\|x\| + d\log\|C\|)$ for d-depth log-space uniform C	one-time, input-size-only trusted setup	q-SBDH + (d,ℓ)-extended PKE (pairing-based zkVPD); ROM for non-interactivity	no	layered circuits (GKR) / zkVPD	[XZZ+19, abstract, §2, Thms. 5–6, §4.4, Thm. 6, p. 26]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Virgo	$O(d \log \lvert C\rvert + \log^2 n)$ (depth d, n inputs)	$O(\lvert C\rvert + n \log n)$	$O(\lvert x\rvert + d \log \lvert C\rvert + \log^2 n)$ (regular C)	transparent	hashing, with the Merkle hash as a random oracle for knowledge soundness (p. 17); "100+ bits" rests on FRI's unproven up-to-capacity soundness conjecture (its §5.2, p. 23; discussed in this survey's §5.3); the up-to-capacity correlated-agreement conjecture was disproved (Crites–Stewart, CRYPTO 2026 [CS26]), which closes one route to proving it (§5.3), and no polynomial-time attack on FRI is known	plausibly	layered circuits (GKR) / FRI-based zkVPD	[ZXZS20, Thm. 3, Table 2]
zk-Pari	$2\,\mathbb{G}_1 + 2\,\mathbb{F}$, 1280 bits on BLS12-381 (§1.1)	Pari's prover plus one MSM of size four (§1.1); Pari: $O(n \log n)$ F + $O(n)$ group ops [DMS26, §1, p. 5792]	3P + $O(\lvert x\rvert)$ F (Thm. 4)	per-circuit; proving key 13 $\mathbb{G}_1$ (624 B) more than Pari's (Table 1)	AGM with random oracles under SDH (Thm. 3), for circuits that satisfy Witness-Column Independence (Assumption 2); perfect zero knowledge in the ROM (Thm. 2)	no	Square R1CS / KZG with an equifficient commitment	[KPS26, §1.1, p. 2, Assumption 2, p. 7, Thm. 2, p. 15, Thm. 3, p. 21, Thm. 4, p. 22, Table 1, p. 23]; [DMS26, §1, p. 5792]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Hyrax	$\approx 10d \log G + \lvert w\rvert^{1/\iota}$ $\mathbb{G}$	linear in $\lvert C\rvert$ if C has sufficient (data) parallelism; $O(d \log G + \lvert w\rvert)$ crypto ops	$O(\lvert x\rvert + d \log G + \lvert w\rvert^{(\iota-1)/\iota})$ with sufficient parallelism (sub-linear without it only for regular wiring, fn. 2); with a redistribution layer (RDL, §6.2) V pays an additional $O(\lvert x\rvert + \lvert w\rvert + NG)$ for N parallel sub-computations of width G, which dominates V on the Merkle-tree and image-scaling benchmarks (§8)	transparent	discrete log + ROM	no	layered circuits (GKR) / Hyrax-commit	[WTS+18, §1, §6.2, Thm. 6, §8]
Aurora	$O(\log^2 N)$	$O(N \log N)$	$O(N)$	transparent	unconditional in ROM	plausibly	R1CS / univariate sum-check + FRI	[BCR+19, Thm. 1.2, Thm. 9.2, p. 40]
Ligero	$\tilde{O}(\sqrt{\lvert C\rvert})$	$O(\lvert C\rvert \log \lvert C\rvert)$	$O(\lvert C\rvert \log \lvert C\rvert)$	transparent	collision-resistant hashing (interactive protocol); ROM once Fiat–Shamir'd (abstract, §1)	plausibly	arith. circuits / interleaved RS (IPCP)	[AHIV17, Thm. 1.1, §6, p. 31]; [ZXZS20, §1.2, p. 3, Table 2]
Bulletproofs	IPA: $2\lceil \log_2 n \rceil$ $\mathbb{G}$ + 2 $\mathbb{Z}_p$; circuit proof $2 \log_2 n + 13$ elements (§5)	IPA: $\approx 8n$ E	IPA: $\approx 4n$ E (one MSM of size $2n + 2 \log n$), linear	transparent	discrete log + ROM (Fiat–Shamir, §4.4)	no	arith. circuits / IPA	[BBB+17, §3, §4.4, §5]
LaBRADOR	58 KB for 2^{20} R1CS constraints mod $2^{64}+1$ (abstract); 47–58 KB from 2^{10} to 2^{20} (Table 1); binary R1CS 49.02 KB at 2^{20} (Table 2); zero-knowledge variant 60.33 KB (§6, p. 540)	not stated; $O(\log \log n)$ iterations, "6 or 7" in practice (§1.1, p. 522)	not stated; not sublinear (§1, p. 520)	CRS of commitment matrices (Fig. 2, p. 534); size not stated	Module-SIS (Thm. 5.1, p. 537); Fiat–Shamir (§5.7, p. 538); zero knowledge: a masking shim with Module-LWE-blinded commitments, costed but not proven (§6)	plausibly	R1CS reduced to dot-product constraints over $\mathbb{Z}_q[X]/(X^{64}+1)$ / Ajtai commitments, recursive	[BS23b, abstract, §1, pp. 519–522, Fig. 2, p. 534, Thm. 5.1, p. 537, §6, pp. 539–541, Tables 1–2, p. 546]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Orion	$O(\log^2 N)$	$O(N)$ field operations and hashes	$O(N)$ (Thm. 5); $O(\log^2 N)$ claimed in the preprocessing setting (Table 1 note; §1, pp. 3–4); the revision cited is unsound, and its commitment's verifier is $O(\sqrt{N})$ [DS25, abstract, p. 5]	transparent (no trusted setup, §1, p. 3)	collision-resistant hash, hiding for zero knowledge (§2.2); made non-interactive by the Fiat–Shamir heuristic (p. 23)	plausibly ("plausible post-quantum security", p. 23)	R1CS / sum-check + Orion-PC with code switching (Virgo's IOP as the inner argument, p. 23)	[XZS22, abstract, Table 1, §1, §2.2, Thm. 5, p. 23]
Plonky2	no asymptotic bound; about 43 KB at codeword rate 1/256 (§8, p. 14)	no asymptotic bound; about 300 ms per recursive proof at rate 1/8 (recursion threshold 2^{12} gates) and about 11.6 s for the 43 KB setting, on a 2021 MacBook Air (§8, p. 14)	not stated	transparent; preprocessing commits to the constant and permutation polynomials in one Merkle tree (§7.1)	none proven: 100 bits of "conjectured" security, from ethSTARK's soundness conjecture (§8, fn. 8); soundness after Fiat–Shamir not analysed (§3.4.3, fn. 3); Poseidon sponge (§4)	not discussed (hash-based)	TurboPLONK-style Plonkish with custom gates / FRI over Goldilocks, evaluation-point and FRI challenges from $\mathbb{F}_p[X]/(X^2-7)$, permutation and combining challenges from $\mathbb{F}_p$ repeated r times (§7.2–7.3)	[Pol22, abstract, §2.1, §3.4.3, §4, §7.1–7.3, §8]
Nova	IVC: $O(\lvert F\rvert)$ field elements (a relaxed-R1CS witness) plus $O(1)$ commitments; compressed $O(\log\lvert F\rvert)$	per step: 2 MSMs of size $O(\lvert F\rvert)$	$O_\lambda(\lvert F\rvert)$; compressed $O(\log\lvert F\rvert)$ or $O(\lvert F\rvert)$	transparent	additively homomorphic commitment (e.g. Pedersen / DLOG), curve cycle; ROM instantiated by a concrete hash (heuristic, fn. 1, §4.2)	no	relaxed R1CS / folding	[KST22, §1.2, Thms. 1–2]
HyperNova	NIVC proof linear in the circuit of each supported function (§8, p. 35); compressed by a non-zero-knowledge SNARK of the user's choice (§8, p. 37)	per step: one MSM of size $O(n+n_j)$ + $O((N+N_j)+(t+t_j)(m+m_j)+(q+q_j)(m+m_j)\,d\log^2 d)$ F (Thm. 3)	folding verifier $O(d\log m)$ F and $O(1)$ group operations (Thm. 1); verifier circuit with one $\mathbb{G}$ scalar multiplication (Thm. 3)	Pedersen key, as for Nova	Pedersen commitments (discrete log, App. E); ROM for Fiat–Shamir, heuristically instantiated with a concrete hash (§1.3, p. 9); zero knowledge honest-verifier, then the Fiat–Shamir heuristic (§8, p. 37)	no	CCS / multi-folding via sum-check	[KS24, §1.3, Thms. 1, 3, §8, App. E]

Table A.2: **Zero-knowledge arguments outside the strict definition (costs in $|C|$).** "Conditional" rows meet the definition only for a restricted class of circuits: the succinctness bound, or, for zk-Pari, the knowledge-soundness proof. Notation as in Table 7.1.

Scheme	Why outside the strict definition	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
STARK	conditional: polylog verifier only for uniform computations (AIR); linear for an arbitrary circuit	$O_\lambda(1)$	$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|\log^2\|C\|)$	$O_\lambda(\log^2\|C\|)$ or $O_\lambda(\|C\|)$	$\log^2$ verifier for uniform (AIR) computations, linear for an arbitrary circuit; hash only	[BBHR18, §2.2, Thm. 3.5]; [ZXZS20, Table 2]; [BCR+19, Fig. 1]
Libra	conditional: proof and verifier grow with depth d; polylog only for low-depth, log-space uniform circuits	$O_\lambda(n)$	$O_\lambda(d\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\| + d\log\|C\|)$	n = input size; the verifier bound needs log-space uniformity	[XZZ+19, Table 1, §4, Thm. 6]
Virgo	conditional: as Libra (depth d; regular circuits)	$O_\lambda(1)$	$O_\lambda(d\log\|C\| + \log^2 n)$	$O_\lambda(\|C\| + n\log n)$	$O_\lambda(\|x\| + d\log\|C\| + \log^2 n)$	n = input size ($\|x\| + \|w\|$); regular circuits (wiring predicates' multilinear extensions evaluable in $O(\log\|C\|)$ time); transparent, hash only	[ZXZS20, Thm. 3, Table 2]
zk-Pari	conditional: knowledge soundness proven only for circuits that satisfy Witness-Column Independence	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|x\|)$	Pari (Square R1CS) with proven perfect zero knowledge; same proof and verifier; setup and prover **derived** from Pari's; not implemented	[KPS26, §1.1, p. 2, Assumption 2, p. 7, Thm. 4, p. 22]; [DMS26, §1, p. 5792]
Hyrax	proof and verifier sublinear, not polylog ($\|w\|^{1/\iota}$, $\|w\|^{(\iota-1)/\iota}$ terms); grow with depth d	$O_\lambda(\|w\|^{(\iota-1)/\iota})$	$O_\lambda(d\log G + \|w\|^{1/\iota})$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\| + d\log G + \|w\|^{(\iota-1)/\iota})$	prover linear and verifier sublinear only with data parallelism (without it, the verifier is sublinear only for regular wiring); setup parameters **derived** from Lemma 5	[WTS+18, §1, §6, Lemma 5, p. 9]
$\text{Spartan}_{\text{DL}}$	proof and verifier $O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\| + \sqrt{\|C\|})$	setup parameters **derived**: Spartan's discrete-log commitment option is Hyrax-PC, whose key is about $\sqrt{\|C\|}$ random group elements	[Set20, Fig. 2, Fig. 4, p. 21]
Aurora	verifier linear in $\|C\|$ (proof is polylog)	$O_\lambda(1)$	$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|C\|)$	random oracle only	[BCR+19, Thm. 1.2, Thm. 9.2, p. 40]
Ligero	proof $\tilde{O}(\sqrt{\|C\|})$ and verifier quasi-linear	$O_\lambda(1)$	$\tilde{O}_\lambda(\sqrt{\|C\|})$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|C\|\log\|C\|)$	proof $\tilde{O}(\sqrt{\|C\|})$; hash only	[AHIV17, Thm. 1.1, §6, p. 31]; [ZXZS20, §1.2, p. 3, Table 2]

continued on next page

Table A.2, continued

Scheme	Why outside the strict definition	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
Bulletproofs	verifier linear in $\|C\|$ (proof is logarithmic)	$O_\lambda(\|C\|)$	$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|C\|)$	transparent generators; short proofs, linear verifier	[BBB+17, §3, §5]
LaBRADOR	verification time not sublinear (§1, p. 520)				not sublinear	Module-SIS, plausibly PQ; 47–58 KB proofs for 2^{10} to 2^{20} constraints; main construction not zero-knowledge; zero knowledge from the paper's own masking shim, costed (60.33 KB against 49.02 KB) but not proven (§6)	[BS23b, abstract, §1, pp. 519–520, §6, pp. 539–541, Tables 1–2, p. 546]
Orion	the revision cited is unsound: an implemented attack opens any commitment to any value [DS25, abstract, §3, pp. 10–15]; besides, its Thm. 5 verifier, stated without proof, is linear, and the $O_\lambda(\log^2\|C\|)$ verifier is claimed only in the preprocessing setting, with no theorem (this survey's §9.1)		$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|C\|)$; claimed $O_\lambda(\log^2\|C\|)$ after preprocessing	no trusted setup, but a randomly sampled expander-based code whose description size is not stated	[XZS22, abstract, Table 1, §1, Thm. 5, p. 23]
Plonky2	deprecated since July 2025 (repository archived in 2026) in favour of Plonky3 [Plonky2]; a draft whitepaper (7 September 2022) with no asymptotic bounds and no full security proof; its 100 bits are "conjectured" and FRI's zero knowledge is marked "TODO"	$O_\lambda(1)$				PLONK-style arithmetization with FRI; setup parameters **derived**: hash only	[Pol22, p. 1, abstract, §3.4, §3.4.3 fn. 3, §3.6 fn. 6, §7.1, §8 fn. 8]
Nova (per step)	an IVC / folding scheme, not itself a zk-SNARK; per step, only the compressed proof in the SXDH variant meets the bound	$O_\lambda(\|F\|)$	IVC $O_\lambda(\|F\|)$; compressed $O_\lambda(\log\|F\|)$	$O_\lambda(\|F\|)$	IVC $O_\lambda(\|F\|)$; compressed $O_\lambda(\|F\|)$ (DLOG) or $O_\lambda(\log\|F\|)$ (SXDH)	$\|F\|$ **(step circuit) plays the role of** $\|C\|$; setup parameters **derived**: a Pedersen key for vectors of length about $\|F\|$	[KST22, §1, Fig. 2, §2, Thms. 1–2]

continued on next page

Table A.2, continued

Scheme	**Why outside the strict definition**	**Setup params (beyond C)**	**Proof size**	**Prover**	**Verifier**	**Notes**	**Source**
HyperNova (per step)	a non-uniform IVC / folding scheme, not itself a zk-SNARK; its NIVC proof is linear in the step circuits, and succinctness needs a separate SNARK that the paper does not fix	$O_\lambda(\|F\|)$	NIVC $O_\lambda(\|F\|)$; compressed: that of the chosen SNARK	$O_\lambda(\|F\|)$	NIVC $O_\lambda(\|F\|)$	$\|F\|$ **(step circuit) plays the role of** $\|C\|$; the NIVC proof is linear in each supported function's circuit; setup parameters **derived**: a Pedersen key	[KS24, abstract, Constr. 1, Thm. 3, §8]